

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN

BÀI BÁO CÁO
MÔN HỌC: KIỂM THỬ PHẦN MỀM

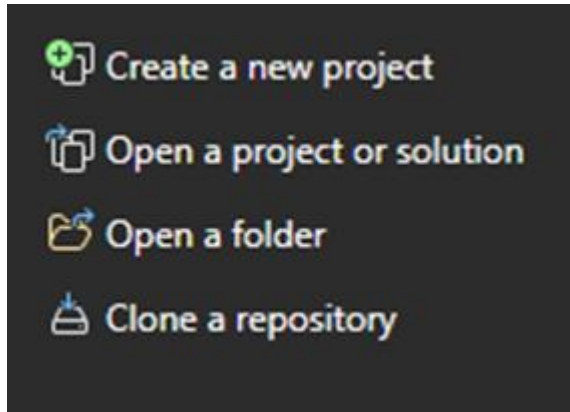
Giảng viên hướng dẫn: **ThS.Nguyễn Thị Ngọc Thanh**
Sinh viên thực hiện: **Nguyễn Thị Anh Thư - 2354050131**

Phần 1: Unit Test

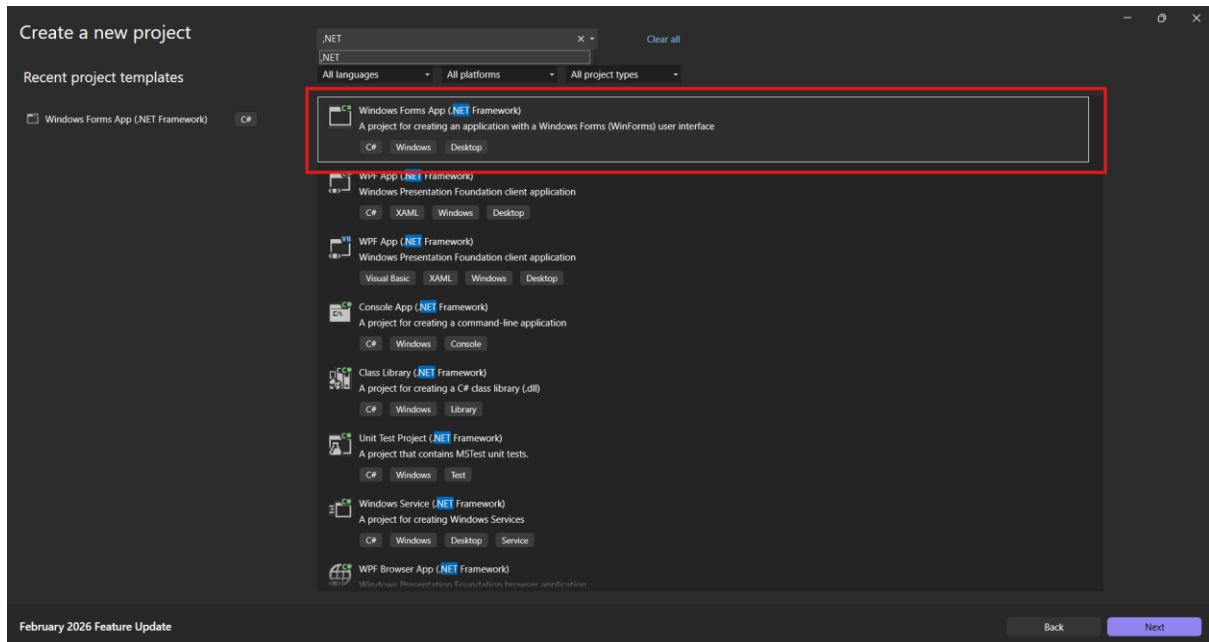
Hình Hộp Chữ Nhật

1. Tạo Project

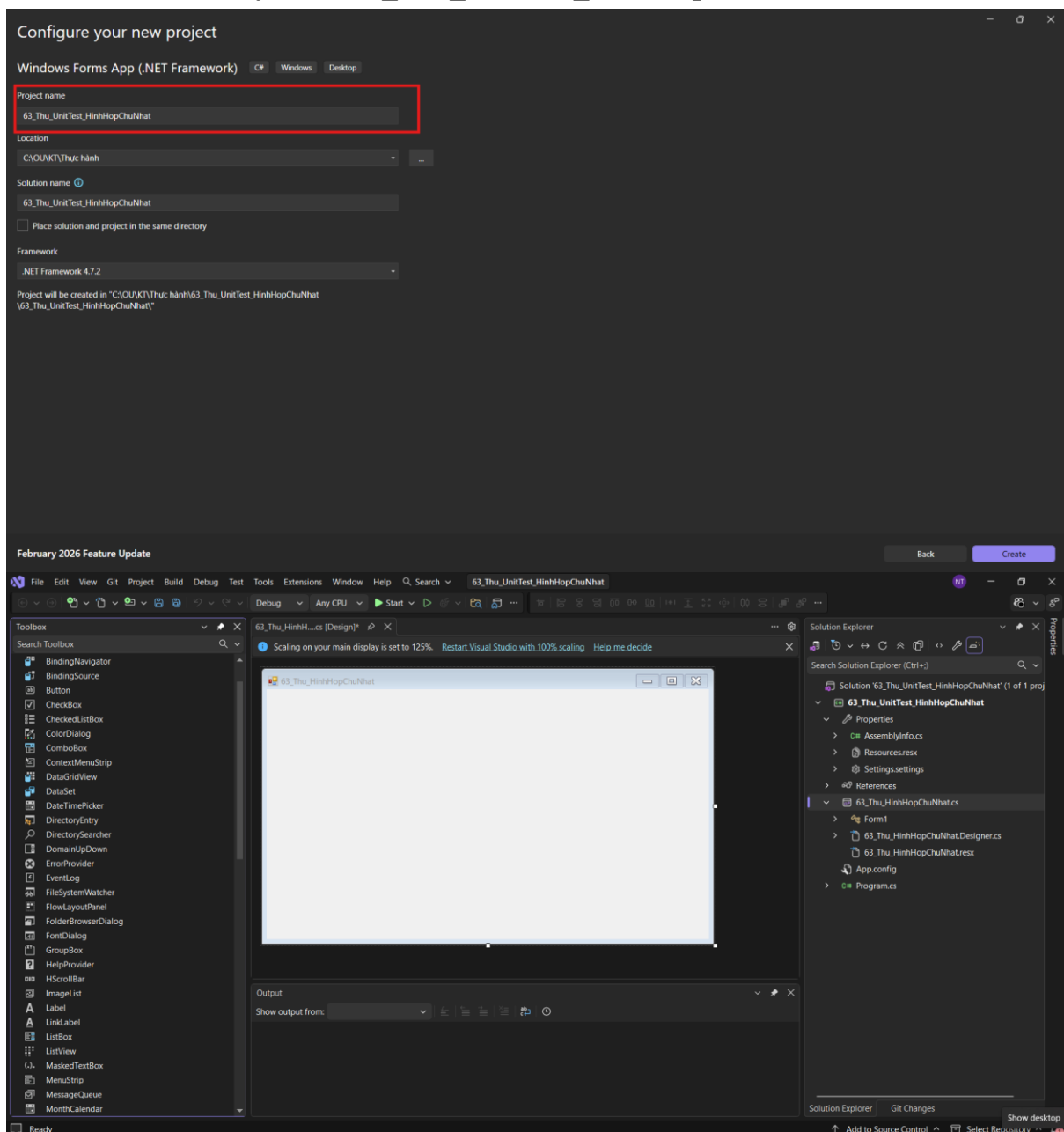
Bước 1: Vào Visual Studio => Create a new project



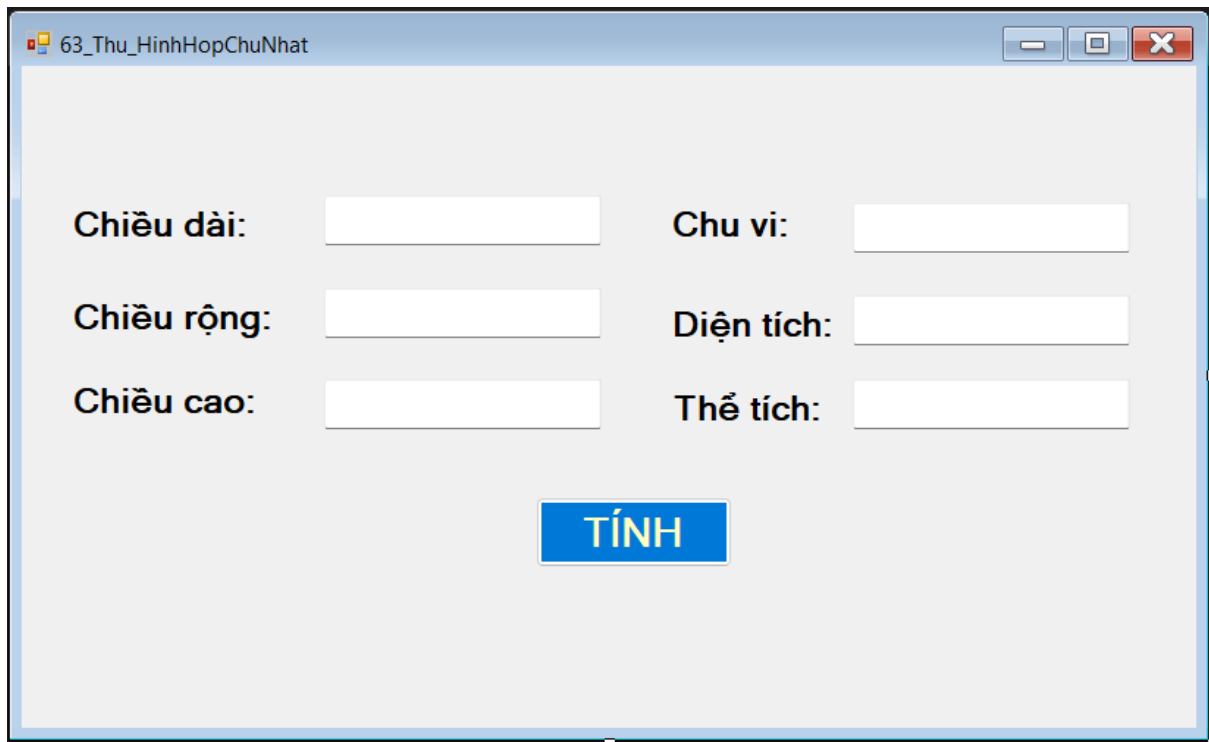
Bước 2: Tìm kiếm “.net” => Windows Forms App (.NET Framework) => Next



Bước 3: Đặt tên Project là: 63_Thu_UnitTest_HinhHopChuNhat

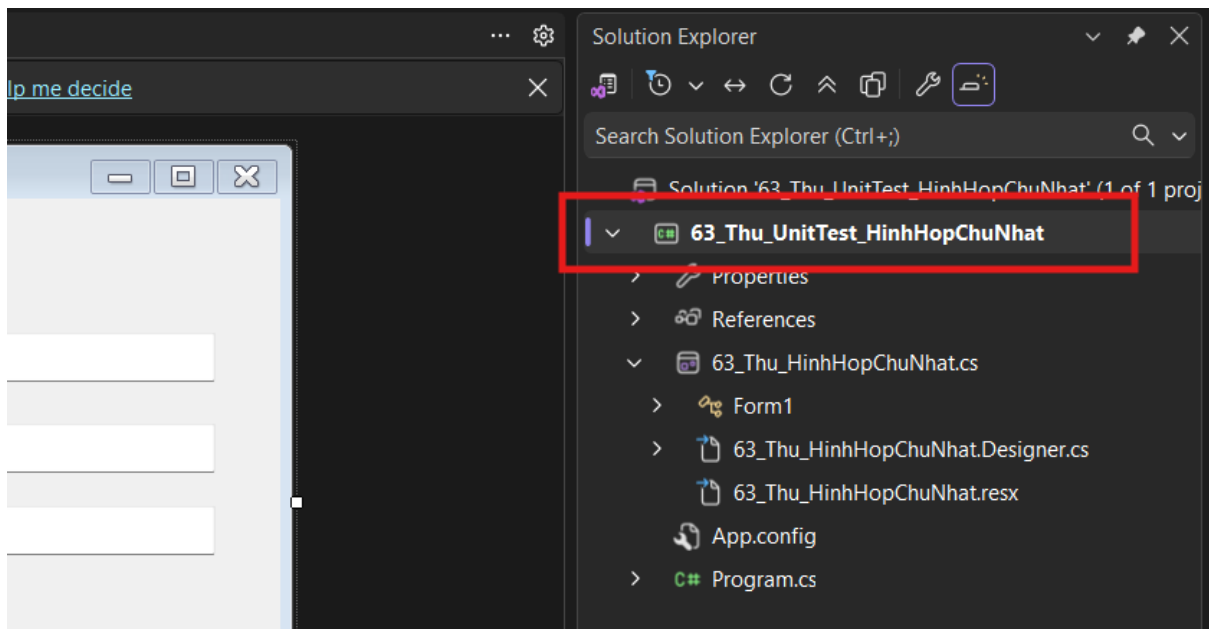


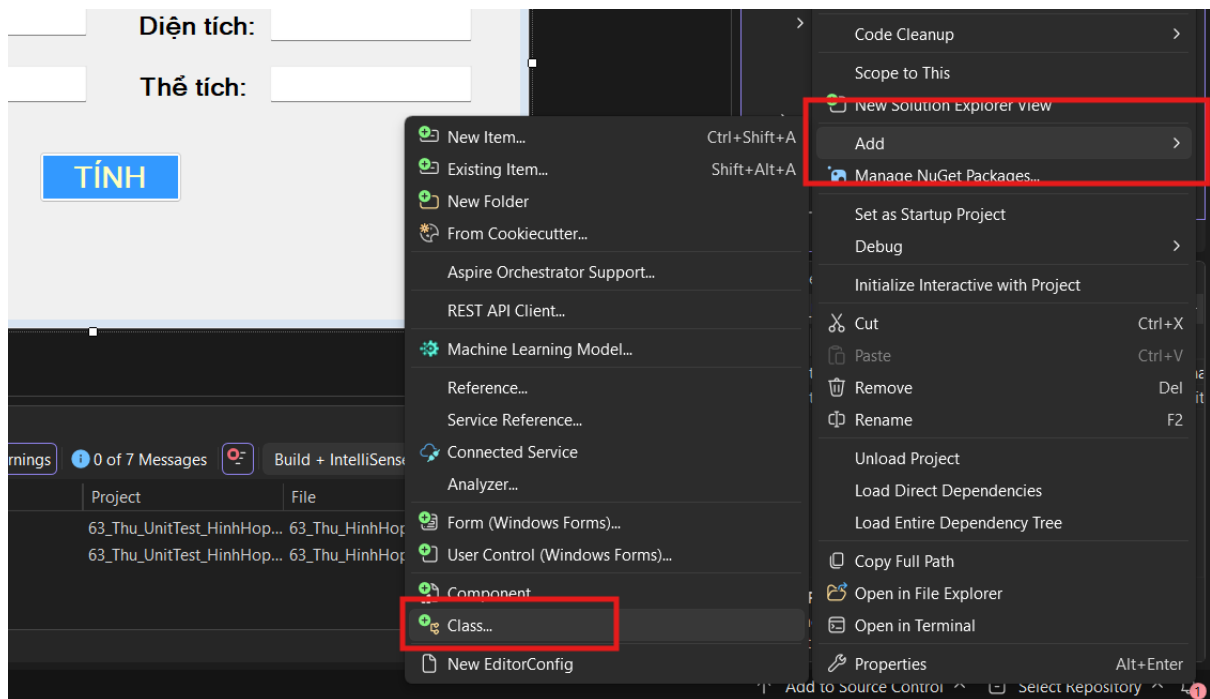
Bước 4: Tạo giao diện cho form



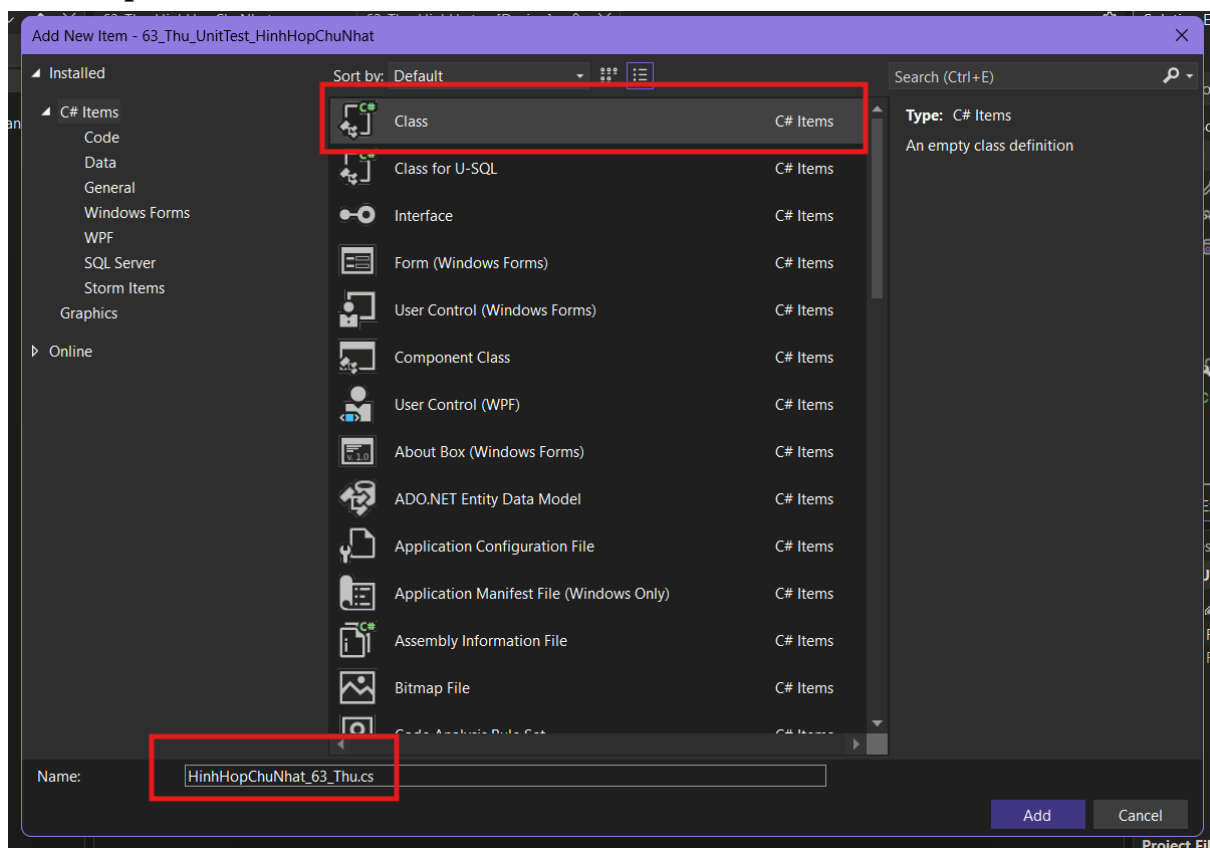
The screenshot shows a Windows application window titled "63_Thu_HinhHopChuNhat". The window contains a form with six input fields arranged in two columns. The left column has labels "Chiều dài:" (Length), "Chiều rộng:" (Width), and "Chiều cao:" (Height). The right column has labels "Chu vi:" (Perimeter), "Diện tích:" (Area), and "Thể tích:" (Volume). Below the input fields is a blue button with the text "TÍNH" (Calculate).

Bước 5: Click chuột phải vào 63_Thu_UnitTest_HinhHopChuNhat -> Add -> Class

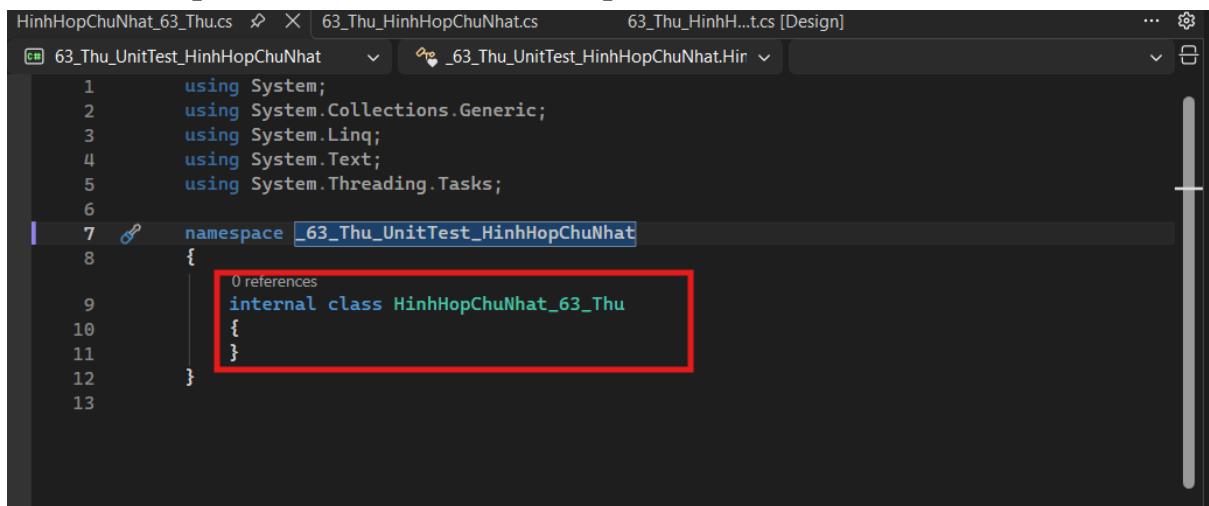




Bước 6: Tạo Class có phương thức tính toán cho đối tượng Hình hộp Chữ nhật mang tên `HinhHopChuNhat_63_Thu.cs`



Bước 7: Sửa phạm vi class từ internal => public

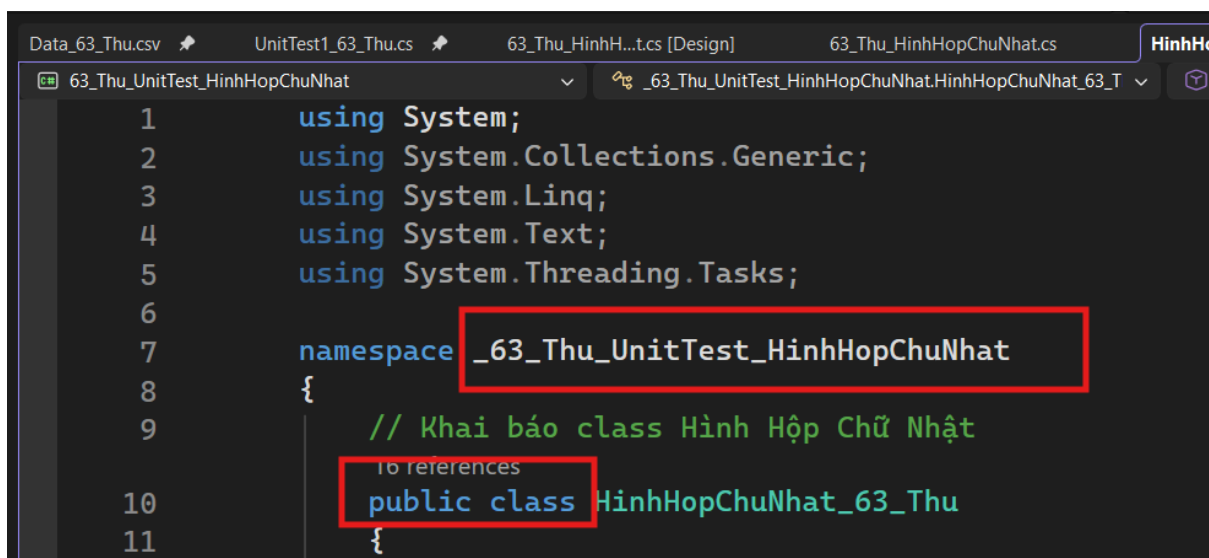


```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace _63_Thu_UnitTest_HìnhHộpChuNhat
8 {
9     0 references
10     internal class HìnhHộpChuNhat_63_Thu
11     {
12     }
13 }
```

Ở đây Project mang tên “63_Thu_UnitTest_HìnhHộpChuNhat” nhưng trong giao diện UnitTest hiển thị “namespace_63_Thu_UnitTest_HìnhHộpChuNhat” là do Tên project bắt đầu bằng:

63

đây là số, nên khi Visual Studio tự sinh namespace, nó sẽ tự thêm dấu gạch dưới _ vào đầu để namespace hợp lệ theo quy tắc ngôn ngữ C#.



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace _63_Thu_UnitTest_HìnhHộpChuNhat
8 {
9     // Khai báo class Hình Hộp Chữ Nhật
10     10 references
11     public class HìnhHộpChuNhat_63_Thu
12     {
13     }
14 }
```

Bước 8: Viết phương thức tính toán Hình Hộp Chữ nhật

```
namespace _63_Thu_UnitTest_HìnhHộpChuNhat
{
    // Khai báo class Hình Hộp Chữ Nhật
    public class HìnhHộpChuNhat_63_Thu
    {
```

```

// Khai báo 3 biến private để lưu chiều dài, rộng, cao
private double dai_63_Thu, rong_63_Thu, cao_63_Thu;
// Constructor (hàm khởi tạo) nhận vào 3 tham số
public HìnhHopChuNhat_63_Thu(double dai_63_Thu, double rong_63_Thu,
double cao_63_Thu)
{
    // Kiểm tra dữ liệu đầu vào: nếu bất kỳ giá trị nào <= 0 thì lỗi
    if (dai_63_Thu <= 0 || rong_63_Thu <= 0 || cao_63_Thu <= 0)
    {
        // Ném ra ngoại lệ (Exception) để báo lỗi
        throw new ArgumentOutOfRangeException();
    }
    // Gán giá trị từ tham số vào biến của class
    this.dai_63_Thu = dai_63_Thu;
    this.rong_63_Thu = rong_63_Thu;
    this.cao_63_Thu = cao_63_Thu;
}
// Phương thức tính diện tích toàn phần
public double dienTich_63_Thu()
{
    // Công thức: 2 * (dài * rộng + rộng * cao + dài * cao)
    return 2 * (dai_63_Thu * rong_63_Thu + rong_63_Thu * cao_63_Thu +
dai_63_Thu * cao_63_Thu);
}
// Phương thức tính thể tích
public double theTich_63_Thu()
{
    // Công thức: dài * rộng * cao
    return dai_63_Thu * rong_63_Thu * cao_63_Thu;
}
// Phương thức tính chu vi (tổng độ dài 12 cạnh)
public double chuVi_63_Thu()
{
    // Công thức: 4 * (dài + rộng + cao)
    return 4 * (dai_63_Thu + rong_63_Thu + cao_63_Thu);
}
}
}

```

Bước 9: Xử lý các nút xem kết quả trên giao diện => double click vào button

- Double Click vào Button Tính ở giao diện để hiện ra màn hình như sau và thực hiện viết lệnh vào đó:

```
HinhHopChuNhat_63_Thu.cs* 63_Thu_Hin...pChuNhat.cs* 63_Thu_HinhH....cs [Design]*
63_Thu_UnitTest_HinhHopChuNhat _63_Thu_UnitTest_HinhHopChuNhatFor tbTheTich_63_Thu
47 }
48
49
50 1 reference
51 private void btnTinh_63_Thu_Click(object sender, EventArgs e)
52 {
53 }
54
55 }
56

1 reference
private void btnTinh_63_Thu_Click(object sender, EventArgs e)
{
    try
    {
        double dai_63_Thu = double.Parse(tbChieuDai_63_Thu.Text);
        double rong_63_Thu = double.Parse(tbChieuRong_63_Thu.Text);
        double cao_63_Thu = double.Parse(tbChieuCao_63_Thu.Text);

        HinhHopChuNhat_63_Thu h_63_Thu = new HinhHopChuNhat_63_Thu(dai_63_Thu, rong_63_Thu, cao_63_Thu);

        tbDienTich_63_Thu.Text = h_63_Thu.dienTich_63_Thu().ToString();
        tbTheTich_63_Thu.Text= h_63_Thu.theTich_63_Thu().ToString();
        tbChuVi_63_Thu.Text = h_63_Thu.chuVi_63_Thu().ToString();
    }
    catch
    {
        MessageBox.Show("Lỗi dữ liệu");
    }
}
```

```
private void btnTinh_63_Thu_Click(object sender, EventArgs e)
{
    try
    {
        double dai_63_Thu = double.Parse(tbChieuDai_63_Thu.Text);
        double rong_63_Thu = double.Parse(tbChieuRong_63_Thu.Text);
        double cao_63_Thu = double.Parse(tbChieuCao_63_Thu.Text);

        HinhHopChuNhat_63_Thu h_63_Thu = new
        HinhHopChuNhat_63_Thu(dai_63_Thu, rong_63_Thu, cao_63_Thu);

        tbDienTich_63_Thu.Text = h_63_Thu.dienTich_63_Thu().ToString();
        tbTheTich_63_Thu.Text= h_63_Thu.theTich_63_Thu().ToString();
        tbChuVi_63_Thu.Text = h_63_Thu.chuVi_63_Thu().ToString();
    }
    catch
    {
        MessageBox.Show("Lỗi dữ liệu");
    }
}
```


2. Mô tả Unit Test

2.1 Mô tả

Unit test này dùng để kiểm tra tính đúng đắn của các phương thức tính toán và xử lý logic trong lớp `HinhHopChuNhat_63_Thu` và tập trung kiểm thử việc khởi tạo đối tượng và các công thức tính toán dựa trên ba yếu tố đầu vào (chiều dài, chiều rộng, chiều cao) với các điều kiện sau:

Điều kiện tồn tại: Chiều dài (d) > 0 , Chiều rộng (r) > 0 , Chiều cao (c) > 0 .

Tính toán Thể tích: Chiều dài $\times$ Chiều rộng $\times$ Chiều cao

Tính toán Diện tích toàn phần: $S = 2 \times (\text{Chiều dài} \times \text{Chiều rộng} + \text{Chiều dài} \times \text{Chiều cao} + \text{Chiều rộng} \times \text{Chiều cao})$

Tính toán Chu vi (tổng 12 cạnh): $4 \times (\text{Chiều dài} + \text{Chiều rộng} + \text{Chiều cao})$

2.2 Bảng các Test case

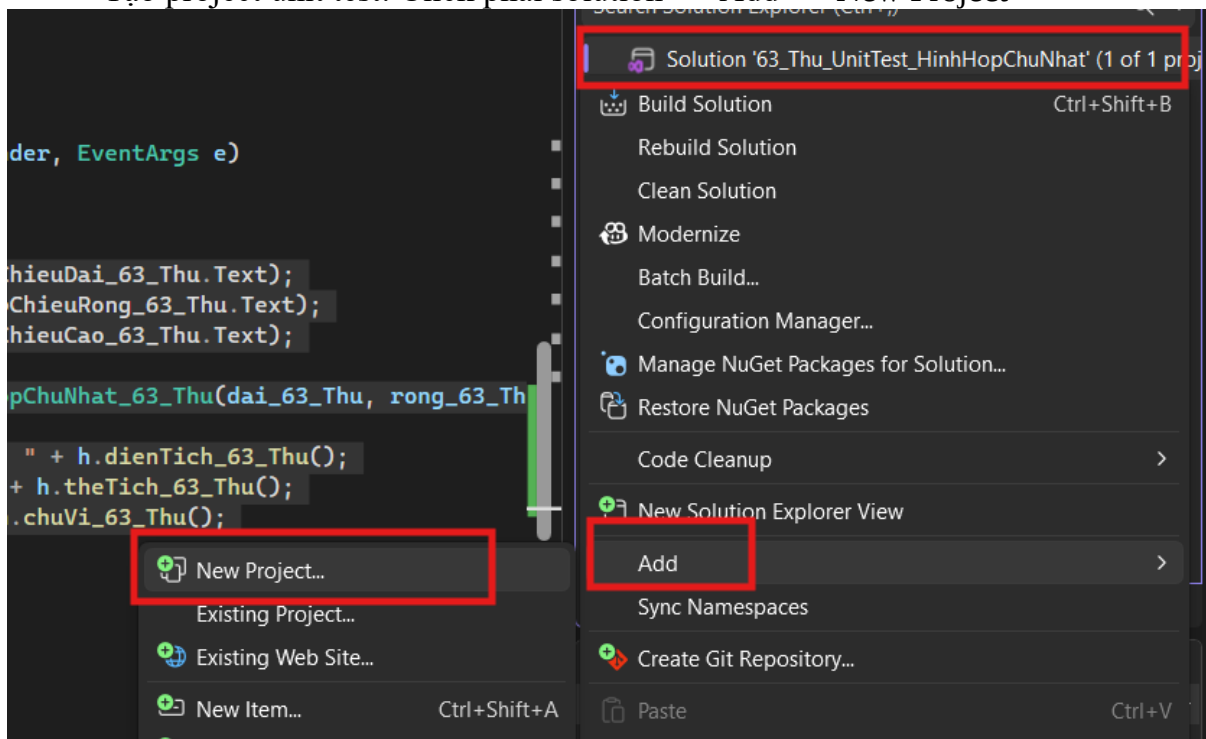
STT	TestCaseID	Chiều dài	Chiều rộng	Chiều cao	Mô tả kiểm thử	Kết quả mong đợi	Trạng thái
1	TC01_63_Thu_HopLe_SoNgunyen	2	3	4	Kiểm tra dữ liệu số nguyên hợp lệ	Tính đúng thể tích = 24, diện tích = 52, chu vi = 36	Pass
2	TC02_63_Thu_HopLe_SoThuc	1.5	2.5	3	Kiểm tra dữ liệu số thực hợp lệ	Tính đúng thể tích = 11.25, diện tích = 31.5, chu vi = 28	Pass
3	TC03_63_Thu_HopLe_HinhLapPhuong	5	5	5	Kiểm tra trường hợp hình lập phương	Tính đúng thể tích = 125, diện tích = 150, chu vi = 60	Pass
4	TC04_63_Thu_MotCanhBang0	0	2	3	Kiểm tra một cạnh bằng 0	Hệ thống báo lỗi (ArgumentOutOfRangeException)	Pass
5	TC05_63_Thu_TatCaBang0	0	0	0	Kiểm tra tất cả cạnh bằng 0	Hệ thống báo lỗi (ArgumentOutOfRangeException)	Pass
6	TC06_63_Thu_SoRatNho	1	2	3	Kiểm tra giá trị rất nhỏ	Tính đúng kết quả với sai số cho phép	Pass
7	TC07_63_Thu_SoLon	1000	2000	3000	Kiểm tra giá trị lớn	Tính đúng thể tích, diện tích, chu vi	Pass
8	TC08_63_Thu_MotCanhAm	-2	3	4	Kiểm tra một cạnh âm	Hệ thống báo lỗi (ArgumentOutOfRangeException)	Pass

9	TC09_63_Thu_TatCaCanhAm	-1	-2	-3	Kiểm tra tất cả cạnh âm	Hệ thống báo lỗi (ArgumentOutOfRangeException)	Pass
---	-------------------------	----	----	----	-------------------------	--	------

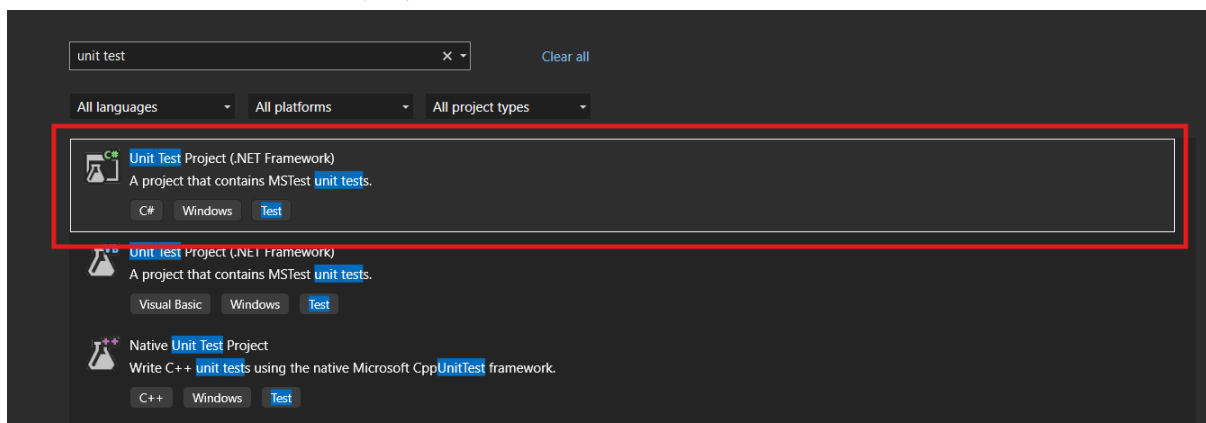
3. Thực hiện Test Case

Bước 1: Tạo loại project là “Unit Test Project” để kiểm thử quá trình thực hiện các phép tính Hình Hộp Chữ nhật

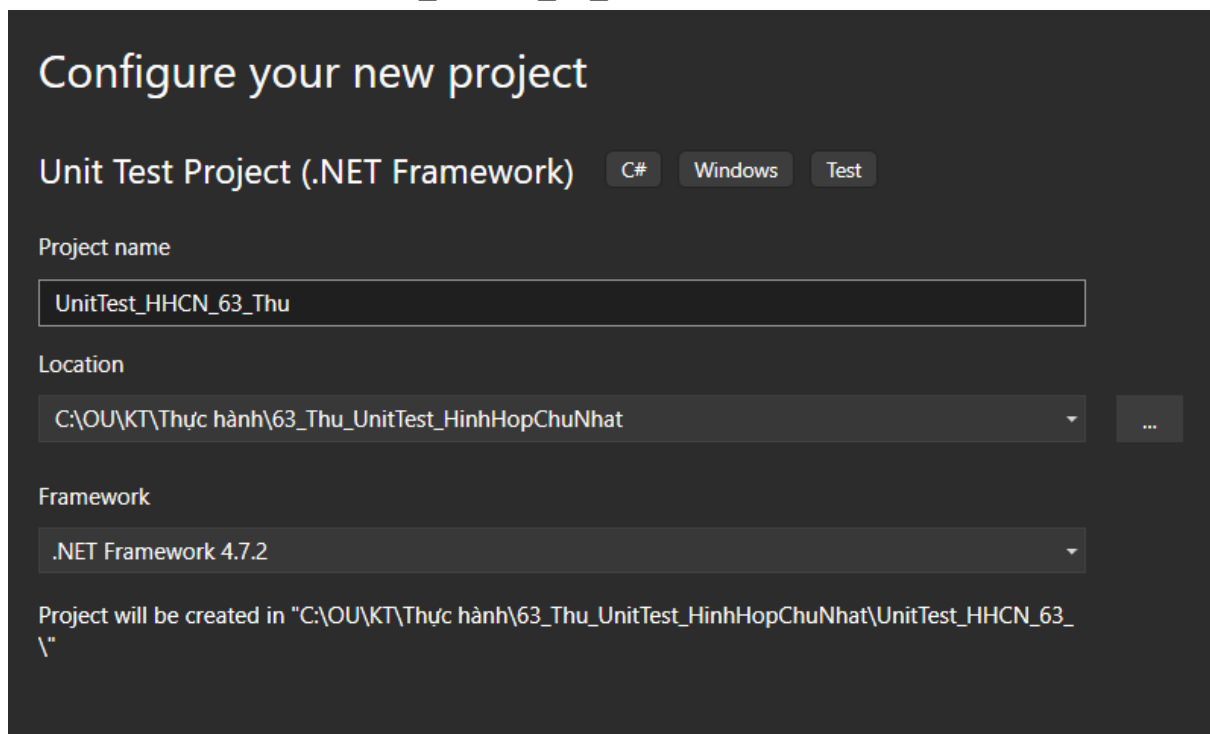
- Tạo project unit test: Click phải solution => Add => New Project



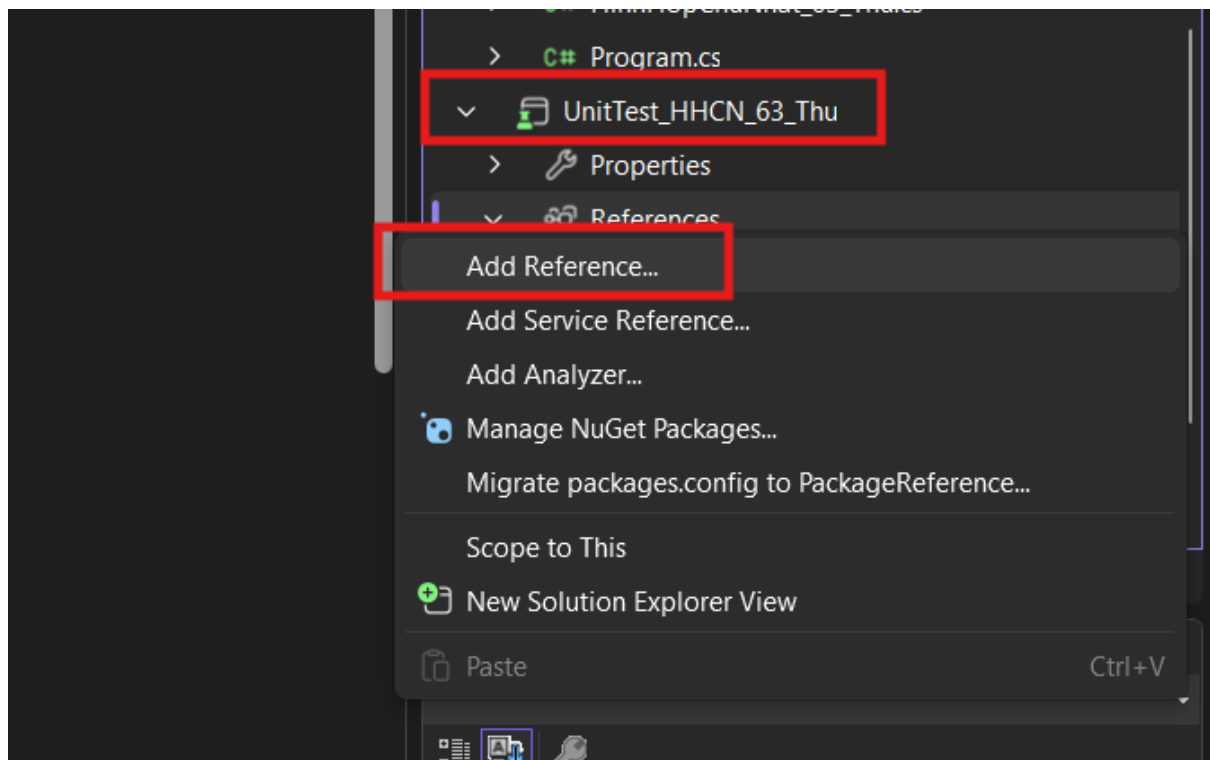
Bước 2: Chọn Unit Test (C#) => Next



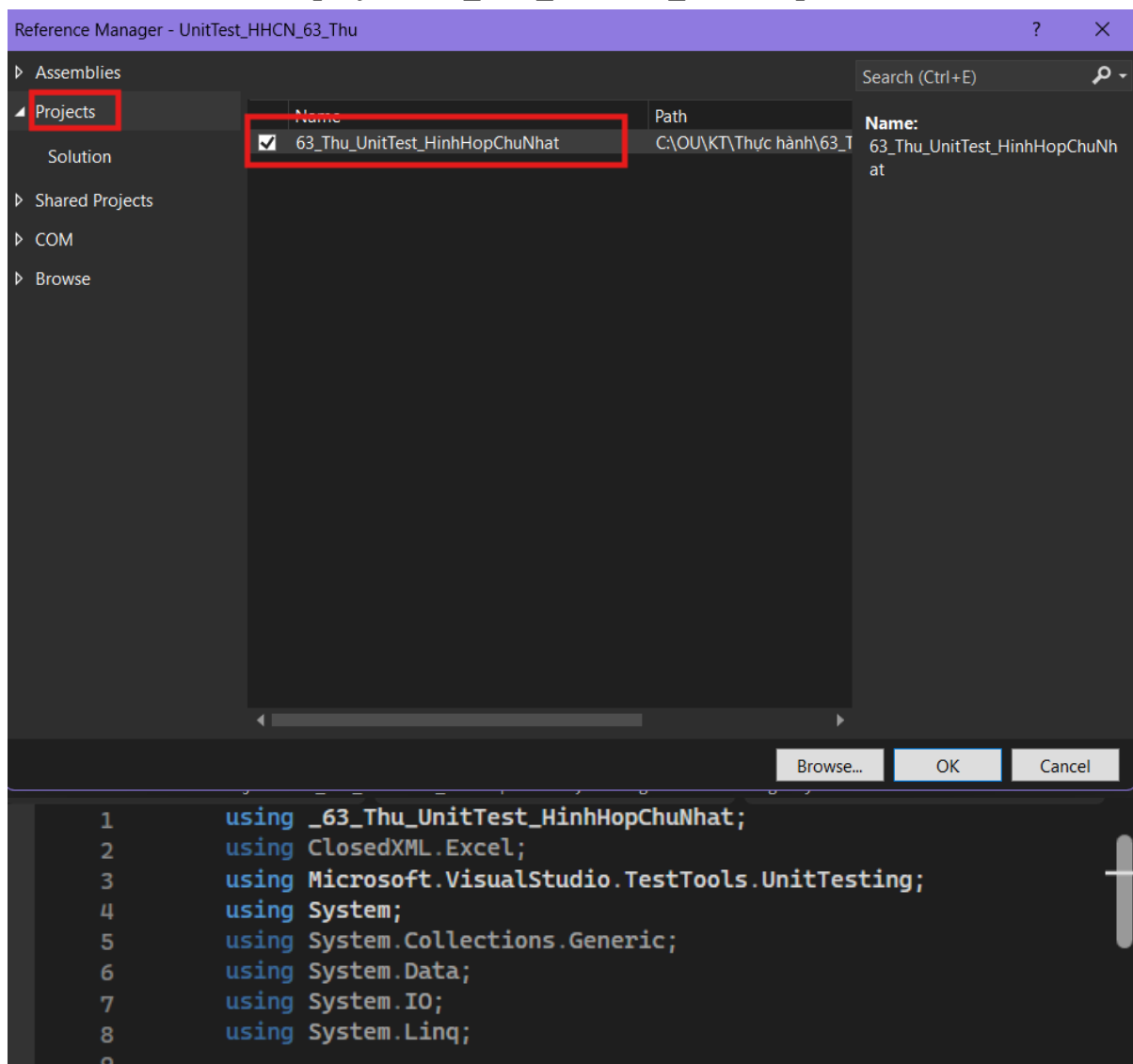
Bước 3: Đặt tên là “UnitTest_HHCN_63_Thu” => Create



Bước 4: Add References từ Unit Test đến Project: Click phải References => Add References



Bước 5: Tick chọn tên project “63_Thu_UnitTest_HinhHopChuNhat” => OK



```
using _63_Thu_UnitTest_HinhHopChuNhat;  
using ClosedXML.Excel;  
using Microsoft.VisualStudio.TestTools.UnitTesting;  
using System;  
using System.Collections.Generic;  
using System.Data;  
using System.IO;  
using System.Linq;
```

Ở đây Project mang tên “63_Thu_UnitTest_HinhHopChuNhat” nhưng sau khi Add Reference thành công thì trong giao diện UnitTest hiển thị “using _63_Thu_UnitTest_HinhHopChuNhat” là do Tên project bắt đầu bằng:

63

đây là số, nên khi Visual Studio tự sinh namespace, nó sẽ tự thêm dấu gạch dưới _ vào đầu để namespace hợp lệ theo quy tắc ngôn ngữ C#.

Bước 6: Viết code cho các test case:

1. TC01_63_Thu_HopLe_SoNguyen()

```
13 0 references
14 public class Test_HHCN_63_Thu
15 {
16     [TestMethod]
17     //TC1: chieuDai = 2, chieuRong = 3, chieuCao = 4, ketQua = true
18     // 0 references
19     public void TC01_63_Thu_HopLe_SoNguyen()
20     {
21         var h_63_Thu = new HìnhHopChuNhat_63_Thu(2, 3, 4);
22
23         Assert.AreEqual(24, h_63_Thu.theTich_63_Thu());
24         Assert.AreEqual(52, h_63_Thu.dienTich_63_Thu());
25         Assert.AreEqual(36, h_63_Thu.chuVi_63_Thu());
26     }
27 }
```

```
[TestMethod]
//TC1: chieuDai = 2, chieuRong = 3, chieuCao = 4, ketQua = true
public void TC01_63_Thu_HopLe_SoNguyen()
{
    var h_63_Thu = new HìnhHopChuNhat_63_Thu(2, 3, 4);

    Assert.AreEqual(24, h_63_Thu.theTich_63_Thu());
    Assert.AreEqual(52, h_63_Thu.dienTich_63_Thu());
    Assert.AreEqual(36, h_63_Thu.chuVi_63_Thu());
}
```

2. TC02_63_Thu_HopLe_SoThuc()

```
25 [TestMethod]
26 //TC2: chieuDai = 1.5, chieuRong = 2.5, chieuCao = 3, ketQua = true
27 // 0 references
28 public void TC02_63_Thu_HopLe_SoThuc()
29 {
30     var h_63_Thu = new HìnhHopChuNhat_63_Thu(1.5, 2.5, 3);
31
32     Assert.AreEqual(11.25, h_63_Thu.theTich_63_Thu());
33     Assert.AreEqual(31.5, h_63_Thu.dienTich_63_Thu());
34     Assert.AreEqual(28, h_63_Thu.chuVi_63_Thu());
35 }
```

```
[TestMethod]
//TC2: chieuDai = 1.5, chieuRong = 2.5, chieuCao = 3, ketQua = true
public void TC02_63_Thu_HopLe_SoThuc()
{
    var h_63_Thu = new HìnhHopChuNhat_63_Thu(1.5, 2.5, 3);

    Assert.AreEqual(11.25, h_63_Thu.theTich_63_Thu());
    Assert.AreEqual(31.5, h_63_Thu.dienTich_63_Thu());
    Assert.AreEqual(28, h_63_Thu.chuVi_63_Thu());
}
```

```
}
```

3. TC03_63_Thu_HopLe_HinhLapPhuong()

```
35 [TestMethod]
36 //TC3: chieuDai = 5, chieuRong = 5, chieuCao = 5, ketQua = true
    0 references
37 public void TC03_63_Thu_HopLe_HinhLapPhuong()
38 {
39     var h_63_Thu = new HinhHopChuNhat_63_Thu(5, 5, 5);
40
41     Assert.AreEqual(125, h_63_Thu.theTich_63_Thu());
42     Assert.AreEqual(150, h_63_Thu.dienTich_63_Thu());
43     Assert.AreEqual(60, h_63_Thu.chuVi_63_Thu());
44 }
```

```
[TestMethod]
//TC3: chieuDai = 5, chieuRong = 5, chieuCao = 5, ketQua = true
public void TC03_63_Thu_HopLe_HinhLapPhuong()
{
    var h_63_Thu = new HinhHopChuNhat_63_Thu(5, 5, 5);

    Assert.AreEqual(125, h_63_Thu.theTich_63_Thu());
    Assert.AreEqual(150, h_63_Thu.dienTich_63_Thu());
    Assert.AreEqual(60, h_63_Thu.chuVi_63_Thu());
}
```

4. TC04_63_Thu_MotCanhBang0()

```
45 [TestMethod]
46 //TC4: chieuDai = 0, chieuRong = 2, chieuCao = 3, ketQua = true
    0 references
47 public void TC04_63_Thu_MotCanhBang0()
48 {
49     Assert.ThrowsException<ArgumentOutOfRangeException>(() =>
50     {
51         new HinhHopChuNhat_63_Thu(0, 2, 3);
52     });
53 }
```

```
[TestMethod]
//TC4: chieuDai = 0, chieuRong = 2, chieuCao = 3, ketQua = true
public void TC04_63_Thu_MotCanhBang0()
{
    Assert.ThrowsException<ArgumentOutOfRangeException>(() =>
    {
        new HinhHopChuNhat_63_Thu(0, 2, 3);
    });
}
```

5. TC05_TatCaBang0()

```
54 [TestMethod]
55 //TC5: chieuDai = 0, chieuRong = 0, chieuCao = 0, ketQua = báo lỗi
56 public void TC05_TatCaBang0()
57 {
58     Assert.ThrowsException<ArgumentOutOfRangeException>(() =>
59     {
60         new HìnhHopChuNhat_63_Thu(0, 0, 0);
61     });
62 }
```

```
[TestMethod]
//TC5: chieuDai = 0, chieuRong = 0, chieuCao = 0, ketQua = báo lỗi
public void TC05_TatCaBang0()
{
    Assert.ThrowsException<ArgumentOutOfRangeException>(() =>
    {
        new HìnhHopChuNhat_63_Thu(0, 0, 0);
    });
}
```

6. TC06_63_Thu_SoRatNho()

```
63 [TestMethod]
64 //TC6: chieuDai = 0.001, chieuRong = 2, chieuCao = 3, ketQua = true
65 public void TC06_63_Thu_SoRatNho()
66 {
67     var h_63_Thu = new HìnhHopChuNhat_63_Thu(0.001, 2, 3);
68
69     Assert.AreEqual(0.006, h_63_Thu.theTich_63_Thu(), 0.001);
70     Assert.AreEqual(12.01, h_63_Thu.dienTich_63_Thu(), 0.01);
71     Assert.AreEqual(20.004, h_63_Thu.chuVi_63_Thu(), 0.001);
72 }
```

```
[TestMethod]
//TC6: chieuDai = 0.001, chieuRong = 2, chieuCao = 3, ketQua = true
public void TC06_63_Thu_SoRatNho()
{
    var h_63_Thu = new HìnhHopChuNhat_63_Thu(0.001, 2, 3);

    Assert.AreEqual(0.006, h_63_Thu.theTich_63_Thu(), 0.001);
    Assert.AreEqual(12.01, h_63_Thu.dienTich_63_Thu(), 0.01);
    Assert.AreEqual(20.004, h_63_Thu.chuVi_63_Thu(), 0.001);
}
```

7. TC07_63_Thu_SoLon()

```
73 [TestMethod]
74 //TC7: chieuDai = 1000, chieuRong = 2000, chieuCao = 3000, ketQua = true
75 0 references
76 public void TC07_63_Thu_SoLon()
77 {
78     var h_63_Thu = new HìnhHopChuNhat_63_Thu(1000, 2000, 3000);
79
80     Assert.AreEqual(6000000000, h_63_Thu.theTich_63_Thu());
81     Assert.AreEqual(22000000, h_63_Thu.dienTich_63_Thu());
82     Assert.AreEqual(24000, h_63_Thu.chuVi_63_Thu());
83 }
```

[TestMethod]

//TC7: chieuDai = 1000, chieuRong = 2000, chieuCao = 3000, ketQua = true

public void TC07_63_Thu_SoLon()

{

var h_63_Thu = new HìnhHopChuNhat_63_Thu(1000, 2000, 3000);

Assert.AreEqual(6000000000, h_63_Thu.theTich_63_Thu());

Assert.AreEqual(22000000, h_63_Thu.dienTich_63_Thu());

Assert.AreEqual(24000, h_63_Thu.chuVi_63_Thu());

}

8. TC08_63_Thu_MotCanhAm()

```
83 [TestMethod]
84 //TC8: chieuDai = -2 (âm), chieuRong = 3, chieuCao = 4, ketQua = báo lỗi
85 0 references
86 public void TC08_63_Thu_MotCanhAm()
87 {
88     Assert.ThrowsException<ArgumentOutOfRangeException>(() =>
89     {
90         new HìnhHopChuNhat_63_Thu(-2, 3, 4);
91     });
92 }
```

[TestMethod]

//TC8: chieuDai = -2 (âm), chieuRong = 3, chieuCao = 4, ketQua = báo lỗi

public void TC08_63_Thu_MotCanhAm()

{

Assert.ThrowsException<ArgumentOutOfRangeException>(() =>

{

new HìnhHopChuNhat_63_Thu(-2, 3, 4);

});

}

9. TC09_63_Thu_TatCaCanhAm()

```
92 [TestMethod]
93 //TC9: chieuDai = -1, chieuRong = -2, chieuCao = -3, ketQua = báo lỗi
94   0 references
95 public void TC09_63_Thu_TatCaCanhAm()
96 {
97     Assert.ThrowsException<ArgumentOutOfRangeException>(() =>
98     {
99         new HìnhHopChuNhat_63_Thu(-1, -2, -3);
100     });
101 }
```

[TestMethod]

//TC9: chieuDai = -1, chieuRong = -2, chieuCao = -3, ketQua = báo lỗi

public void TC09_63_Thu_TatCaCanhAm()

{

Assert.ThrowsException<ArgumentOutOfRangeException>(() =>

{

new HìnhHopChuNhat_63_Thu(-1, -2, -3);

});

}

10. TC10_63_Thu_SaiKieuDuLieu()

```
101 [TestMethod]
102 //TC10: chieuDai hoặc chieuRong hoặc chieuCao = a, ketQua = báo lỗi
103   0 references
104 public void TC10_63_Thu_SaiKieuDuLieu()
105 {
106     Assert.ThrowsException<FormatException>(() =>
107     {
108         Convert.ToDouble("a");
109     });
110 }
```

[TestMethod]

//TC10: chieuDai hoặc chieuRong hoặc chieuCao = a, ketQua = báo lỗi

public void TC10_63_Thu_SaiKieuDuLieu()

{

Assert.ThrowsException<FormatException>(() =>

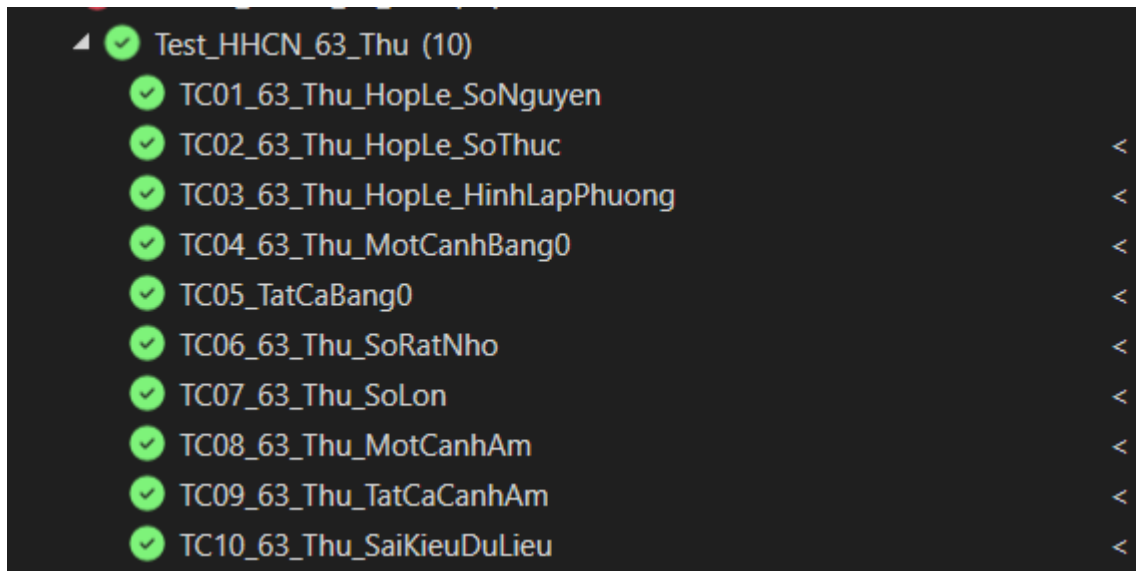
{

Convert.ToDouble("a");

});

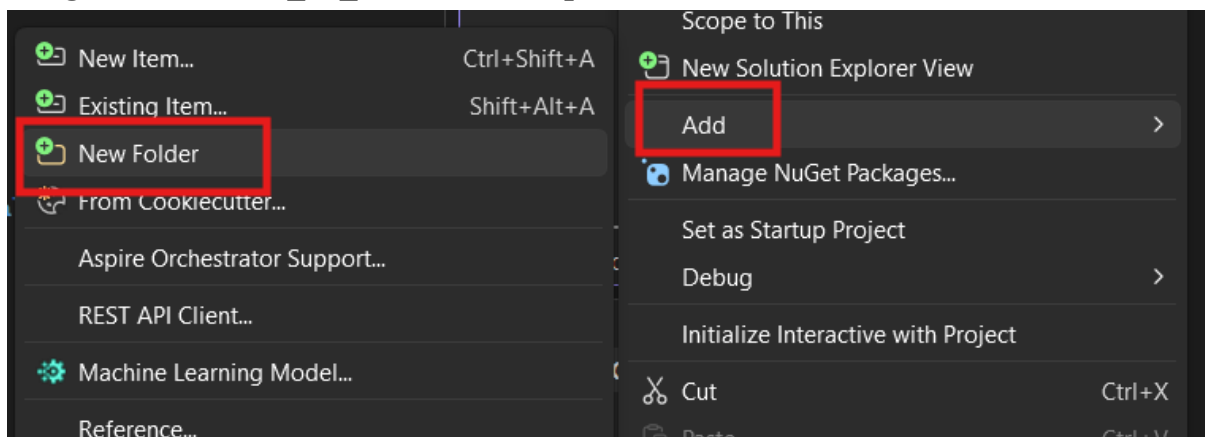
}

Bước 7: Chạy test case: 10 passed, 0 failed

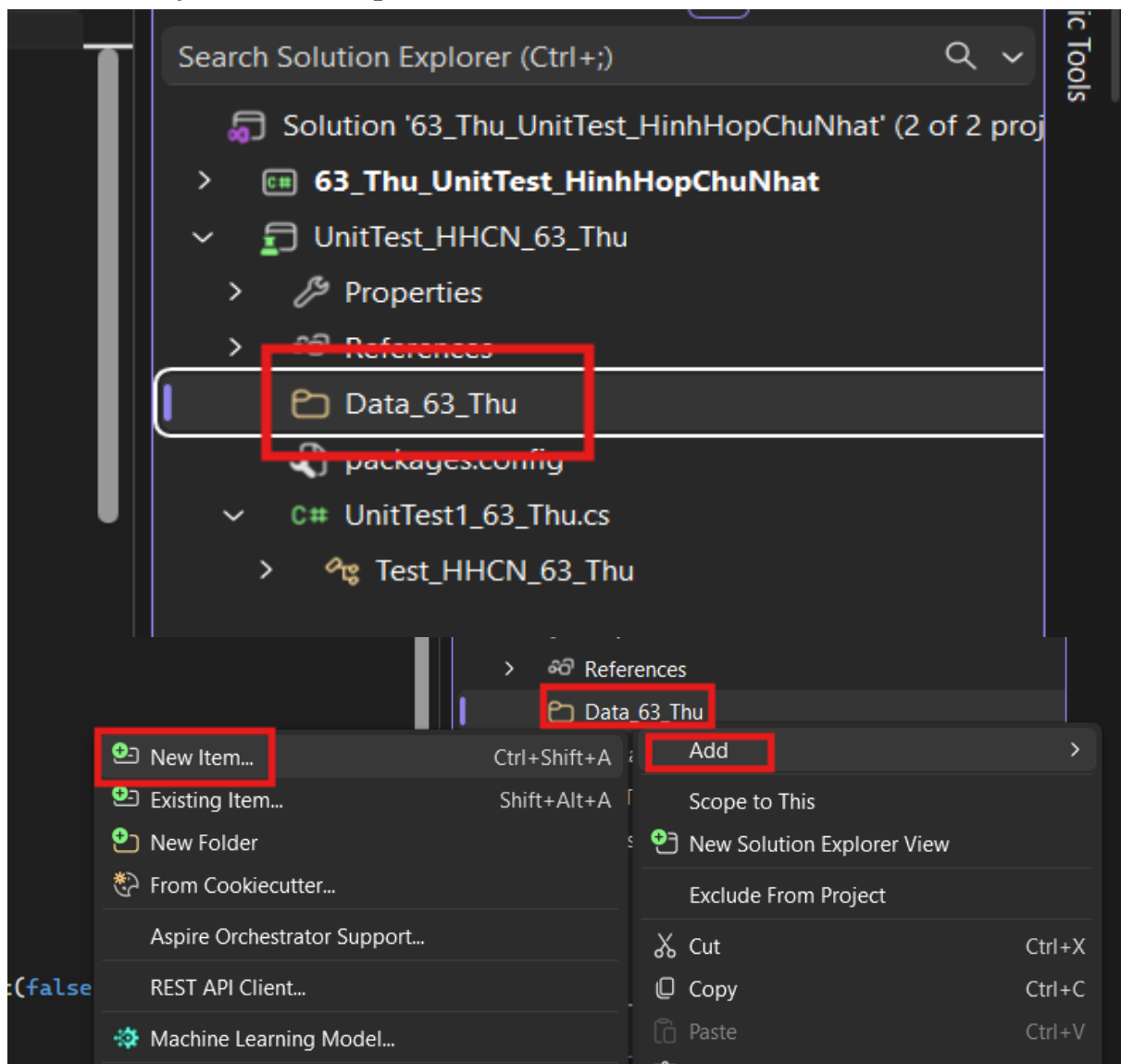


4. Thực hiện Test từ File .CSV

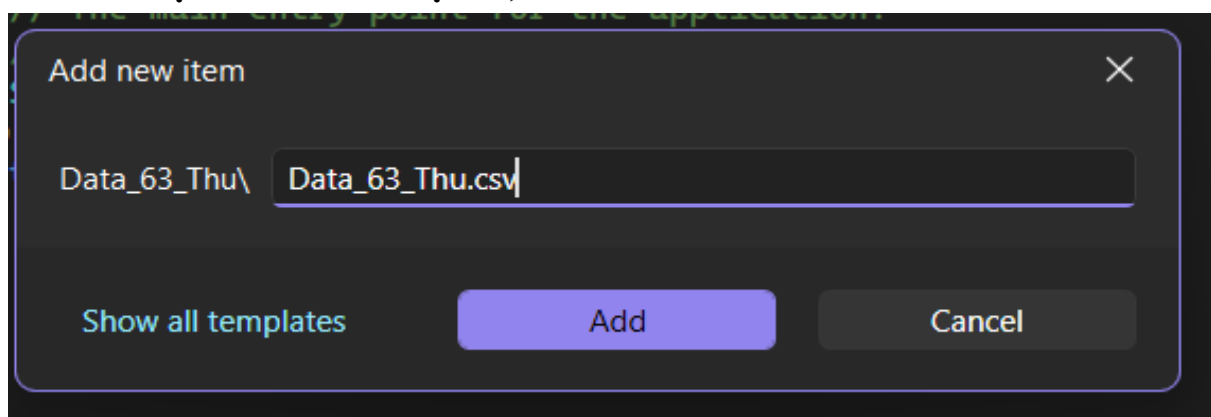
Bước 1: Tạo thư mục Data_63_Thu trong project test ,tạo file TestData_63_Thu.csv và đặt trong thư mục Data_63_Thu để thiết lập dữ liệu test có sẵn

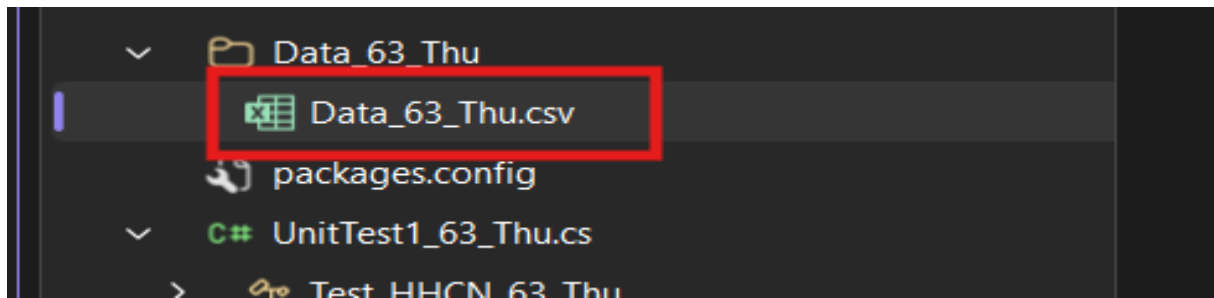


Bước 2: Tạo file .csv: Click phải Data => Add => New Item



Bước 3: Chọn Text File => Đặt tên, đổi thành đuôi .csv => Add



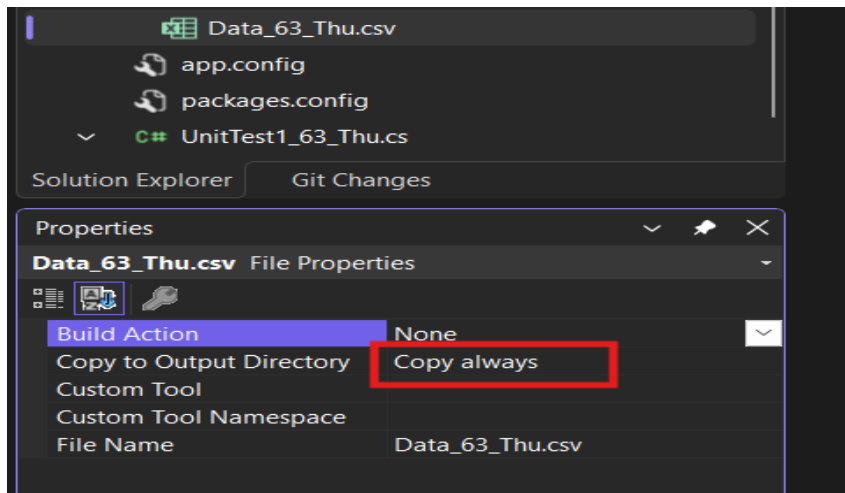


Bước 4: Tạo dữ liệu trong file Data_63_Thu.csv

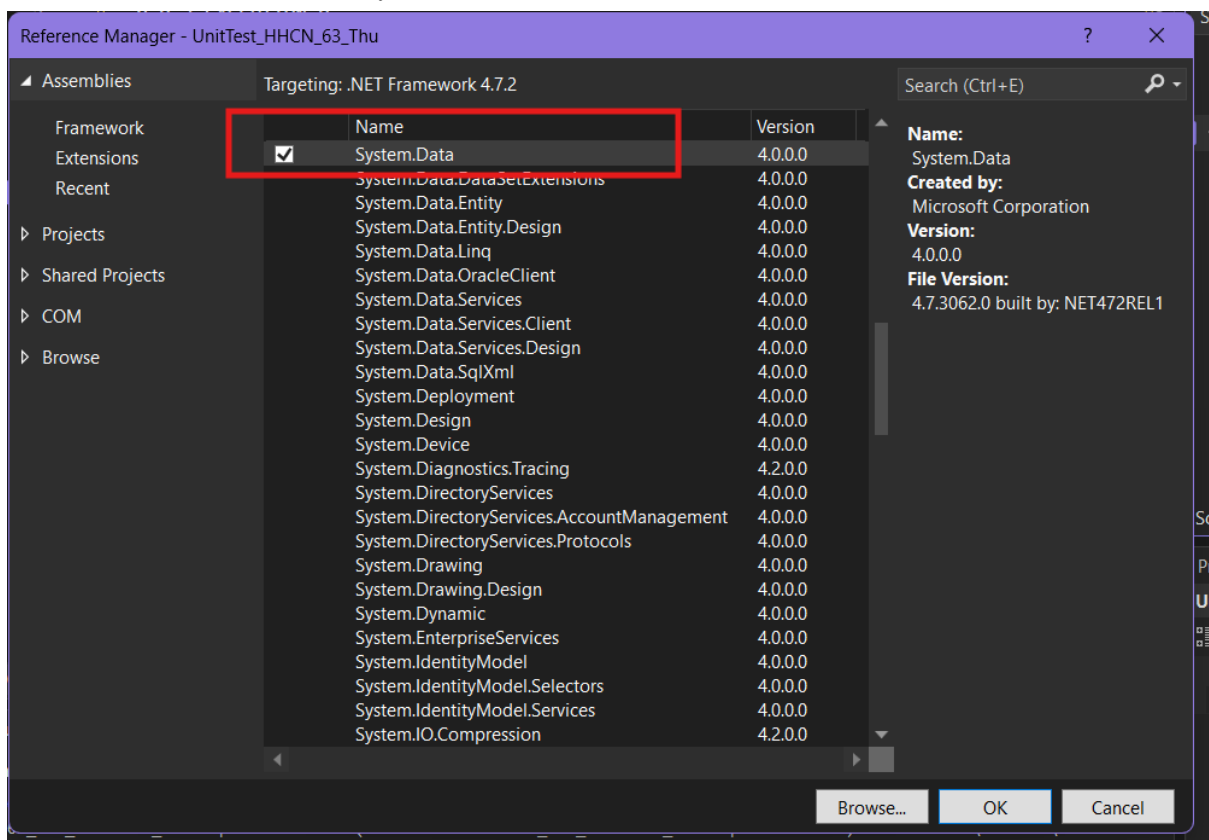
```
UnitTest1_63_Thu.cs* | Data_63_Thu.csv | 63_Thu_HinhH...t.cs [Design] | 63_Thu_HinhHopChui...t.cs [Design]
1      dai_63_Thu,rong_63_Thu,cao_63_Thu,type_63_Thu,expected_63_Thu
2      2,3,4,TheTich,24
3      1.5,2.5,3,TheTich,11.25
4      5,5,5,TheTich,125
5      0,2,3,EXCEPTION,0
6      0,0,0,EXCEPTION,0
7      0.001,2,3,TheTich,0.006
8      1000,2000,3000,TheTich,6000000000
9      -2,3,4,EXCEPTION,0
10     -1,-2,-3,EXCEPTION,0
11     a,2,3,EXCEPTION,0
```

```
dai_63_Thu,rong_63_Thu,cao_63_Thu,type_63_Thu,expected_63_Thu
2,3,4,TheTich,24
1.5,2.5,3,TheTich,11.25
5,5,5,TheTich,125
0,2,3,EXCEPTION,0
0,0,0,EXCEPTION,0
0.001,2,3,TheTich,0.006
1000,2000,3000,TheTich,6000000000
-2,3,4,EXCEPTION,0
-1,-2,-3,EXCEPTION,0
a,2,3,EXCEPTION,0
```

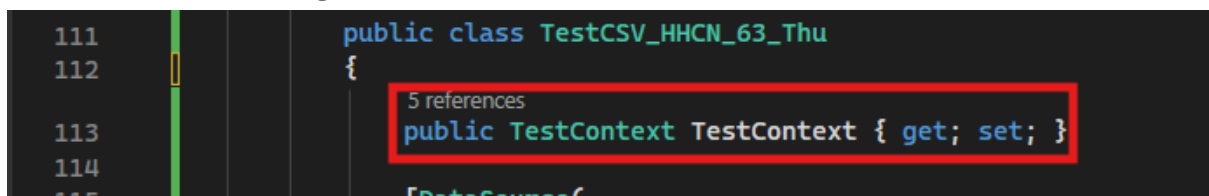
Bước 5: Click vào file .csv, thiết lập thuộc tính “Copy to Output Directory” thành “Copy always”



Bước 6: Add References từ Unit Test đến Project: Click phải References => Add References => Recent => Tick chọn System.Data => Ok



Bước 7: Tạo đối tượng TestContext



Bước 8: Viết test case để test dữ liệu từ file .csv

```
109     }
110     [TestClass]
111     0 references
112     public class TestCSV_HHCN_63_Thu
113     {
114         5 references
115         public TestContext TestContext { get; set; }
116
117         [DataSource(
118             "Microsoft.VisualStudio.TestTools.DataSource.CSV",
119             "|DataDirectory|\\Data_63_Thu\\Data_63_Thu.csv",
120             "Data_63_Thu#csv",
121             DataAccessMethod.Sequential)]
122         [TestMethod]
123         0 references
124         public void TC11_TestWithDataSource_63_Thu()
125         {
126             string daiStr_63_Thu = TestContext.DataRow[0].ToString();
127             string rongStr_63_Thu = TestContext.DataRow[1].ToString();
128             string caoStr_63_Thu = TestContext.DataRow[2].ToString();
129             string type_63_Thu = TestContext.DataRow[3].ToString();
130             double expected_63_Thu = double.Parse(TestContext.DataRow[4].ToString());
131
132             // Trường hợp dữ liệu sai kiểu (ví dụ "a")
133             if (!double.TryParse(daiStr_63_Thu, out double dai_63_Thu) ||
134                 !double.TryParse(rongStr_63_Thu, out double rong_63_Thu) ||
135                 !double.TryParse(caoStr_63_Thu, out double cao_63_Thu))
136             {
137                 Assert.ThrowsException<FormatException>(() =>
138                 {
139                     Convert.ToDouble(daiStr_63_Thu);
140                 });
141                 return;
142             }
143             // Trường hợp dữ liệu không hợp lệ (âm hoặc bằng 0)
144             if (type_63_Thu == "EXCEPTION")
145             {
146                 Assert.ThrowsException<ArgumentOutOfRangeException>(() =>
147                 {
148                     new HìnhHộpChuNhat_63_Thu(dai_63_Thu, rong_63_Thu, cao_63_Thu);
149                 });
150                 return;
151             }
152             var h_63_Thu = new HìnhHộpChuNhat_63_Thu(dai_63_Thu, rong_63_Thu, cao_63_Thu);
153             double actual_63_Thu = 0;
154             if (type_63_Thu == "TheTich")
155                 actual_63_Thu = h_63_Thu.theTich_63_Thu();
156             else if (type_63_Thu == "DienTich")
157                 actual_63_Thu = h_63_Thu.dienTich_63_Thu();
158             else if (type_63_Thu == "ChuVi")
159                 actual_63_Thu = h_63_Thu.chuVi_63_Thu();
160             Assert.AreEqual(expected_63_Thu, actual_63_Thu, 0.001);
161         }
162     }
163 }
```

```
[TestClass]
public class TestCSV_HHCN_63_Thu
{
    public TestContext TestContext { get; set; }

    [DataSource(
        "Microsoft.VisualStudio.TestTools.DataSource.CSV",
        "|DataDirectory|\\Data_63_Thu\\Data_63_Thu.csv",
        "Data_63_Thu#csv",
```

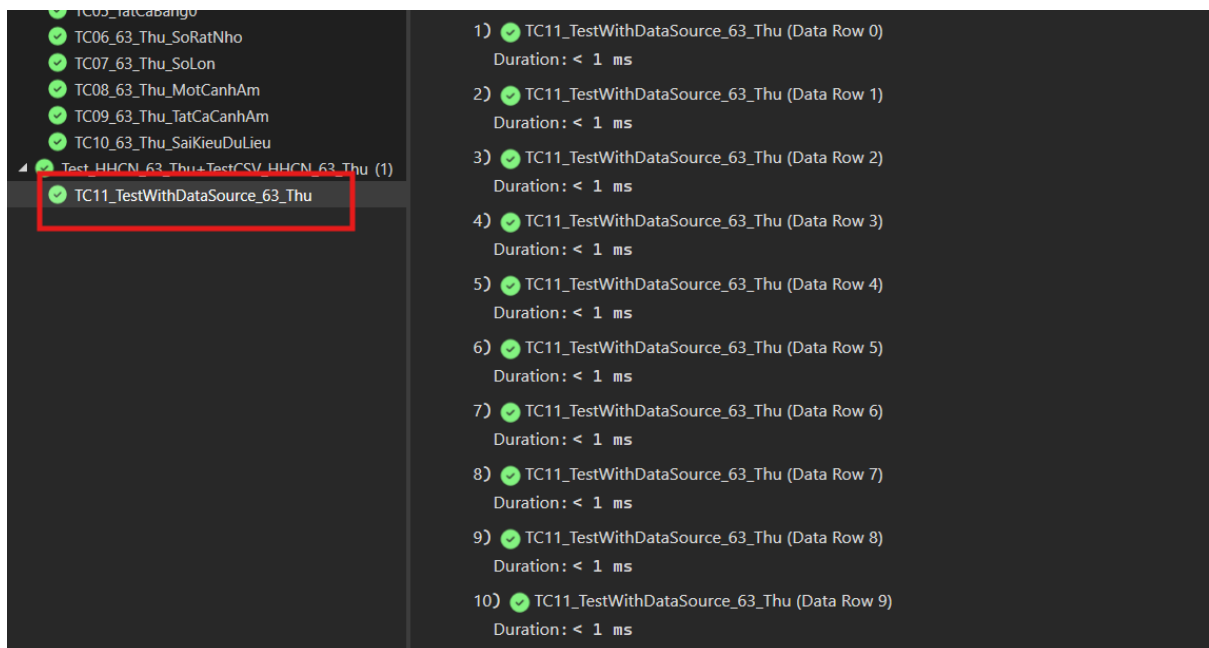
```

        DataAccessMethod.Sequential)]
[TestMethod]
public void TC11_TestWithDataSource_63_Thu()
{
    string daiStr_63_Thu = TestContext.DataRow[0].ToString();
    string rongStr_63_Thu = TestContext.DataRow[1].ToString();
    string caoStr_63_Thu = TestContext.DataRow[2].ToString();
    string type_63_Thu = TestContext.DataRow[3].ToString();
    double expected_63_Thu = double.Parse(TestContext.DataRow[4].ToString());

    // Trường hợp dữ liệu sai kiểu (ví dụ "a")
    if (!double.TryParse(daiStr_63_Thu, out double dai_63_Thu) ||
        !double.TryParse(rongStr_63_Thu, out double rong_63_Thu) ||
        !double.TryParse(caoStr_63_Thu, out double cao_63_Thu))
    {
        Assert.ThrowsException<FormatException>(() =>
        {
            Convert.ToDouble(daiStr_63_Thu);
        });
        return;
    }
    // Trường hợp dữ liệu không hợp lệ (âm hoặc bằng 0)
    if (type_63_Thu == "EXCEPTION")
    {
        Assert.ThrowsException<ArgumentOutOfRangeException>(() =>
        {
            new HìnhHopChuNhat_63_Thu(dai_63_Thu, rong_63_Thu, cao_63_Thu);
        });
        return;
    }
    var h_63_Thu = new HìnhHopChuNhat_63_Thu(dai_63_Thu, rong_63_Thu,
cao_63_Thu);
    double actual_63_Thu = 0;
    if (type_63_Thu == "TheTich")
        actual_63_Thu = h_63_Thu.theTich_63_Thu();
    else if (type_63_Thu == "DienTich")
        actual_63_Thu = h_63_Thu.dienTich_63_Thu();
    else if (type_63_Thu == "ChuVi")
        actual_63_Thu = h_63_Thu.chuVi_63_Thu();
    Assert.AreEqual(expected_63_Thu, actual_63_Thu, 0.001);
}
}

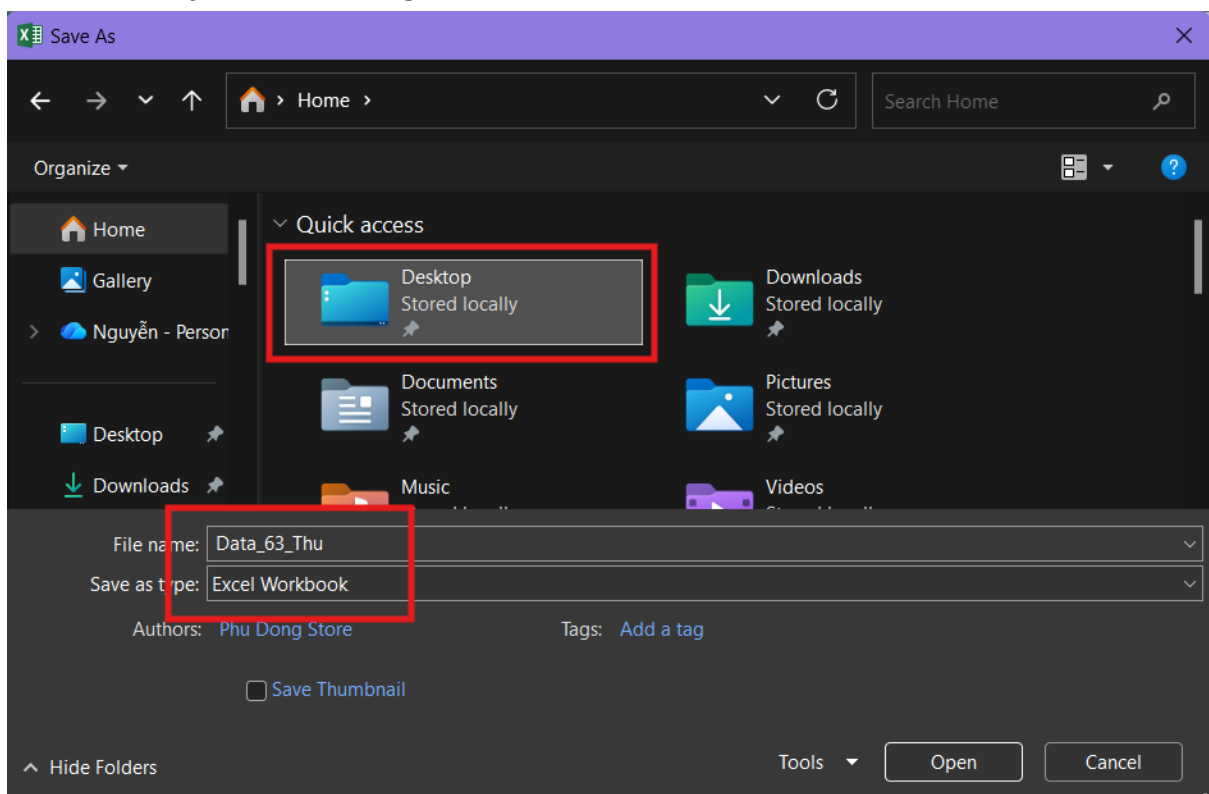
```

Bước 9: Chạy test, kết quả passed



5. Thực hiện Test từ File.XLSX

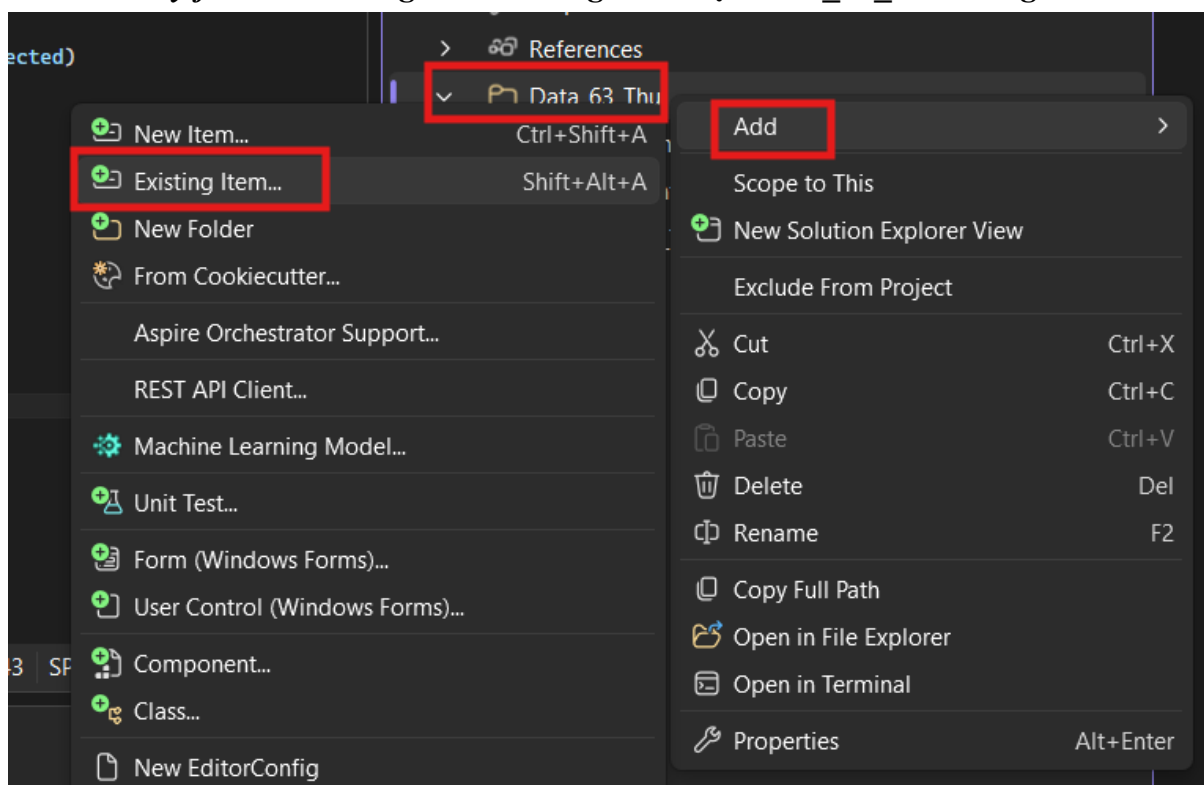
Bước 1: Tạo file Excel mang tên: Data_63_Thu.xlsx

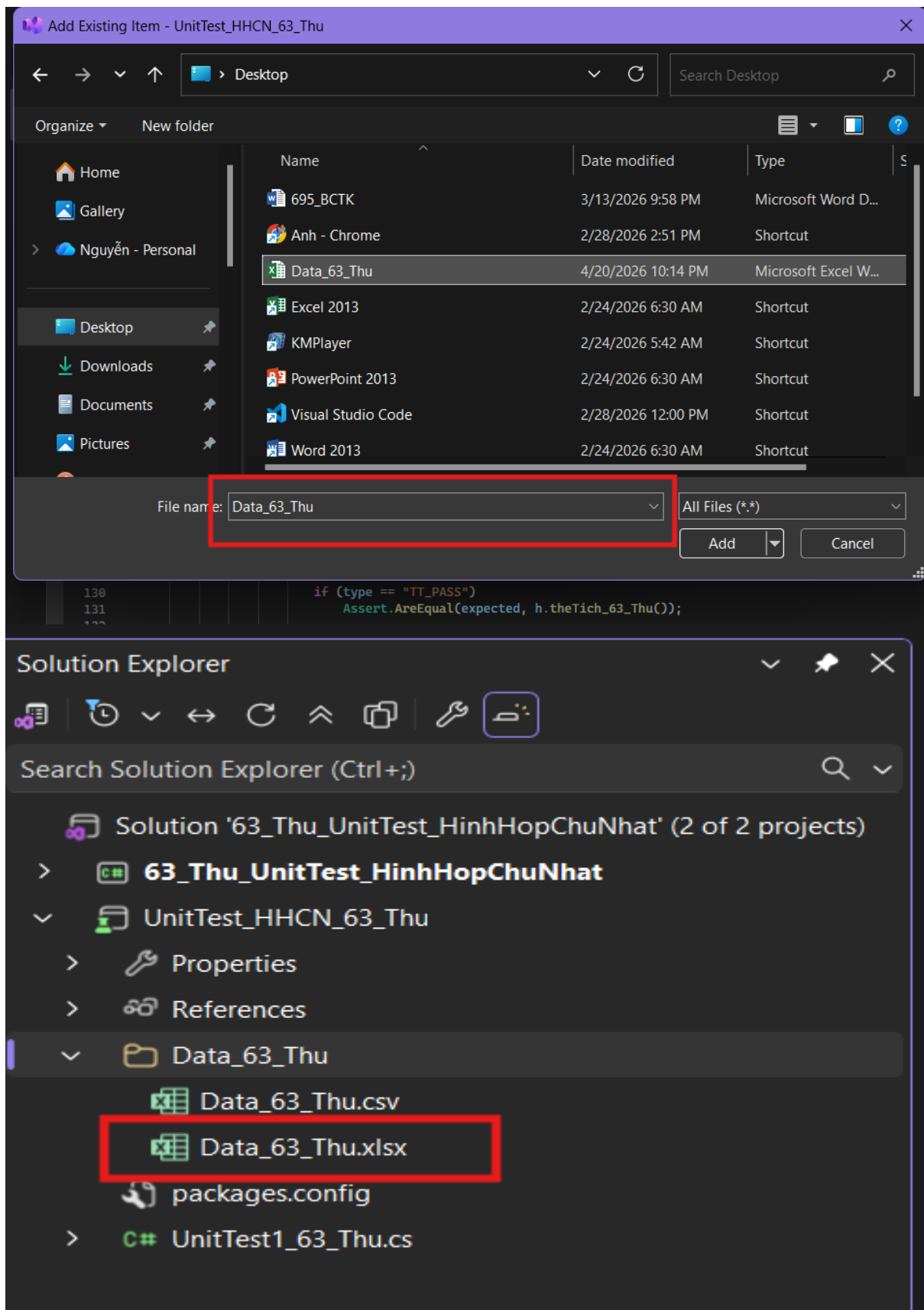


Bước 2: Trong file Excel tạo bảng dữ liệu

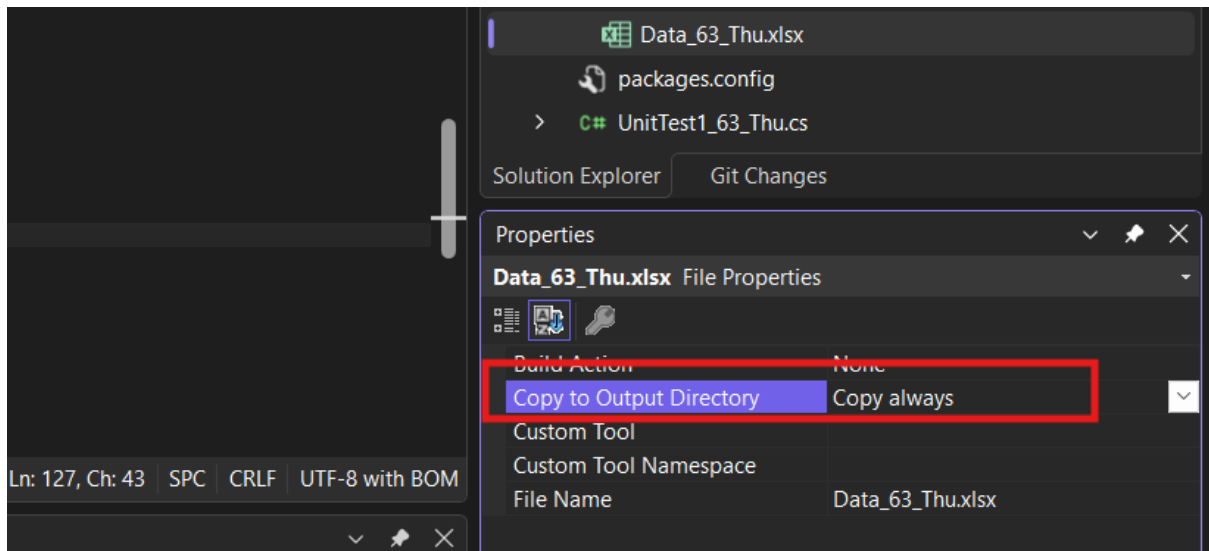
A	B	C	D	E
dai_63_Thu	rong_63_Thu	cao_63_Thu	type_63_Thu	expected_63_Thu
2	3	4	TheTich	24
1.5	2.5	3	TheTich	11.25
5	5	5	TheTich	125
0	2	3	EXCEPTION	0
0	0	0	EXCEPTION	0
0.001	2	3	TheTich	0.006
1000	2000	3000	TheTich	6000000000
-2	3	4	EXCEPTION	0

Bước 3: Đẩy file Excel từ ngoài vào trong Thư mục Data_63_Thu trong Visual Studio



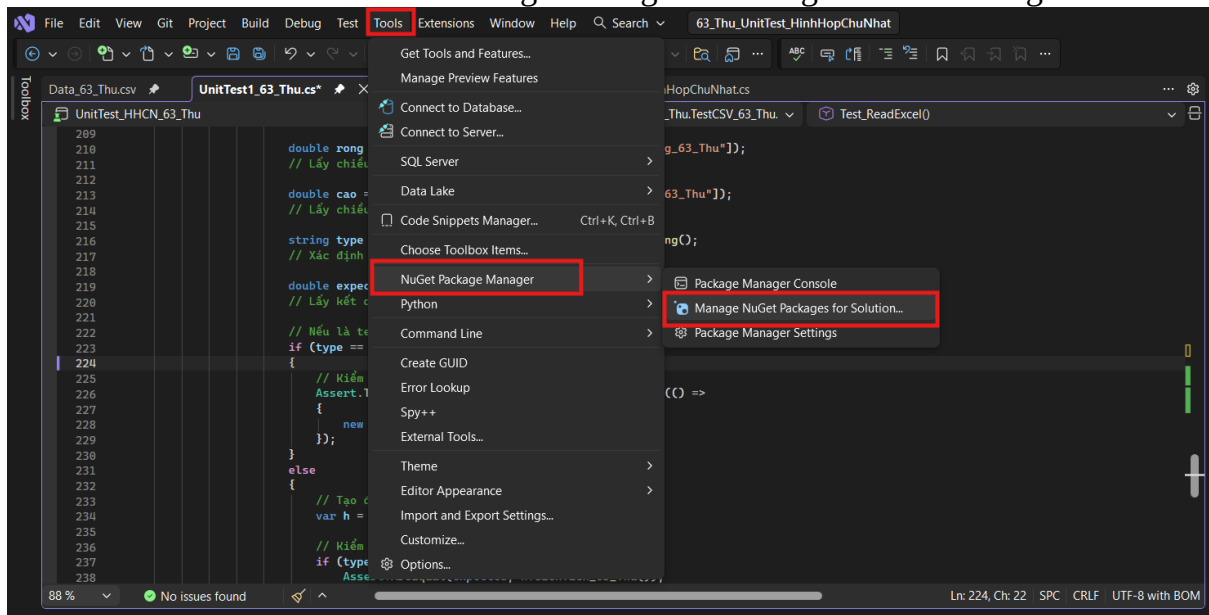


Bước 4: Click vào *Data_63_Thu.xlsx*, thiết lập thuộc tính “Copy to Output Directory” thành “Copy always”

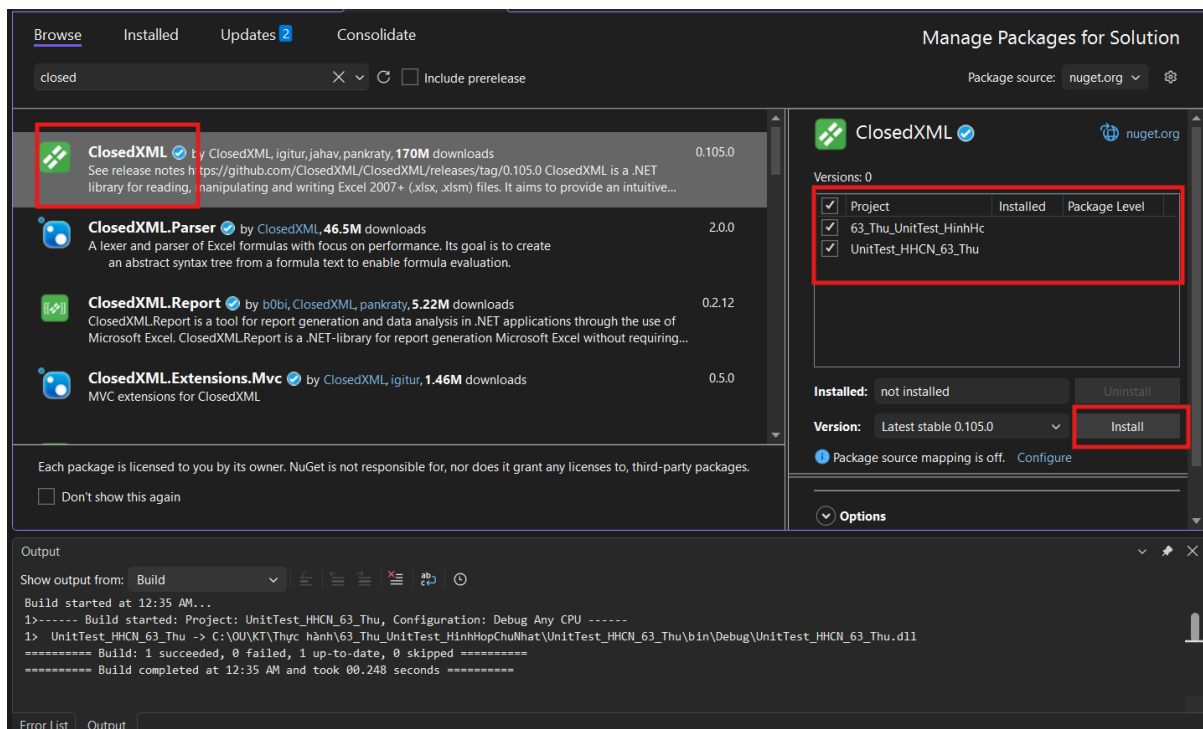


Bước 5: Cài đặt CloseXML

- Menu > Tools > NuGet Package Manager > Manage NuGet Packages for Solution



Search “ClosedXML” > Chọn “ClosedXML” > Tick Project > Install



Bước 6 Tiến hành viết code để Test

```
[TestClass]
public class TestExcel_HHCN_63_Thu
{
    public TestContext TestContext { get; set; }

    [DataSource(
        "System.Data.OleDb",
        "Provider=Microsoft.ACE.OLEDB.12.0;Data
Source=|DataDirectory|\\Data_63_Thu\\Data_63_Thu.xlsx;Extended
Properties='Excel 12.0 Xml;HDR=YES';",
        "Sheet1$",
        DataAccessMethod.Sequential)]
    [TestMethod]
    public void TC12_TestWithDataSource_Excel_63_Thu()
    {
        string daiStr_63_Thu = TestContext.DataRow[0].ToString();
        string rongStr_63_Thu = TestContext.DataRow[1].ToString();
        string caoStr_63_Thu = TestContext.DataRow[2].ToString();
        string type_63_Thu = TestContext.DataRow[3].ToString();
        double expected_63_Thu = double.Parse(TestContext.DataRow[4].ToString());

        double dai_63_Thu, rong_63_Thu, cao_63_Thu;

        // Test dữ liệu sai kiểu (ví dụ "a")
        if (!double.TryParse(daiStr_63_Thu, out dai_63_Thu) ||
```

```

        !double.TryParse(rongStr_63_Thu, out rong_63_Thu) ||
        !double.TryParse(caoStra_63_Thu, out cao_63_Thu))
    {
        Assert.ThrowsException<FormatException>(() =>
        {
            Convert.ToDouble(daiStr_63_Thu);
        });
        return;
    }

    // Test dữ liệu lỗi (âm hoặc 0)
    if (type_63_Thu == "EXCEPTION")
    {
        Assert.ThrowsException<ArgumentOutOfRangeException>(() =>
        {
            new      HìnhHopChuNhat_63_Thu(dai_63_Thu,      rong_63_Thu,
cao_63_Thu);
        });
        return;
    }

    var h_63_Thu = new HìnhHopChuNhat_63_Thu(dai_63_Thu, rong_63_Thu,
cao_63_Thu);

    double actual_63_Thu = 0;

    if (type_63_Thu == "TheTich")
        actual_63_Thu = h_63_Thu.theTich_63_Thu();
    else if (type_63_Thu == "DienTich")
        actual_63_Thu = h_63_Thu.dienTich_63_Thu();
    else if (type_63_Thu == "ChuVi")
        actual_63_Thu = h_63_Thu.chuVi_63_Thu();

    Assert.AreEqual(expected_63_Thu, actual_63_Thu, 0.001);
}
}

```

Bước 7: Chạy test, kết quả Passed

The screenshot displays a test execution interface with two main panels. The left panel lists test cases, and the right panel shows the execution results for a selected test case.

Test Cases List (Left Panel):

- TC05_tatCabangO
- TC06_63_Thu_SoRatNho
- TC07_63_Thu_SoLon
- TC08_63_Thu_MotCanhAm
- TC09_63_Thu_TatCaCanhAm
- TC10_63_Thu_SaiKieuDuLieu
- Test_HHCN_63_Thu+TestCSV_HHCN_63_Thu (1)
- TC11_TestWithDataSource_63_Thu
- Test_HHCN_63_Thu+TestExcel_HHCN_63_Thu (1)**
- TC12_TestWithDataSource_Excel_63_Thu**

Execution Results (Right Panel):

1) **TC12_TestWithDataSource_Excel_63_Thu (Data Row 0)**
Duration: < 1 ms

2) **TC12_TestWithDataSource_Excel_63_Thu (Data Row 1)**
Duration: < 1 ms

3) **TC12_TestWithDataSource_Excel_63_Thu (Data Row 2)**
Duration: < 1 ms

4) **TC12_TestWithDataSource_Excel_63_Thu (Data Row 3)**
Duration: < 1 ms

5) **TC12_TestWithDataSource_Excel_63_Thu (Data Row 4)**
Duration: < 1 ms

6) **TC12_TestWithDataSource_Excel_63_Thu (Data Row 5)**
Duration: < 1 ms

7) **TC12_TestWithDataSource_Excel_63_Thu (Data Row 6)**
Duration: < 1 ms

8) **TC12_TestWithDataSource_Excel_63_Thu (Data Row 7)**
Duration: < 1 ms

Phần 2: Web Driver

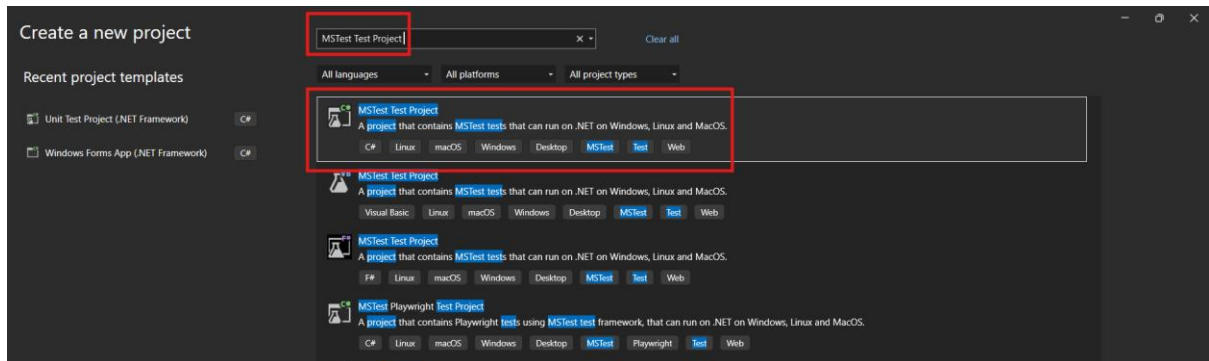
Trang Web: [Demoblaze.com](https://demoblaze.com)

Chức năng:

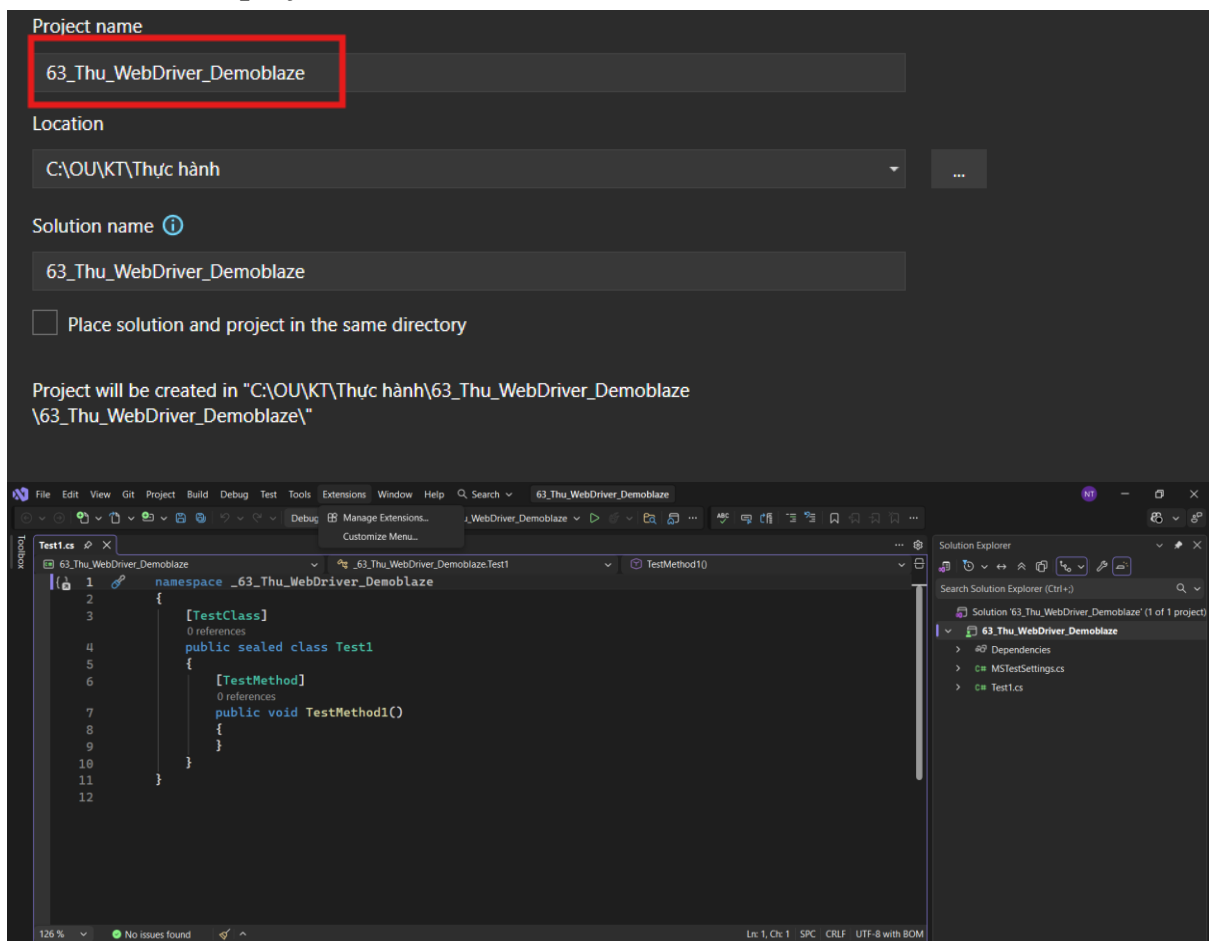
- Kiểm tra chức năng thêm sản phẩm vào giỏ hàng.
- Kiểm tra chức năng xóa sản phẩm khỏi giỏ hàng.

1. Tạo project

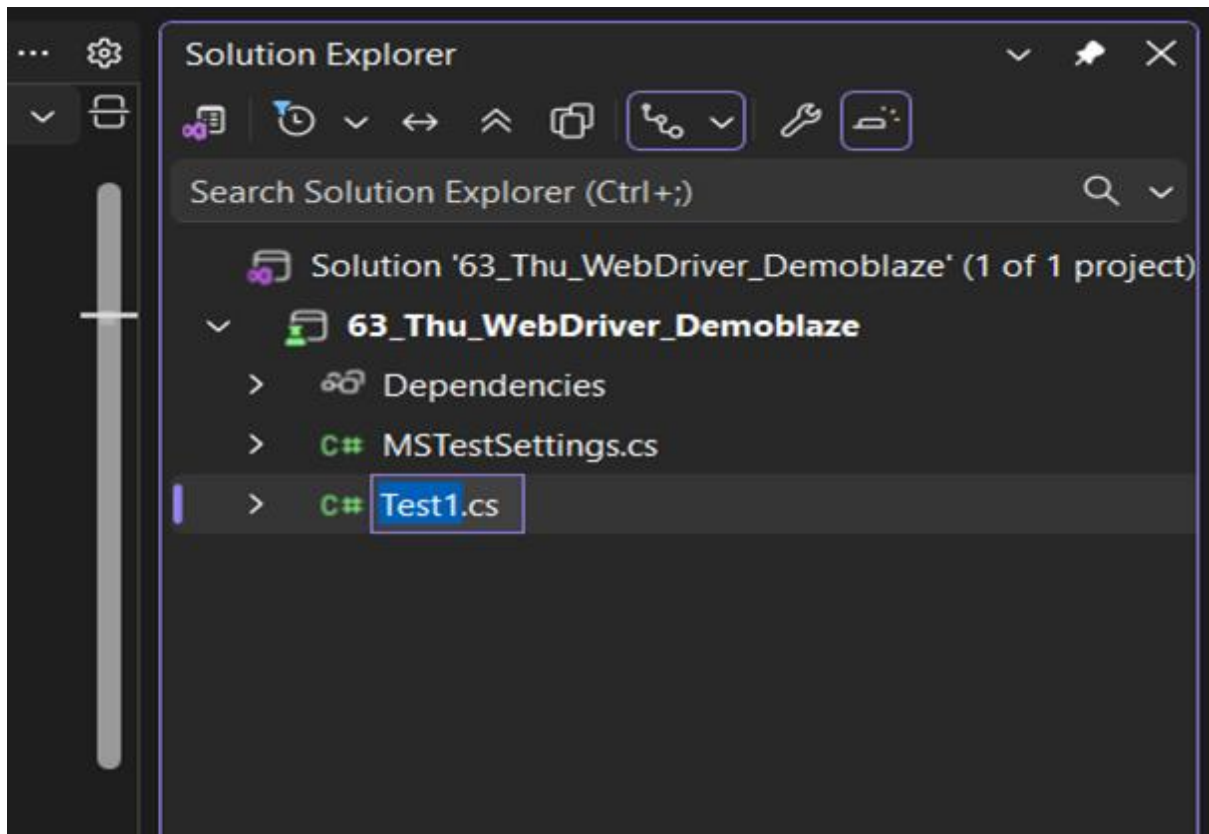
Bước 1: Tạo project MSTest Test Project “63_Thu_WebDriver_Demoblaze”



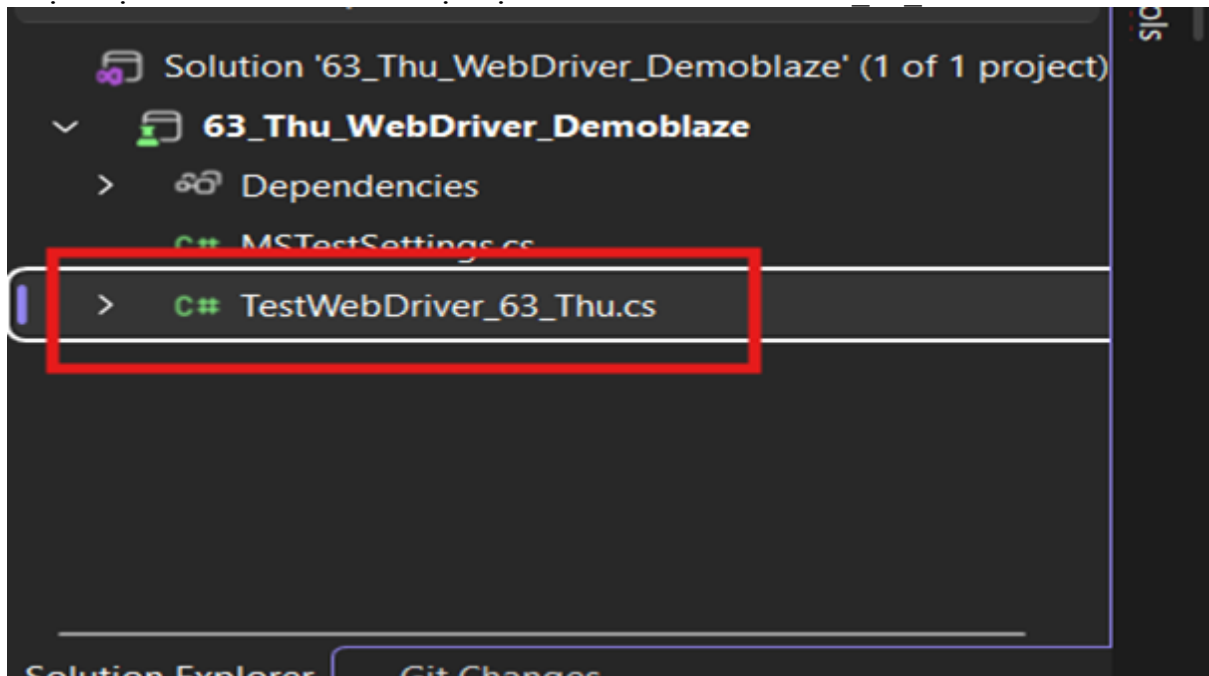
Bước 2: Đặt tên project => Chọn nơi lưu trữ => Next



Bước 3: Tạo Class để test các chức năng

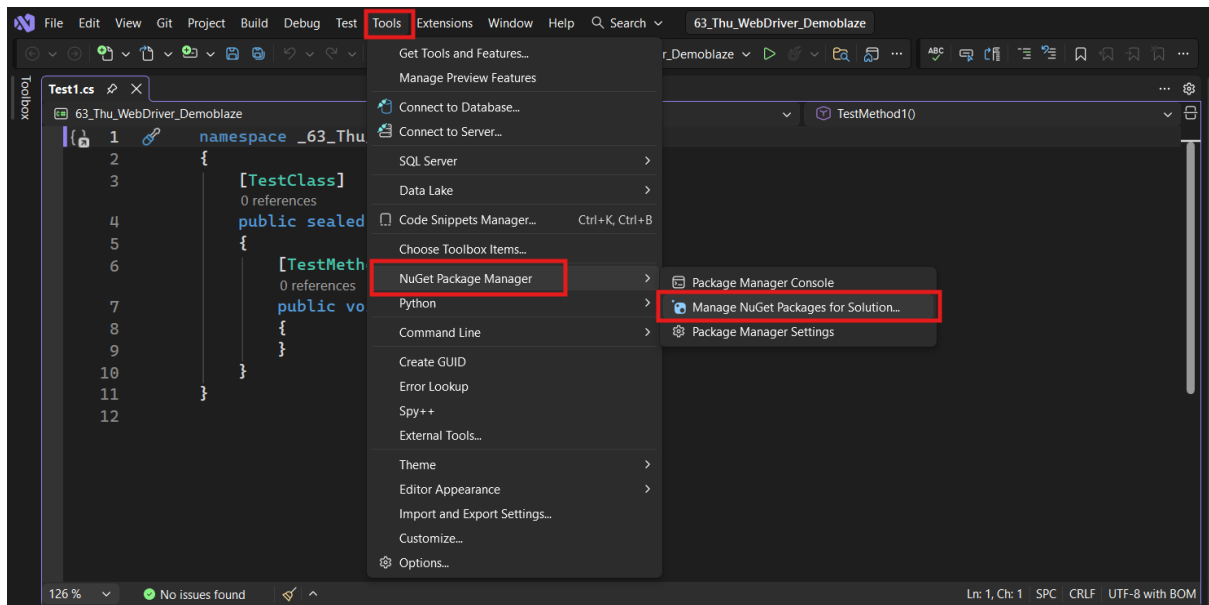


Thực hiện đổi tên Class Test mặc định thành “TestWebRiver 63 Thu”

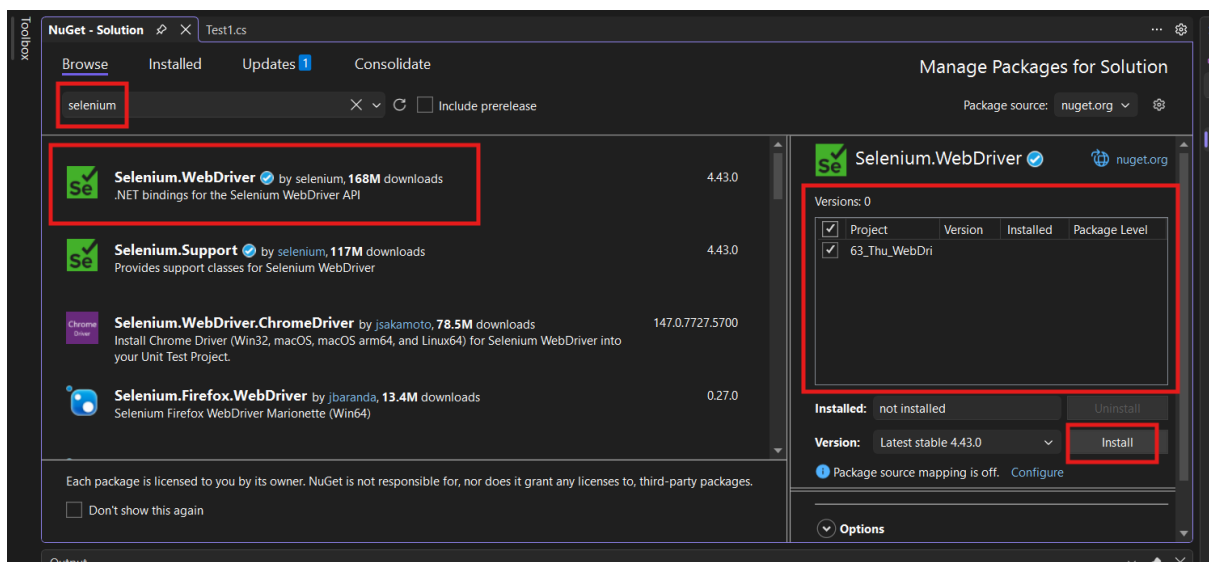


2. Cài đặt thư viện WebDriver

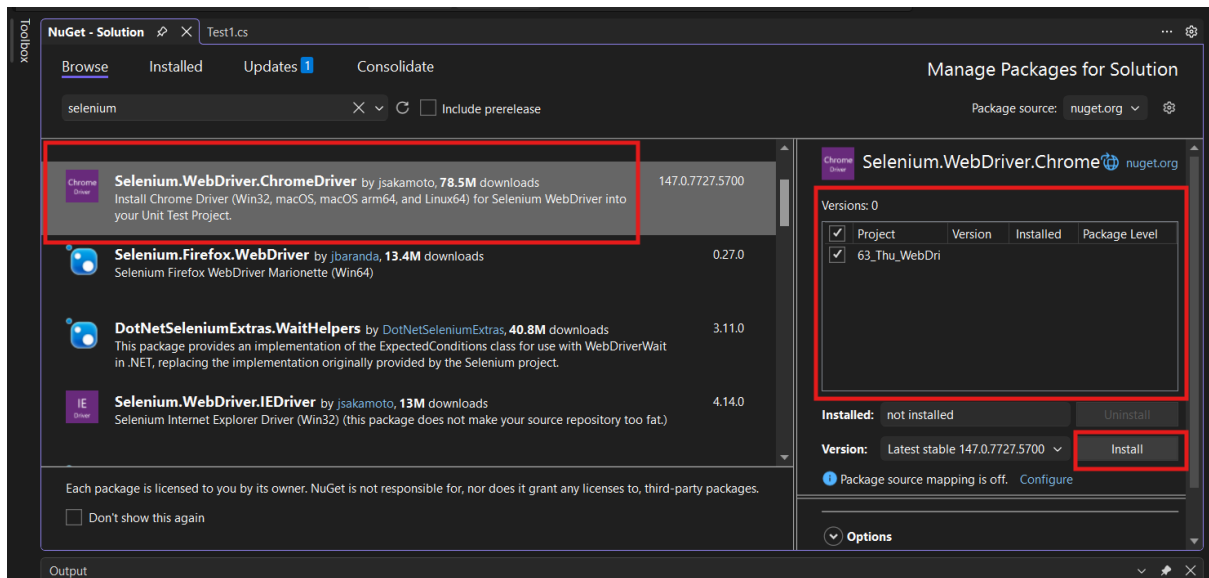
Bước 1: Click phải vào project => Manage NuGet Packages...



Bước 2: Click tab Browse => Tìm từ khóa Selenium.WebDriver => Chọn Selenium.WebDriver => Install



Bước 3: Chọn Selenium.WebDriver.ChromeDriver => Install



3. Mô tả trang web Demoblazed

Demoblaze là một website thương mại điện tử mô phỏng, cung cấp môi trường thực hành kiểm thử các chức năng mua sắm trực tuyến. Trang web cho phép người dùng xem danh sách sản phẩm, xem thông tin chi tiết từng sản phẩm, thêm sản phẩm vào giỏ hàng, xóa sản phẩm khỏi giỏ hàng và thực hiện đặt hàng.

Giao diện trang web được tổ chức theo mô hình mua sắm trực tuyến phổ biến, bao gồm:

- **Trang chủ (Home):** hiển thị danh sách các sản phẩm theo từng danh mục như điện thoại, laptop và màn hình.
- **Trang chi tiết sản phẩm:** hiển thị thông tin chi tiết của sản phẩm như tên, giá và mô tả.
- **Giỏ hàng (Cart):** hiển thị các sản phẩm mà người dùng đã thêm vào giỏ hàng và cho phép xóa sản phẩm.
- **Đặt hàng (Place Order):** hỗ trợ người dùng nhập thông tin để hoàn tất đơn hàng.

Trong bài kiểm thử này, tập trung vào hai chức năng chính:

1. **Thêm sản phẩm vào giỏ hàng (Add to cart)**
2. **Xóa sản phẩm khỏi giỏ hàng (Delete product)**

4. Khởi tạo dữ liệu dùng chung

```
21 driver_63_Thu = new ChromeDriver();
22
23 driver_63_Thu.Manage().Window.Maximize();
24
25 driver_63_Thu.Navigate().GoToUrl("https://www.demoblaze.com/");
26
27 wait_63_Thu = new WebDriverWait(
28     driver_63_Thu,
29     TimeSpan.FromSeconds(2000)
30 );
31
32 // Hàm tìm phần tử
33 31 references 6/6 passing
34 public IWebElement Find_63_Thu(By by)
35 {
36     return wait_63_Thu.Until(
37         d => d.FindElement(by)
38     );
39
40 9 references 5/5 passing
41 public void AcceptAlert_63_Thu()
42 {
43     wait_63_Thu.Until(d =>
44     {
45         try
46         {
47             d.SwitchTo().Alert().Accept();
48             return true;
49         }
50         catch
51         {
52             return false;
53         }
54     });
55
56 // Chạy sau mỗi test
57 [TestCleanup]
58 0 references
59 public void Cleanup_63_Thu()
60 {
61     driver_63_Thu.Quit();
62 }
```

```
driver_63_Thu = new ChromeDriver();

driver_63_Thu.Manage().Window.Maximize();

driver_63_Thu.Navigate().GoToUrl("https://www.demoblaze.com/");

wait_63_Thu = new WebDriverWait(
    driver_63_Thu,
    TimeSpan.FromSeconds(2000)
);
// Hàm tìm phần tử
public IWebElement Find_63_Thu(By by)
{
    return wait_63_Thu.Until(
        d => d.FindElement(by)
    );
}
```

```

public void AcceptAlert_63_Thu()
{
    wait_63_Thu.Until(d =>
    {
        try
        {
            d.SwitchTo().Alert().Accept();
            return true;
        }
        catch
        {
            return false;
        }
    });
}
// Chạy sau mỗi test
[TestCleanup]
public void Cleanup_63_Thu()
{
    driver_63_Thu.Quit();
}

```

5. Chức năng Thêm vào Giỏ hàng

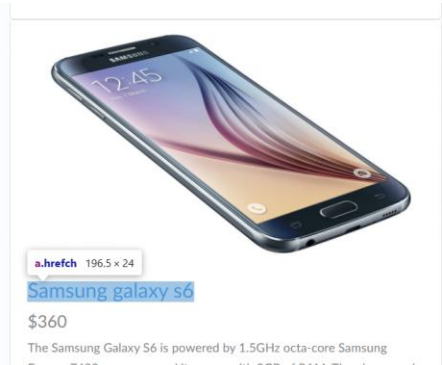
5.1 Mô tả chức năng

Cho phép người dùng lựa chọn sản phẩm từ danh sách hiển thị trên trang chủ và thêm sản phẩm vào giỏ hàng thông qua nút **Add to cart**. Sau khi nhấn nút thêm, hệ thống sẽ kiểm tra trạng thái sản phẩm, lưu thông tin sản phẩm vào giỏ hàng của người dùng và hiển thị thông báo xác nhận **Product added**. Người dùng có thể tiếp tục thêm nhiều sản phẩm khác hoặc truy cập giỏ hàng để kiểm tra danh sách sản phẩm đã chọn.

5.2 Bắt element

5.2.1 Trường Sản phẩm

1. By.LinkText

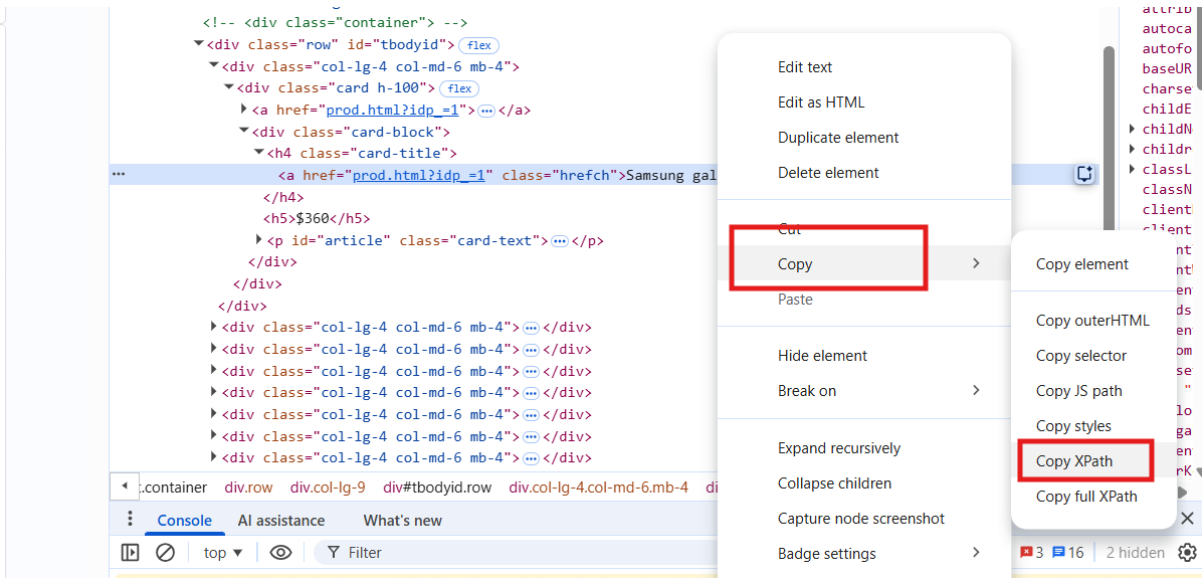


The screenshot shows a Samsung Galaxy S6 smartphone. Below the image, the text "a hrefch 196.5 x 24" is visible, followed by "Samsung galaxy s6" and "\$360". The phone is powered by a 1.5GHz octa-core Samsung processor.

```
<!-- <div class="container"> -->
<div class="row" id="tbodyid">
  <div class="col-lg-4 col-md-6 mb-4">
    <div class="card h-100">
      <div class="card-block">
        <a href="prod.html?idp=1" class="hrefch">Samsung galaxy s6</a>
      </div>
      <h4 class="card-title">
        <h5>$360</h5>
      </h4>
      <p id="article" class="card-text">
    </p>
  </div>
</div>
</div>
</div>
```

Find_63_Thu(By.LinkText("Samsung galaxy s6")).Click();

2. By.XPath



The screenshot shows the same Samsung Galaxy S6 product page. A context menu is open over the link "Samsung galaxy s6". The menu options include "Edit text", "Edit as HTML", "Duplicate element", "Delete element", "Cut", "Copy", "Paste", "Hide element", "Break on", "Expand recursively", "Collapse children", "Capture node screenshot", and "Badge settings". The "Copy" option is highlighted with a red box. A sub-menu is also visible, showing options like "Copy element", "Copy outerHTML", "Copy selector", "Copy JS path", "Copy styles", "Copy XPath", and "Copy full XPath". The "Copy XPath" option is highlighted with a red box.

```
<!-- <div class="container"> -->
<div class="row" id="tbodyid">
  <div class="col-lg-4 col-md-6 mb-4">
    <div class="card h-100">
      <div class="card-block">
        <a href="prod.html?idp=1" class="hrefch">Samsung gal
      </div>
      <h4 class="card-title">
        <h5>$360</h5>
      </h4>
      <p id="article" class="card-text">
    </p>
  </div>
</div>
</div>
</div>
```

Find_63_Thu(By.XPath("//*[@id='tbodyid']/div[1]/div/div/h4/a")).Click();

3. By.ClassName



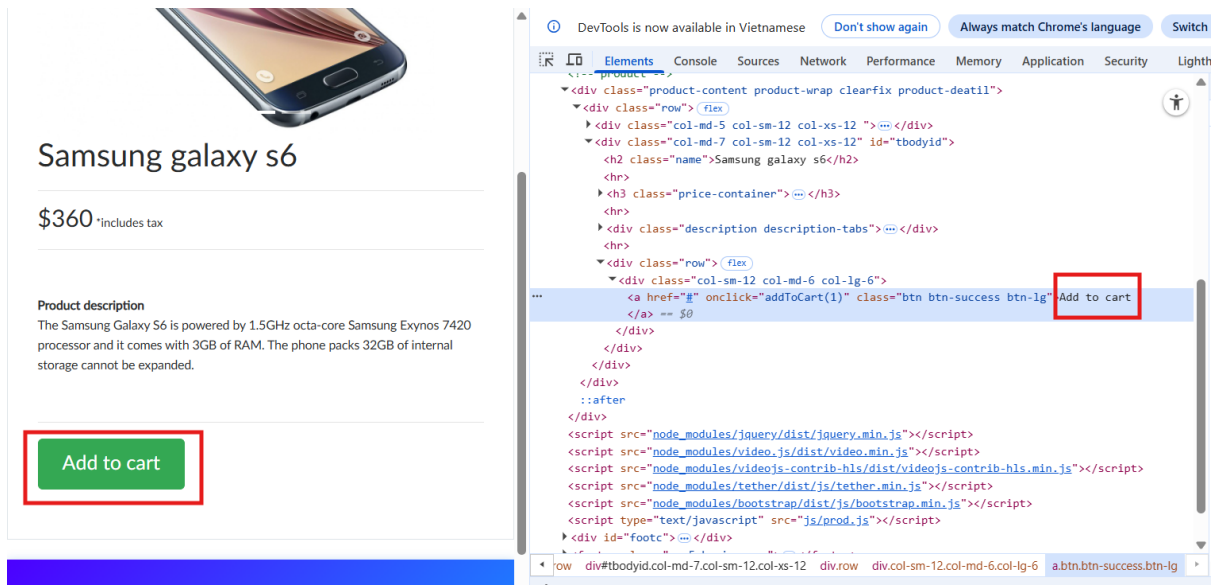
The screenshot shows the same Samsung Galaxy S6 product page. The link "Samsung galaxy s6" is highlighted with a red box. The href attribute of the link is also highlighted with a red box. The href attribute is "prod.html?idp=1".

```
<!-- <div class="container"> -->
<div class="row" id="tbodyid">
  <div class="col-lg-4 col-md-6 mb-4">
    <div class="card h-100">
      <div class="card-block">
        <a href="prod.html?idp=1" class="hrefch">Samsung galaxy s6</a>
      </div>
      <h4 class="card-title">
        <h5>$360</h5>
      </h4>
      <p id="article" class="card-text">
    </p>
  </div>
</div>
</div>
</div>
```

Find_63_Thu(By.ClassName("hrefch")).Click();

5.2.2 Nút Add to cart

1. By.LinkText



The image shows a product page for the Samsung Galaxy S6. The page includes a product image, the name "Samsung galaxy s6", the price "\$360 *includes tax", and a product description. At the bottom, there is a green "Add to cart" button. To the right, the DevTools DOM tree is open, showing the HTML structure. The "Add to cart" button is highlighted with a red box in the DOM tree, with its class attribute being "btn btn-success btn-lg".

Find_63_Thu(By.LinkText("Samsung galaxy s6")).Click();

2. By.PartialLinkText

Một phần của Text, có thể là các từ như sau: "Add", "to", "cart"

Find_63_Th(By.PartialLinkText("Add")).Click();

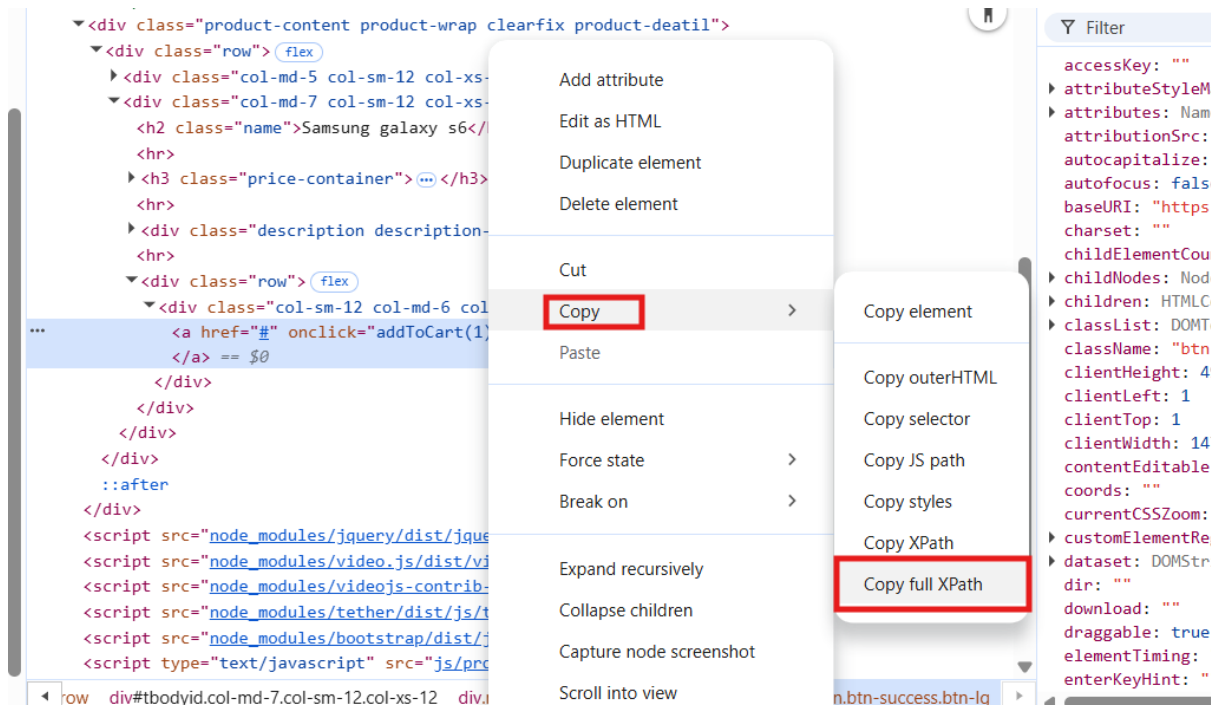
3. By.CssSelector



The image shows the DevTools DOM tree with the "Add to cart" button selected. The button's class attribute is "btn btn-success btn-lg", which is highlighted with a red box. The CSS selector "a.btn.btn-success.btn-lg" is shown in the breadcrumb at the bottom of the DOM tree.

Find_63_Thu(By.CssSelector("a.btn.btn-success.btn-lg")).Click();

4. *By.XPath*

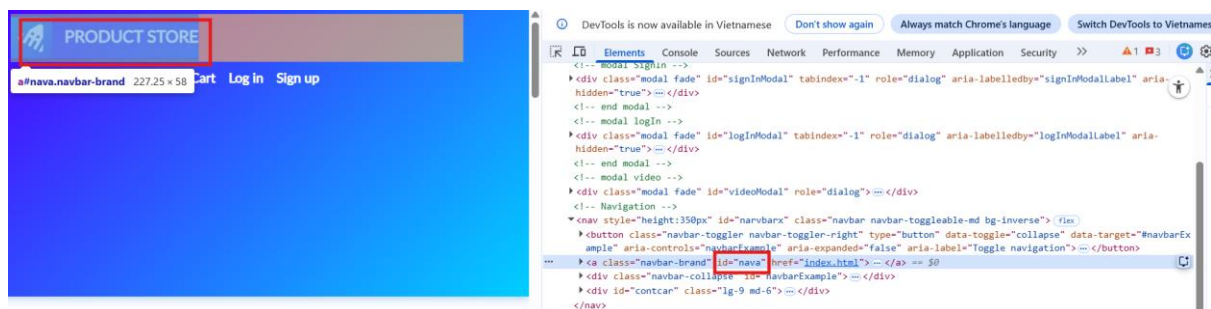


- XPath full của nút là: /html/body/div[5]/div/div[2]/div[2]/div/a

```
Find_63_Thu(By.XPath("/html/body/div[5]/div/div[2]/div[2]/div/a")).Click();
```

5.2.3 Nút Home

1. *By.Id*



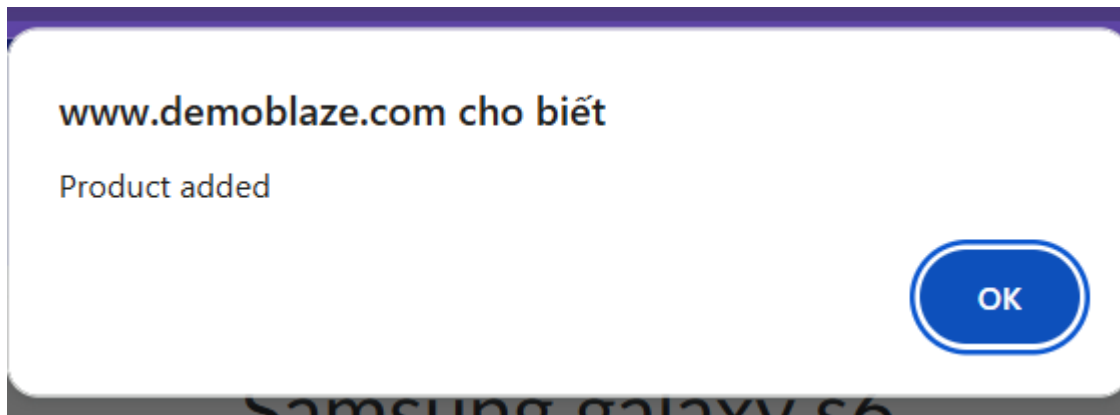
```
Find_63_Thu(By.Id("nava")).Click();
```

5.2.4 Alert xác nhân

1. SwitchTo().Alert()

Trong website Demoblaze, sau khi người dùng nhấn nút Add to cart, hệ thống sẽ hiển thị một hộp thoại thông báo (Alert) với nội dung:

Product added



```
var alert = driver_63_Thu.SwitchTo().Alert();
Assert.AreEqual("Product added", alert.Text);
alert.Accept();
```

Alert là một hộp thoại bật lên của trình duyệt dùng để thông báo kết quả thao tác của người dùng. Vì Alert không thuộc cấu trúc HTML của trang web nên không thể sử dụng các phương thức định vị như:

FindElement()
By.Id()
By.ClassName()
By.XPath()

Trong Selenium, để làm việc với Alert cần sử dụng phương thức:

SwitchTo().Alert()

Phương thức này giúp chuyển quyền điều khiển từ trang web hiện tại sang hộp thoại Ví dụ trong bài:

```
var alert = driver_63_Thu.SwitchTo().Alert();
Assert.AreEqual("Product added", alert.Text);
alert.Accept();
```

Ý nghĩa:

SwitchTo().Alert() : chuyển sang Alert

alert.Text : lấy nội dung thông báo

alert.Accept() : nhấn nút OK để đóng Alert

Việc xử lý Alert là cần thiết vì nếu Alert chưa được đóng, hệ thống sẽ không cho phép thực hiện các thao tác tiếp theo trên trang web.

5.3 Thực hiện test case

5.3.1 Bảng test case

STT	TestCaseID	Sản phẩm	Thao tác	Kết quả mong đợi	Kết quả thực tế	Trạng thái
1	TC1_63_Thu_ThemSanPhamThanhCon	Samsung galaxy s6	Thêm 1 sản phẩm vào giỏ	Hiển thị thông báo "Product	Hiển thị alert "Product	Pass

	g			added”	added”	
2	TC2_63_Thu_ThemNhiềuSanPham	Samsung galaxy s6, Nokia lumia 1520	Thêm 2 sản phẩm khác nhau vào giỏ	Cả 2 sản phẩm được thêm thành công	Cả 2 sản phẩm hiển thị trong giỏ hàng	Pass
3	TC3_63_Thu_ThemTrungSanPham	Samsung galaxy s6	Thêm cùng sản phẩm 2 lần	Hệ thống cho phép thêm trùng	Sản phẩm được thêm 2 lần vào giỏ	Pass

5.3.2 Viết code cho các test case

1. TC1_63_Thu_ThemSanPhamThanhCong()

```

60 [TestMethod]
61 // TC1: Thêm sản phẩm "Samsung galaxy s6" thành công
62 public void TC1_63_Thu_ThemSanPhamThanhCong()
63 {
64     Find_63_Thu(By.LinkText("Samsung galaxy s6")).Click();
65
66     Find_63_Thu(By.LinkText("Add to cart")).Click();
67
68     var alert = wait_63_Thu.Until(d =>
69     {
70         try
71         {
72             return d.SwitchTo().Alert();
73         }
74         catch
75         {
76             return null;
77         }
78     });
79     Thread.Sleep(3000);
80     AcceptAlert_63_Thu();
81 }

```

```

[TestMethod]
// TC1: Thêm sản phẩm "Samsung galaxy s6" thành công
public void TC1_63_Thu_ThemSanPhamThanhCong()
{
    Find_63_Thu(By.LinkText("Samsung galaxy s6")).Click();

    Find_63_Thu(By.LinkText("Add to cart")).Click();

    var alert = wait_63_Thu.Until(d =>

```

```

    {
        try
        {
            return d.SwitchTo().Alert();
        }
        catch
        {
            return null;
        }
    });
    Thread.Sleep(3000);
    AcceptAlert_63_Thu();
}

```

2. TC2_63_Thu_ThemNhieuSanPham()

```

83 [TestMethod]
84 // TC2: Thêm 2 sản phẩm: "Samsung galaxy s6" và "Nokia lumia 1520"
85 // 0 references
86 public void TC2_63_Thu_ThemNhieuSanPham()
87 {
88     // Sản phẩm 1
89     Find_63_Thu(By.LinkText("Samsung galaxy s6")).Click();
90     Find_63_Thu(By.LinkText("Add to cart")).Click();
91     AcceptAlert_63_Thu();
92     // Quay về Home
93     Find_63_Thu(By.Id("nava")).Click();
94     // Sản phẩm 2
95     Find_63_Thu(By.LinkText("Nokia lumia 1520")).Click();
96     Find_63_Thu(By.LinkText("Add to cart")).Click();
97     AcceptAlert_63_Thu();
98     // Vào giỏ hàng
99     Find_63_Thu(By.Id("cartur")).Click();
100    Thread.Sleep(3000);

```

```

[TestMethod]
// TC2: Thêm 2 sản phẩm: "Samsung galaxy s6" và "Nokia lumia 1520"
public void TC2_63_Thu_ThemNhieuSanPham()
{
    // Sản phẩm 1
    Find_63_Thu(By.LinkText("Samsung galaxy s6")).Click();
    Find_63_Thu(By.LinkText("Add to cart")).Click();
    AcceptAlert_63_Thu();
    // Quay về Home
    Find_63_Thu(By.Id("nava")).Click();
    // Sản phẩm 2
    Find_63_Thu(By.LinkText("Nokia lumia 1520")).Click();
    Find_63_Thu(By.LinkText("Add to cart")).Click();
    AcceptAlert_63_Thu();
}

```

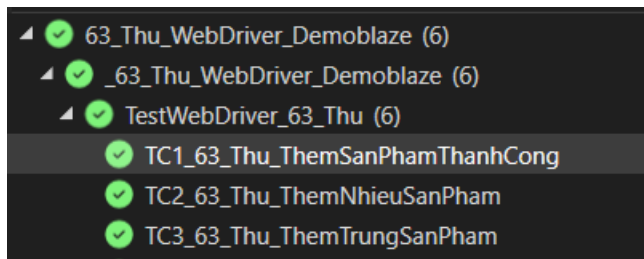
```
// Vào giỏ hàng
Find_63_Thu(By.Id("cartur")).Click();
Thread.Sleep(3000);
}
```

3. TC3_63_Thu_ThemTrungSanPham()

```
102 [TestMethod]
103 // TC3: Thêm trùng sản phẩm "Samsung galaxy s6"
104 // 0 references
105 public void TC3_63_Thu_ThemTrungSanPham()
106 {
107     Find_63_Thu(By.LinkText("Samsung galaxy s6")).Click();
108     Find_63_Thu(By.LinkText("Add to cart")).Click();
109     AcceptAlert_63_Thu();
110     // Quay về Home
111     Find_63_Thu(By.Id("nava")).Click();
112     // Thêm lại cùng sản phẩm
113     Find_63_Thu(By.XPath("//*[@id='tbodyid']/div[1]/div/div/h4/a")).Click();
114     Find_63_Thu(By.LinkText("Add to cart")).Click();
115     AcceptAlert_63_Thu();
116     // Mở giỏ hàng
117     Find_63_Thu(By.Id("cartur")).Click();
118     Thread.Sleep(3000);
119 }
```

```
[TestMethod]
// TC3: Thêm trùng sản phẩm "Samsung galaxy s6"
public void TC3_63_Thu_ThemTrungSanPham()
{
    Find_63_Thu(By.LinkText("Samsung galaxy s6")).Click();
    Find_63_Thu(By.LinkText("Add to cart")).Click();
    AcceptAlert_63_Thu();
    // Quay về Home
    Find_63_Thu(By.Id("nava")).Click();
    // Thêm lại cùng sản phẩm
    Find_63_Thu(By.XPath("//*[@id='tbodyid']/div[1]/div/div/h4/a")).Click();
    Find_63_Thu(By.LinkText("Add to cart")).Click();
    AcceptAlert_63_Thu();
    // Mở giỏ hàng
    Find_63_Thu(By.Id("cartur")).Click();
    Thread.Sleep(3000);
}
```

5.4 Kết quả chạy test 3 passed, 0 fail



6. Chức năng Xóa khỏi Giỏ hàng

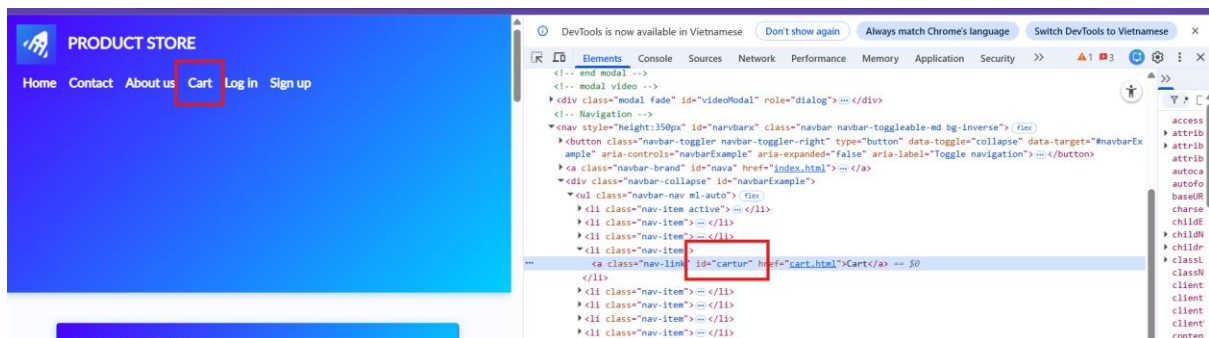
6.1 Mô tả chức năng

Cho phép người dùng quản lý giỏ hàng bằng cách xóa sản phẩm đã thêm trước đó thông qua nút **Delete** tương ứng với từng sản phẩm. Sau khi thực hiện thao tác xóa, hệ thống sẽ cập nhật lại danh sách sản phẩm trong giỏ hàng, loại bỏ sản phẩm đã chọn và hiển thị trạng thái giỏ hàng mới nhất để người dùng tiếp tục mua sắm hoặc thực hiện thanh toán.

6.2 Bắt element

6.2.1 Nút Cart

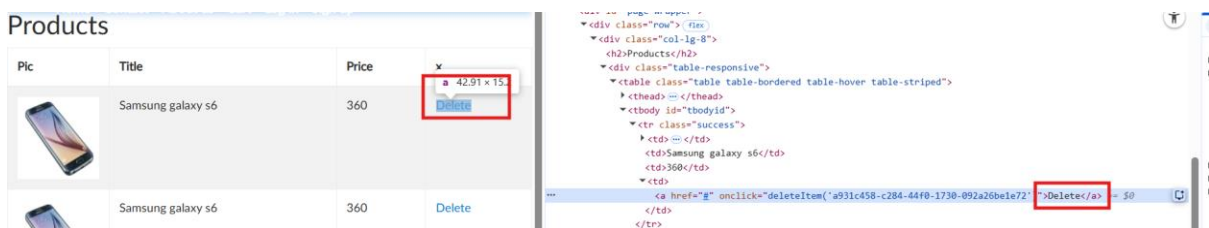
1. By.Id



Find_63_Thu(By.Id("cartur")).Click();

6.2.2 Nút Delete

1. By.LinkText



Find_63_Thu(By.LinkText("Delete")).Click();

2. By.XPath



Find_63_Thu(By.XPath("//*[@id=\"tbodyid\"/tr/td[4]/a")).Click();

3. By.TagName (nhiều sản phẩm)

Do có cùng TagName là <a>, nên phải kết hợp với Text để bắt đúng element



```
// Lấy tất cả thẻ <a>
var links = Find_63_Thu(By.TagName("a"));
// Duyệt từng thẻ <a>
foreach (var link in links)
{
    // Kiểm tra nếu text là "Delete"
    if (link.Text.Trim() == "Delete")
    {
        link.Click(); // Click nút Delete
        break; // Thoát vòng lặp sau khi click
    }
}
```

6.3 Thực hiện test case

6.3.1 Bảng test case

STT	TestCaseID	Sản phẩm	Thao tác	Kết quả mong đợi	Kết quả thực tế	Trạng thái
4	TC4_63_Thu_XoaSanPhamTonTai	Samsung galaxy s6	Xóa sản phẩm khỏi giỏ	Sản phẩm bị xóa khỏi giỏ hàng	Sản phẩm không còn trong giỏ	Pass
5	TC5_63_Thu_ThemHaiSanPhamTrungVaXoaMotSanPham	Samsung galaxy s6	Thêm 2 sản phẩm trùng và xóa 1 sản phẩm	Hệ thống xóa đúng 1 sản phẩm, còn lại 1 sản phẩm	1 sản phẩm bị xóa, 1 sản phẩm còn lại	Pass
6	TC6_63_Thu_XoaMotSanPhamTrongNhieuSanPham	Samsung galaxy s6, Nokia lumia 1520	Thêm 2 sản phẩm khác nhau và xóa 1 sản phẩm	Chỉ xóa sản phẩm được chọn	1 sản phẩm bị xóa, sản phẩm còn lại vẫn giữ nguyên	Pass

6.3.2 Viết code cho các test case

1. TC4_63_Thu_XoaSanPhamTonTai()

```

119 [TestMethod]
120 // TC4: Xóa sản phẩm "Samsung galaxy s6" tồn tại
121 // 0 references
122 public void TC4_63_Thu_XoaSanPhamTonTai()
123 {
124     Find_63_Thu(By.LinkText("Samsung galaxy s6")).Click();
125     Find_63_Thu(By.LinkText("Add to cart")).Click();
126     AcceptAlert_63_Thu();
127     Find_63_Thu(By.Id("cartur")).Click();
128     Find_63_Thu(By.LinkText("Delete")).Click();
129     Thread.Sleep(3000);
130 }

```

```

[TestMethod]
// TC4: Xóa sản phẩm "Samsung galaxy s6" tồn tại
public void TC4_63_Thu_XoaSanPhamTonTai()
{
    Find_63_Thu(By.LinkText("Samsung galaxy s6")).Click();
    Find_63_Thu(By.LinkText("Add to cart")).Click();
    AcceptAlert_63_Thu();
    Find_63_Thu(By.Id("cartur")).Click();
    Find_63_Thu(By.LinkText("Delete")).Click();
    Thread.Sleep(3000);
}

```

2. TC5_63_Thu_ThemHaiSanPhamTrungVaXoaMotSanPham()

```
130 [TestMethod]
131 // TC5: Thêm 2 sản phẩm "Samsung galaxy s6" và xóa sản phẩm đầu tiên
132 public void TC5_63_Thu_ThemHaiSanPhamTrungVaXoaMotSanPham()
133 {
134     // thêm lần 1
135     Find_63_Thu(By.LinkText("Samsung galaxy s6")).Click();
136     Find_63_Thu(By.LinkText("Add to cart")).Click();
137     AcceptAlert_63_Thu();
138     // về home
139     Find_63_Thu(By.Id("nava")).Click();
140     // thêm lần 2
141     Find_63_Thu(By.LinkText("Samsung galaxy s6")).Click();
142     Find_63_Thu(By.LinkText("Add to cart")).Click();
143     AcceptAlert_63_Thu();
144     // mở cart
145     Find_63_Thu(By.Id("cartur")).Click();
146     Thread.Sleep(3000);
147     // lấy danh sách nút delete
148     var nutXoa = driver_63_Thu.FindElements(By.LinkText("Delete"));
149     // xóa sản phẩm đầu tiên
150     nutXoa[0].Click();
151     Thread.Sleep(3000);
152 }
```

[TestMethod]

// TC5: Thêm 2 sản phẩm "Samsung galaxy s6" và xóa sản phẩm đầu tiên

public void TC5_63_Thu_ThemHaiSanPhamTrungVaXoaMotSanPham()

{

// thêm lần 1

Find_63_Thu(By.LinkText("Samsung galaxy s6")).Click();

Find_63_Thu(By.LinkText("Add to cart")).Click();

AcceptAlert_63_Thu();

// về home

Find_63_Thu(By.Id("nava")).Click();

// thêm lần 2

Find_63_Thu(By.LinkText("Samsung galaxy s6")).Click();

Find_63_Thu(By.LinkText("Add to cart")).Click();

AcceptAlert_63_Thu();

// mở cart

Find_63_Thu(By.Id("cartur")).Click();

Thread.Sleep(3000);

// lấy danh sách nút delete

var nutXoa = driver_63_Thu.FindElements(By.LinkText("Delete"));

// xóa sản phẩm đầu tiên

nutXoa[0].Click();

Thread.Sleep(3000);

}

3. TC6_63_Thu_XoaMotSanPhamTrongNhiềuSanPham()

```
153 [TestMethod]
154 // TC6: Thêm 2 sản phẩm khác nhau "Samsung galaxy s6" và "Nokia Lumia 1520"
155 public void TC6_63_Thu_XoaMotSanPhamTrongNhiềuSanPham()
156 {
157     // Sản phẩm 1
158     Find_63_Thu(By.LinkText("Samsung galaxy s6")).Click();
159     Find_63_Thu(By.LinkText("Add to cart")).Click();
160     AcceptAlert_63_Thu();
161     // Quay về Home
162     Find_63_Thu(By.Id("nava")).Click();
163     // Sản phẩm 2
164     Find_63_Thu(By.LinkText("Nokia lumia 1520")).Click();
165     Find_63_Thu(By.LinkText("Add to cart")).Click();
166     AcceptAlert_63_Thu();
167     // Mở giỏ hàng
168     Find_63_Thu(By.Id("cartur")).Click();
169     // Xóa đúng sản phẩm "Nokia Lumia 1520"
170     Find_63_Thu(By.XPath("//td[text()='Nokia lumia 1520']/following-sibling::td/a")).Click();
171     Thread.Sleep(3000);
172 }
```

```
[TestMethod]
// TC6: Thêm 2 sản phẩm khác nhau "Samsung galaxy s6" và "Nokia Lumia 1520"
public void TC6_63_Thu_XoaMotSanPhamTrongNhiềuSanPham()
{
    // Sản phẩm 1
    Find_63_Thu(By.LinkText("Samsung galaxy s6")).Click();
    Find_63_Thu(By.LinkText("Add to cart")).Click();
    AcceptAlert_63_Thu();
    // Quay về Home
    Find_63_Thu(By.Id("nava")).Click();
    // Sản phẩm 2
    Find_63_Thu(By.LinkText("Nokia lumia 1520")).Click();
    Find_63_Thu(By.LinkText("Add to cart")).Click();
    AcceptAlert_63_Thu();
    // Mở giỏ hàng
    Find_63_Thu(By.Id("cartur")).Click();
    // Xóa đúng sản phẩm "Nokia Lumia 1520"
    Find_63_Thu(By.XPath("//td[text()='Nokia lumia 1520']/following-sibling::td/a")).Click();
    Thread.Sleep(3000);
}
```

Để xóa đúng sản phẩm Nokia lumia 1520, sử dụng XPath:

Find_63_Thu(
By.XPath("//td[text()='Nokia lumia 1520']/following-sibling::td/a")
).Click();

Giải thích từng phần:

1. //td[text()='Nokia lumia 1520']

Tìm ô (td) có nội dung đúng bằng:

Nokia lumia 1520

để xác định đúng dòng chứa sản phẩm cần xóa.

2. */following-sibling::td*

Di chuyển sang ô (td) kế bên cùng hàng.

Trong bảng giỏ hàng, ô kế bên chứa nút:

Delete

3. */a*

Chọn thẻ liên kết (a) bên trong ô đó để thực hiện thao tác xóa.

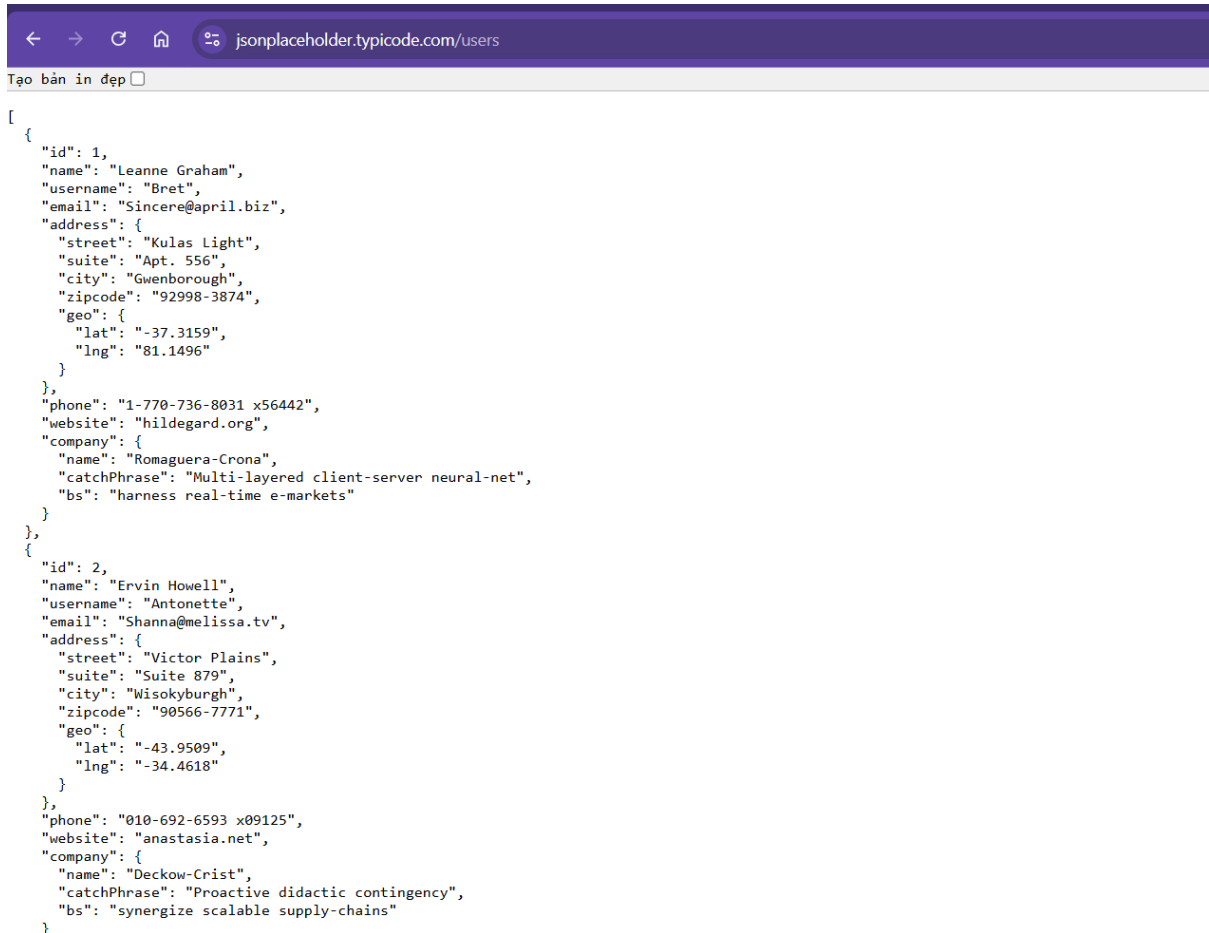
6.4 *Kết quả chạy test 3 passed, 0 fail*

✓ TC4_63_Thu_XoaSanPhamTonTai	9.4 sec
✓ TC5_63_Thu_ThemHaiSanPhamTrungVaXoaMotSanPham	13.6 sec
✓ TC6_63_Thu_XoaMotSanPhamTrongNhiềuSanPham	11.6 sec

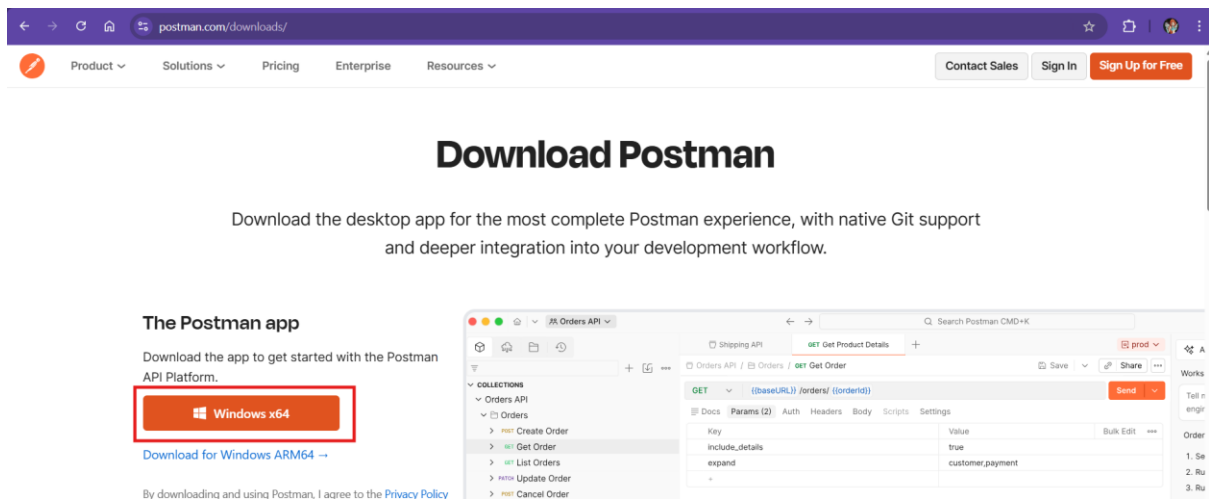
Phần 3: API PostMan

1. Test API trên Postman của Jsonplace

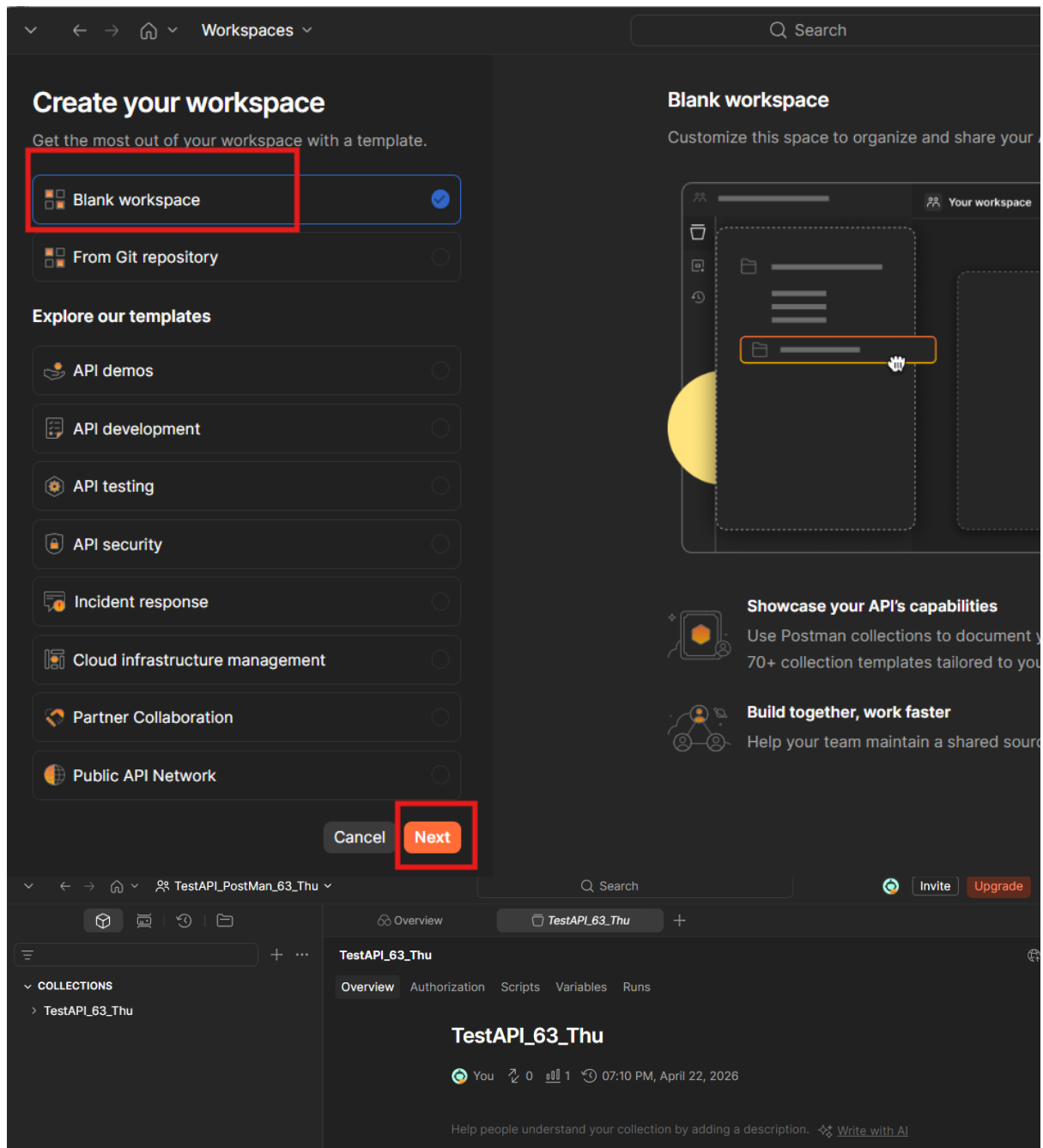
- Ở mục này chúng ta sẽ sử dụng API test của web <https://jsonplaceholder.typicode.com/>



Bước 1: Truy cập vào trang web: <https://www.postman.com/downloads/>



Bước 2: Tạo Workspace



Bước 3: Đặt tên Workspace là: "TestAPI_PostMan_63_Thu"

Workspaces

Create your workspace

Name

TestAPI_PostMan_63_Thu

Select workspace type

 Internal

Build and test APIs within your team.



 Partner

Share APIs securely with trusted partners.

[Upgrade plan](#)

 Public

Make APIs accessible to the world.

☐

Manage access

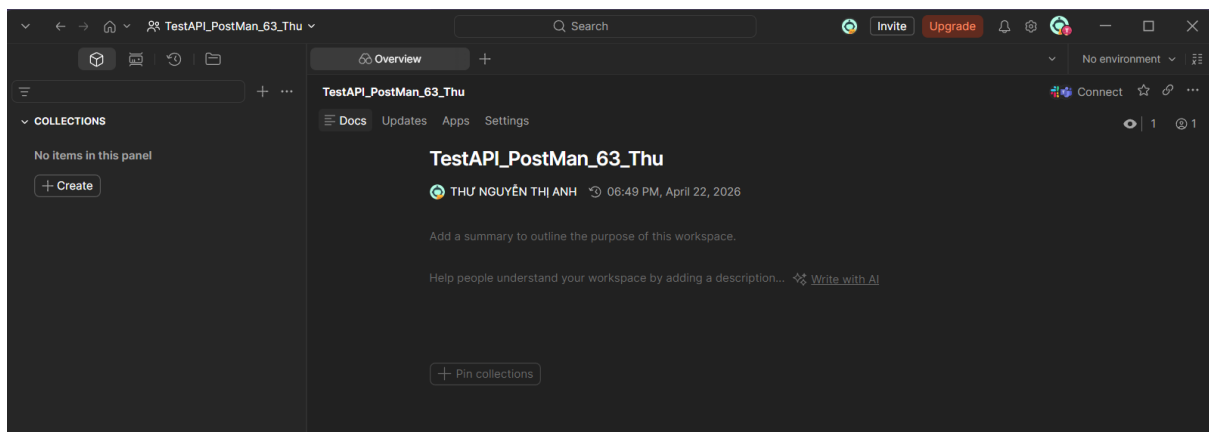
☒ Everyone in THƯ NGUYỄN THỊ ANH

1 member

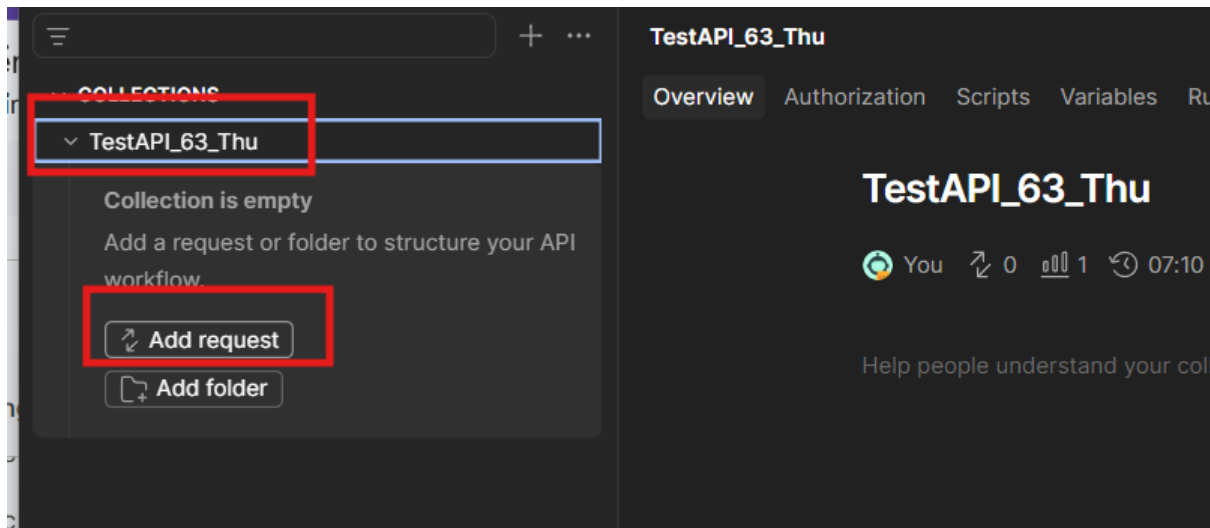
☐ Only you and invited people

Back

Create



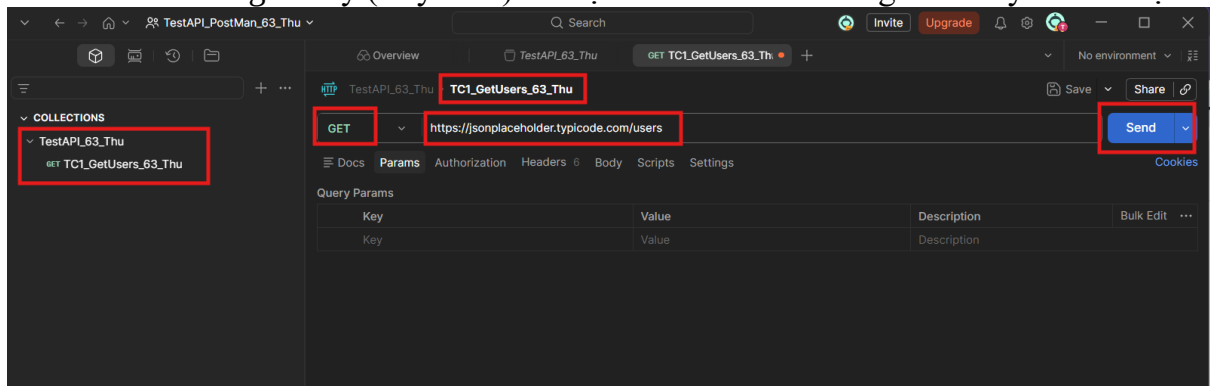
Bước 4: Tạo Collection mới đặt tên là: “TestAPI_63_Thu”

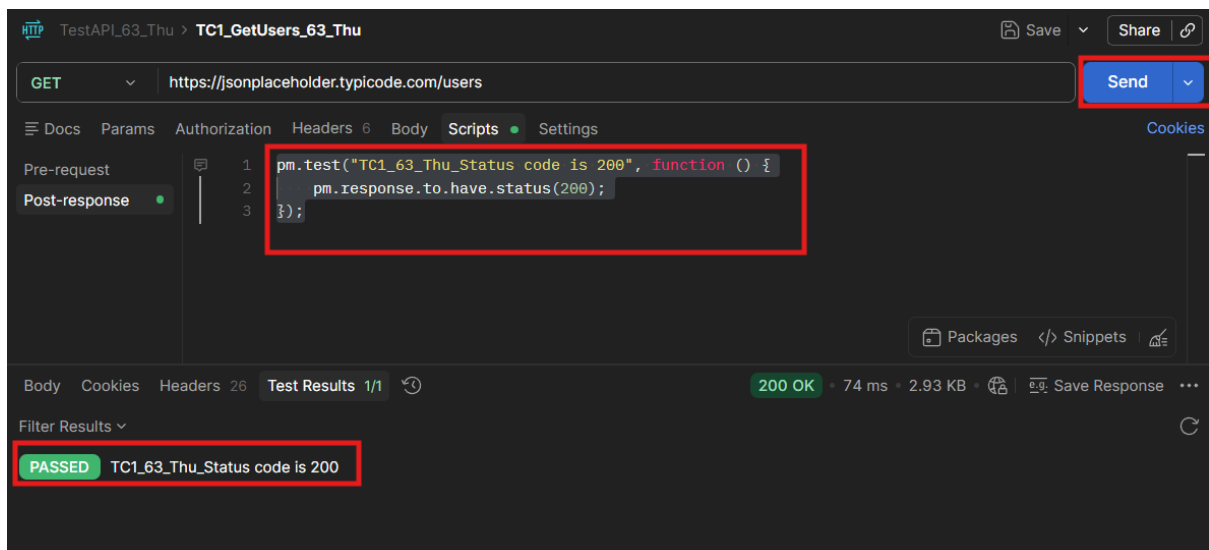
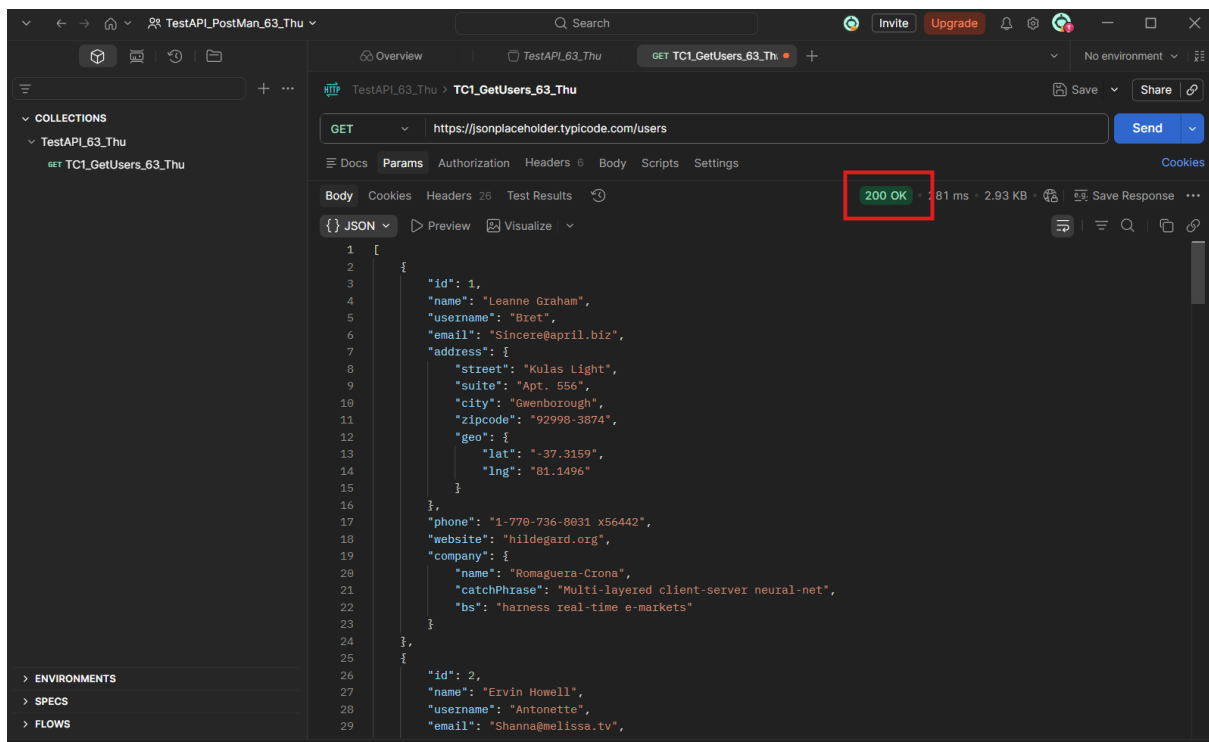


Bước 5: Thực hiện Test với các Phương Thức

5.1 Phương thức GET

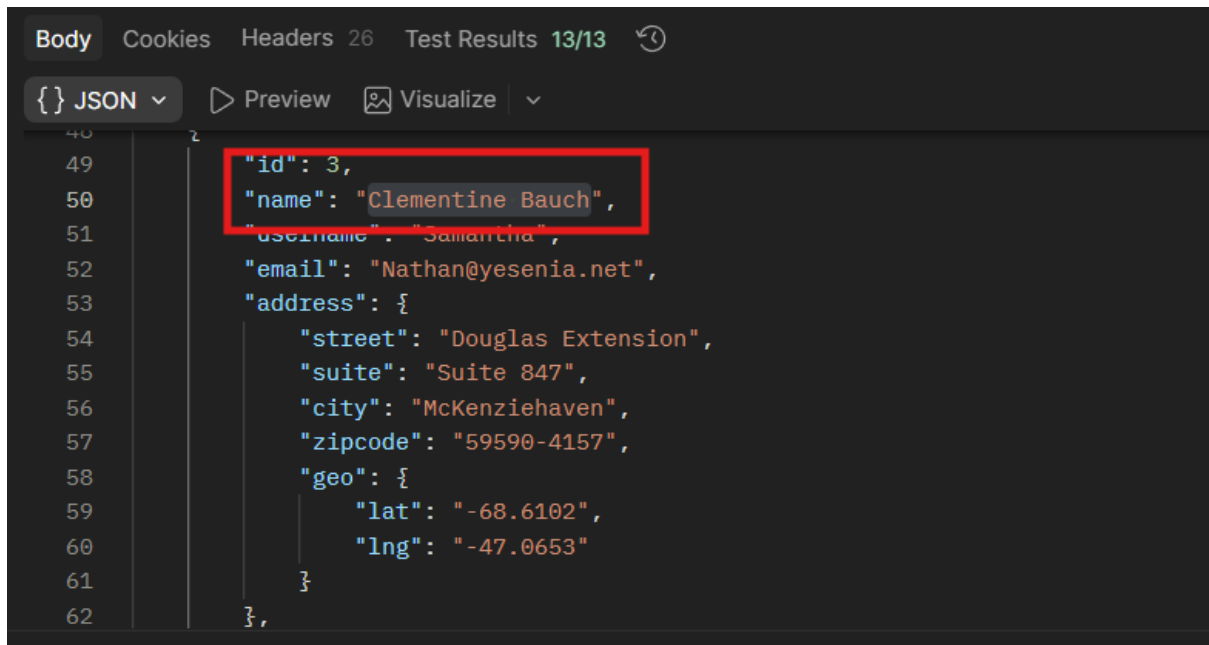
- GET: Dùng để lấy (truy xuất) dữ liệu từ server mà không làm thay đổi dữ liệu





1. TC1_63_Thu_Status code is 200

```
pm.test("TC1_63_Thu_Status code is 200", function () {
  pm.response.to.have.status(200);
});
```



2. TC2_63_Thu_Body matches string

```
// TC2_63_Thu_Body matches string
pm.test("TC2_63_Thu_Body matches string", function () {
  pm.expect(pm.response.text()).to.include("Clementine Bauch");
});
```



3. TC3_63_Thu_Response contains McKenziehaven

```
//TC3_63_Thu_Response contains McKenziehaven
pm.test("TC3_63_Thu_Response contains McKenziehaven", function () {
  pm.expect(pm.response.text()).to.include("McKenziehaven"); // body chứa chuỗi này
});
```

PASSED TC1_63_Thu_Status code is 200

PASSED TC2_63_Thu_Body matches string

PASSED TC3_63_Thu_Response contains McKenziehaven

4. TC4_63_Thu_Response is JSON array

```
//TC4_63_Thu_Response is JSON array
pm.test("TC4_63_Thu_Response is JSON array", function () {
  let jsonData = pm.response.json(); // chuyển response sang JSON
  pm.expect(jsonData).to.be.an("array"); // kiểm tra kiểu là mảng
});
```

PASSED TC4_63_Thu_Response is JSON array

Do có dạng:

```
[
  {...},
  {...}
]
```

Dấu [] = array → PASS

5. TC5_63_Thu_Array

```
//TC5_63_Thu_Array
pm.test("TC5_63_Thu_Array length > 0", function () {
  let jsonData = pm.response.json();
  pm.expect(jsonData.length).to.be.above(0); // mảng có ít nhất 1 phần tử
});
```



```
209      {
210        "id": 10,
211        "name": "Clementina DuBuque",
212        "username": "Moriah.Stanton",
213        "email": "Rey.Padberg@karina.biz",
214        "address": {
215          "street": "Kattie Turnpike",
216          "suite": "Suite 198",
217          "city": "Lebsackbury",
218          "zipcode": "31428-2261",
219          "geo": {
220            "lat": "-38.2386",
```

Connect Git Console Terminal

PASSED TC5_63_Thu_Array length > 0

10 user -> length = 10 -> PASS

6. TC6_63_Thu_Check first user info

```
//TC6_63_Thu_Check first user info
pm.test("TC6_63_Thu_Check first user info", function () {
  let jsonData = pm.response.json();

  pm.expect(jsonData[0].id).to.eql(1); // id user đầu tiên = 1
  pm.expect(jsonData[0].name).to.be.a("string"); // name là chuỗi
  pm.expect(jsonData[0].email).to.include("@"); // email hợp lệ (có @)
});
```

```
"id": 1,
"name": "Leanne Graham",
"username": "Bret",
"email": "Sincere@april.biz",
"address": {
```

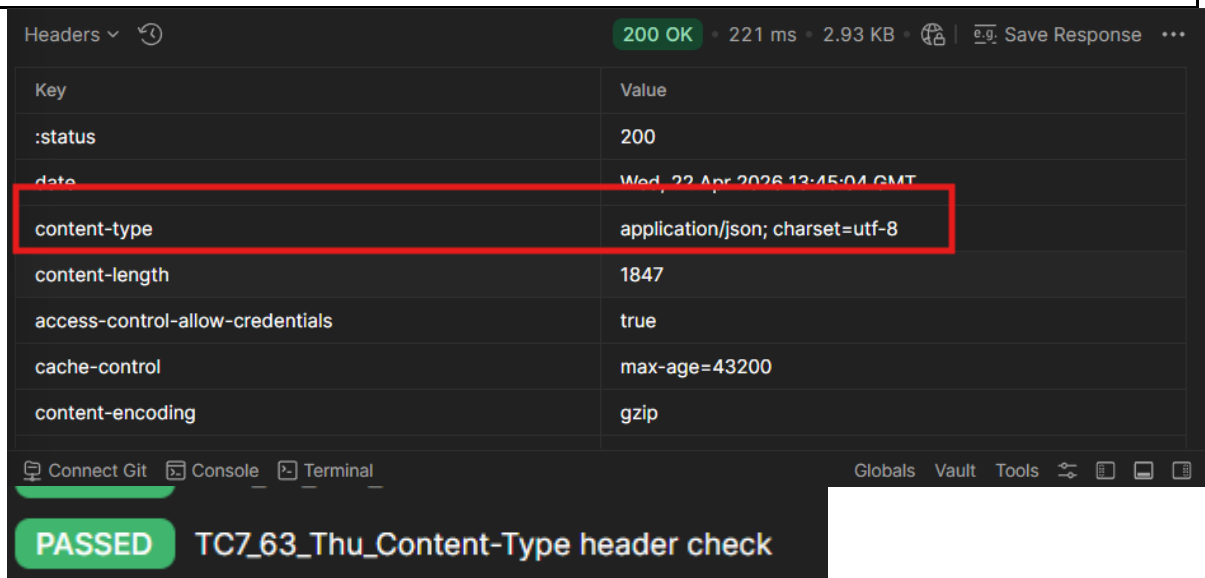
PASSED TC6_63_Thu_Check first user info

7. TC7_63_Thu_Content_Type

```
//TC7_63_Thu_Content_Type
pm.test("TC7_63_Thu_Content-Type header check", function () {
  pm.response.to.have.header("Content-Type"); // kiểm tra có header

  pm.expect(pm.response.headers.get("Content-Type"))
```

```
.to.include("application/json"); // header là JSON
});
```

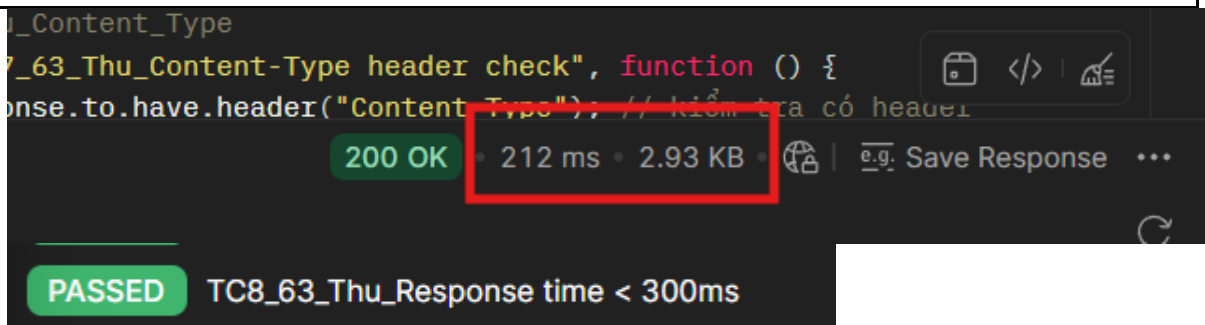


Key	Value
:status	200
date	Wed, 22 Apr 2026 13:45:04 GMT
content-type	application/json; charset=utf-8
content-length	1847
access-control-allow-credentials	true
cache-control	max-age=43200
content-encoding	gzip

PASSED TC7_63_Thu_Content-Type header check

8. TC8_63_Thu_Response time

```
//TC8_63_Thu_Response time
pm.test("TC8_63_Thu_Response time < 300ms", function () {
  pm.expect(pm.response.responseTime).to.be.below(300); // thời gian phản hồi < 300ms
});
```



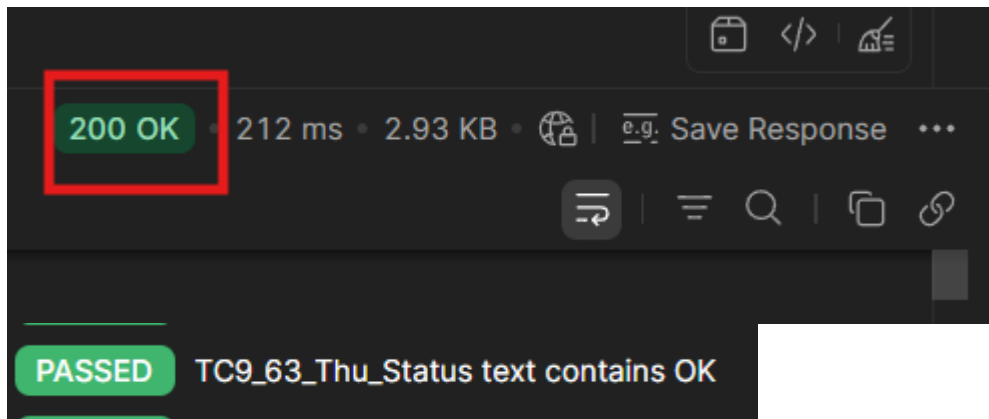
TC7_63_Thu_Content-Type header check, function () {
pm.response.to.have.header("Content-Type"); // kiểm tra có header

200 OK • 212 ms • 2.93 KB • Save Response

PASSED TC8_63_Thu_Response time < 300ms

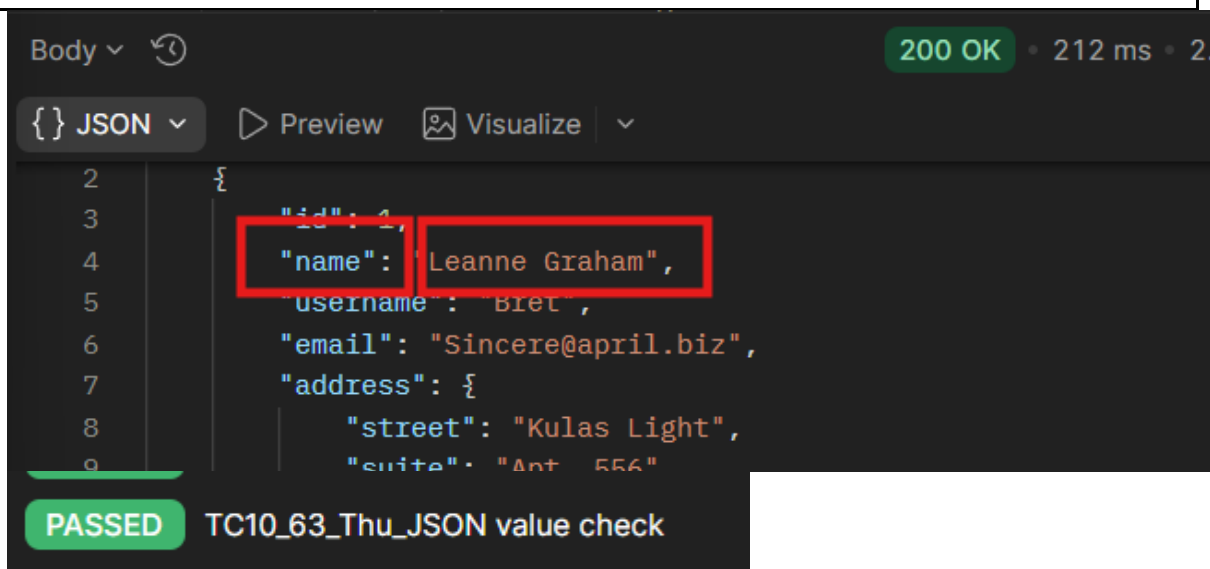
9. TC9_63_Thu_Status

```
//TC9_63_Thu_Status
pm.test("TC9_63_Thu_Status text contains OK", function () {
  pm.expect(pm.response.status).to.include("OK"); // status text có chữ OK
});
```



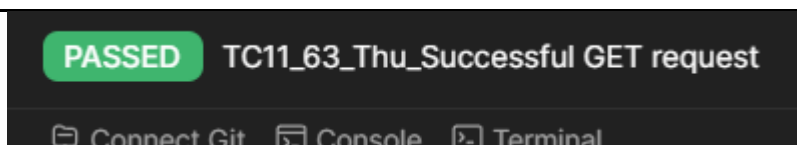
10. *TC10_63_Thu_JSON value*

```
//TC10_63_Thu_JSON value
pm.test("TC10_63_Thu_JSON value check", function () {
  let jsonData = pm.response.json();
  pm.expect(jsonData[0].name).to.eql("Leanne Graham"); // kiểm tra giá trị cụ thể
});
```



11. *TC11_63_Thu_Successful_GET*

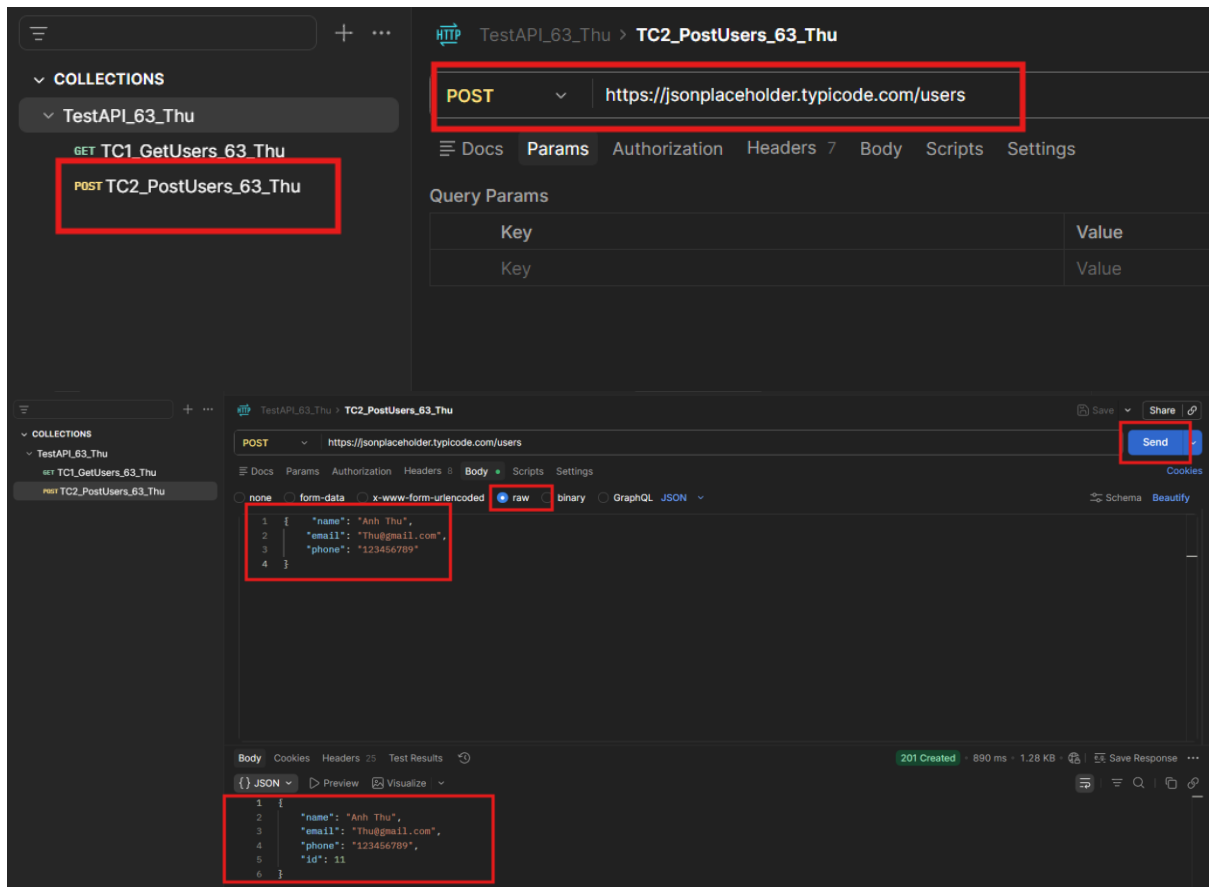
```
//TC11_63_Thu_Successful_GET
pm.test("TC11_63_Thu_Successful GET request", function () {
  pm.expect(pm.response.code).to.be.oneOf([200]); // GET → 200
});
```



5.2 Phương thức POST

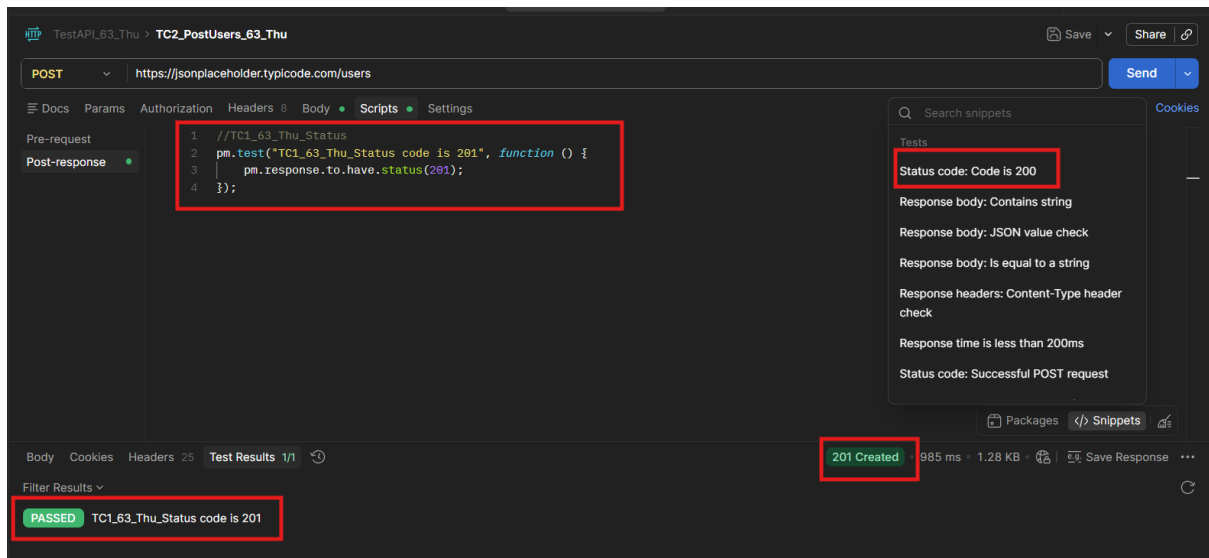
- Thực hiện requests Post trong collection TestAPI_63_Thu đặt tên là TC2_PostUsers_63_Thu
- POST: Dùng để tạo mới dữ liệu trên server bằng cách gửi dữ liệu trong body.

Dữ liệu POST



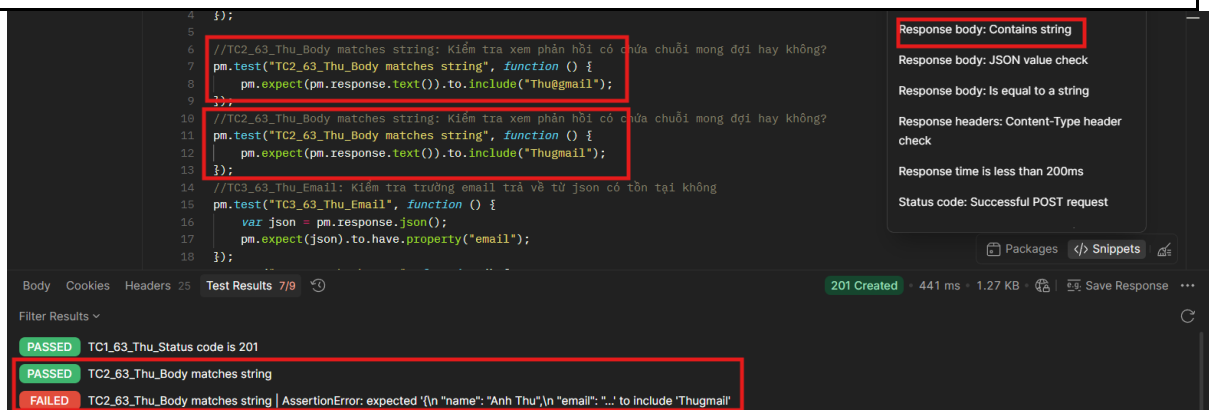
```
{  "name": "Anh Thu",
  "email": "Thu@gmail.com",
  "phone": "123456789"
}
```

Viết test script TC2_PostUsers_63_Thu



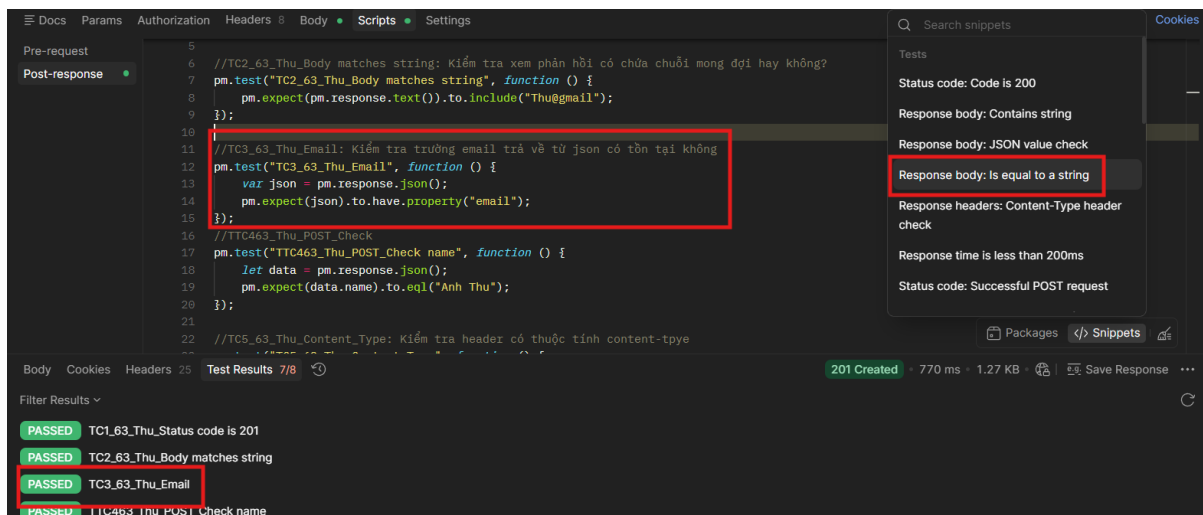
1. TC1_63_Thu_Status

```
//TC1_63_Thu_Status
pm.test("TC1_63_Thu_Status code is 201", function () {
  pm.response.to.have.status(201);
});
```



2. TC2_63_Thu_Body matches string

```
//TC2_63_Thu_Body matches string
pm.test("TC2_63_Thu_Body matches string", function () {
  pm.expect(pm.response.text()).to.include("Thu@gmail");
});
//TC2_63_Thu_Body matches string, trường hợp Fail
pm.test("TC2_63_Thu_Body matches string", function () {
  pm.expect(pm.response.text()).to.include("Thugmail");
});
```



3. *TC3_63_Thu_Email*

```
//TC3_63_Thu_Email: Kiểm tra trường email trả về từ json có tồn tại không
pm.test("TC3_63_Thu_Email", function () {
  var json = pm.response.json();
  pm.expect(json).to.have.property("email");
});
```

4. *TC4_63_Thu_POST_Check name*

```
//TC4_63_Thu_POST_Check
pm.test("TC4_63_Thu_POST_Check name", function () {
  let data = pm.response.json();
  pm.expect(data.name).to.eql("Anh Thu");
});
```

5. *TC5_63_Thu_Content_Type*

```
//TC5_63_Thu_Content_Type: Kiểm tra header có thuộc tính content-tpye
pm.test("TC5_63_Thu_Content_Type", function () {
  pm.response.to.have.header("Content-Type");
});
```

6. *TC6_63_Thu_Response time < 300ms*

```
//TC6_63_Thu_Response time < 300ms: Kiểm tra thời gian phản hồi < 300ms
pm.test("TC6_63_Thu_Response time < 300ms", function () {
  pm.expect(pm.response.responseTime).to.be.below(300);
});
```

7. *TC7_63_Thu_Successful POST request*

```
//TC7_63_Thu_Successful POST request: Kiểm tra trạng thái thành công của yêu cầu POST
pm.test("TC7_63_Thu_Successful POST request", function () {
  pm.expect(pm.response.code).to.be.oneOf([201, 202]);
});
```

8. *TC8_63_Thu_Status name*

```
//TC8_63_Thu_Status name: Kiểm tra trạng thái có chứa chuỗi "Created"
pm.test("TC8_63_Thu_Status name", function () {
  pm.response.to.have.status("Created");
});
```

9. *TC9_63_Thu_Check Schema*

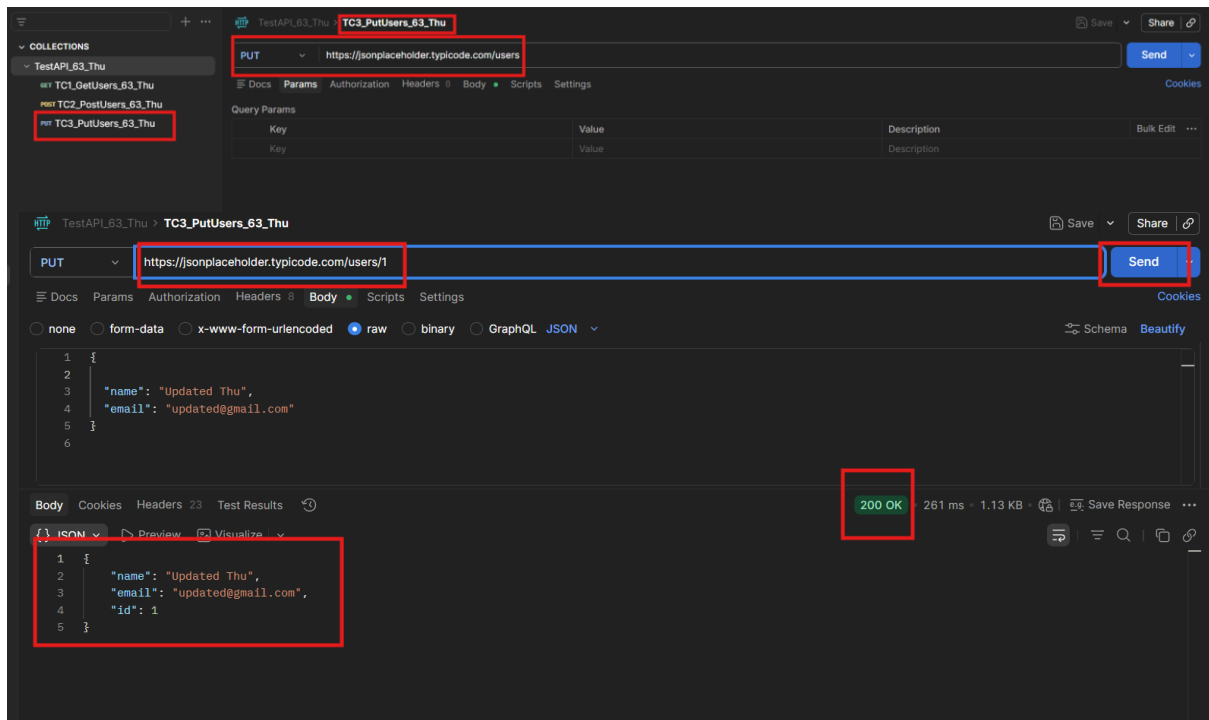
```
//TC9_63_Thu_Check Schema: Kiểm tra dữ liệu trả về có đúng cấu trúc schema hay không
var schema_63_Thu = {
  "type": "object",
  "properties": {
    "id": { "type": ["string", "number"] },
    "name": { "type": "string" },
    "email": { "type": "string" },
    "phone": { "type": "string" },
    "city": { "type": "string" }
  },
  "required": ["id", "name", "email"]
};
```

```
PASSED TC4_63_Thu_POST_Check name
PASSED TC5_63_Thu_Content_Type
PASSED TC6_63_Thu_Response time < 300ms
PASSED TC7_63_Thu_Successful POST request
PASSED TC8_63_Thu_Status name
```

5.3 *Phương thức PUT*

- PUT: Dùng để cập nhật toàn bộ dữ liệu của một tài nguyên trên server.

Dữ liệu PUT



```
{
  "name": "Updated Thu",
  "email": "updated@gmail.com"
}
```

1. TC1_63_Thu_Status code is 200

```
// TC1_63_Thu_Status code is 200
pm.test("TC1_63_Thu_Status code is 200", function () {
  pm.response.to.have.status(200);
});
```

2. TC2_63_Thu_Response not empty

```
// TC2: Kiểm tra response trả về không rỗng, đảm bảo server có phản hồi dữ liệu
pm.test("TC2_63_Thu_Response not empty", function () {
  pm.expect(pm.response.text()).to.not.be.empty;
});
```

3. TC3_63_Thu_Response contains Updated Thu

```
// TC3: Kiểm tra trong response có chứa chuỗi 'Updated Thu', xác nhận dữ liệu đã
được cập nhật
pm.test("TC3_63_Thu_Response contains Updated Thu", function () {
```



```
pm.expect(pm.response.text()).to.include("Updated Thu");
});
```

4. TC4_63_Thu_Response is JSON object

```
// TC4: Kiểm tra kiểu dữ liệu trả về là object JSON, đúng định dạng API
pm.test("TC4_63_Thu_Response is JSON object", function () {
  let data = pm.response.json();
  pm.expect(data).to.be.an("object");
});
```

5. TC5_63_Thu_Check id exists

```
// TC5: Kiểm tra response có chứa trường id, đảm bảo đối tượng có định danh
pm.test("TC5_63_Thu_Check id exists", function () {
  let data = pm.response.json();
  pm.expect(data).to.have.property("id");
});
```

6. TC6_63_Thu_Check name

```
// TC6: Kiểm tra giá trị trường name sau khi cập nhật có đúng như dữ liệu gửi lên
pm.test("TC6_63_Thu_Check name", function () {
  let data = pm.response.json();
  pm.expect(data.name).to.eql("Updated Thu");
});
```

7. TC7_63_Thu_Check email

```
// TC7: Kiểm tra giá trị email trả về có đúng với email đã cập nhật
pm.test("TC7_63_Thu_Check email", function () {
  let data = pm.response.json();
  pm.expect(data.email).to.eql("updated@gmail.com");
});
```

8. TC8_63_Thu_Email format

```
// TC8: Kiểm tra định dạng email hợp lệ bằng cách đảm bảo có ký tự '@'
pm.test("TC8_63_Thu_Email format", function () {
  let data = pm.response.json();
  pm.expect(data.email).to.include("@");
});
```

9. TC9_63_Thu_Content-Type header check

```
// TC9: Kiểm tra header Content-Type có tồn tại và đúng định dạng application/json
pm.test("TC9_63_Thu_Content-Type header check", function () {
  pm.response.to.have.header("Content-Type");
});
```

```
pm.expect(pm.response.headers.get("Content-Type"))
    .to.include("application/json");
});
```

10. TC10_63_Thu_Response time < 500ms

```
// TC10: Kiểm tra thời gian phản hồi của API nhỏ hơn 500ms, đảm bảo hiệu năng
chấp nhận được
pm.test("TC10_63_Thu_Response time < 500ms", function () {
    pm.expect(pm.response.responseTime).to.be.below(500);
});
```

11. TC11_63_Thu_Status text OK

```
// TC11: Kiểm tra status text của response có chứa 'OK', xác nhận trạng thái thành
công
pm.test("TC11_63_Thu_Status text OK", function () {
    pm.expect(pm.response.status).to.include("OK");
});
```

12. TC12_63_Thu_Response code valid

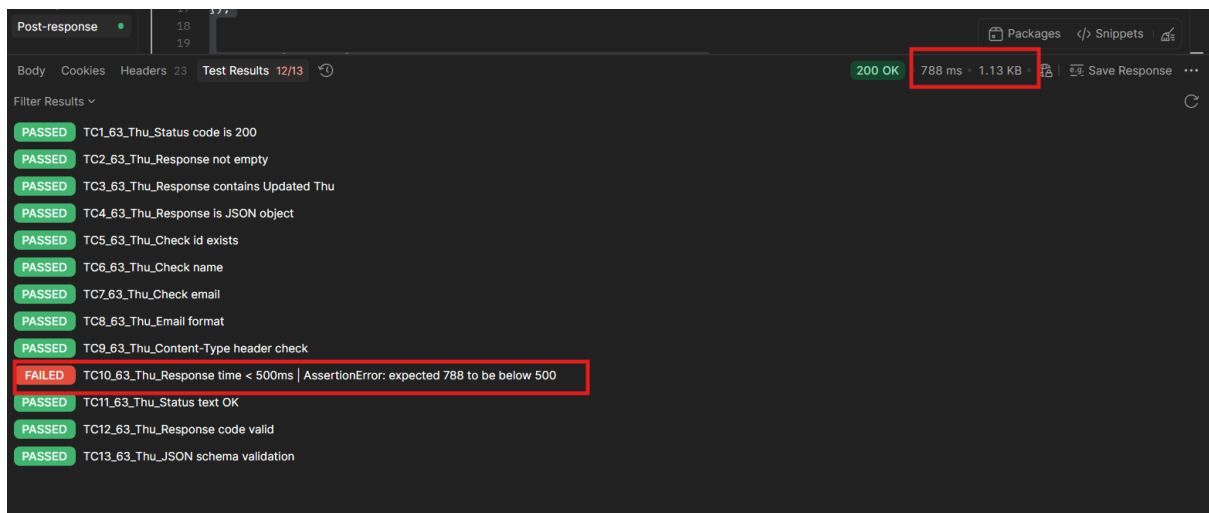
```
// TC12: Kiểm tra mã trạng thái nằm trong danh sách hợp lệ (200 hoặc 201)
pm.test("TC12_63_Thu_Response code valid", function () {
    pm.expect(pm.response.code).to.be.oneOf([200, 201]);
});
```

13. TC13_63_Thu_JSON schema validation

```
// TC13: Kiểm tra cấu trúc JSON trả về có đúng schema mong đợi (id, name, email)
pm.test("TC13_63_Thu_JSON schema validation", function () {
    const schema = {
        type: "object",
        properties: {
            id: { type: ["number", "string"] },
            name: { type: "string" },
            email: { type: "string" }
        },
        required: ["id", "name", "email"]
    };

    pm.expect(tv4.validate(pm.response.json(), schema)).to.be.true;
});
```

Kết quả test

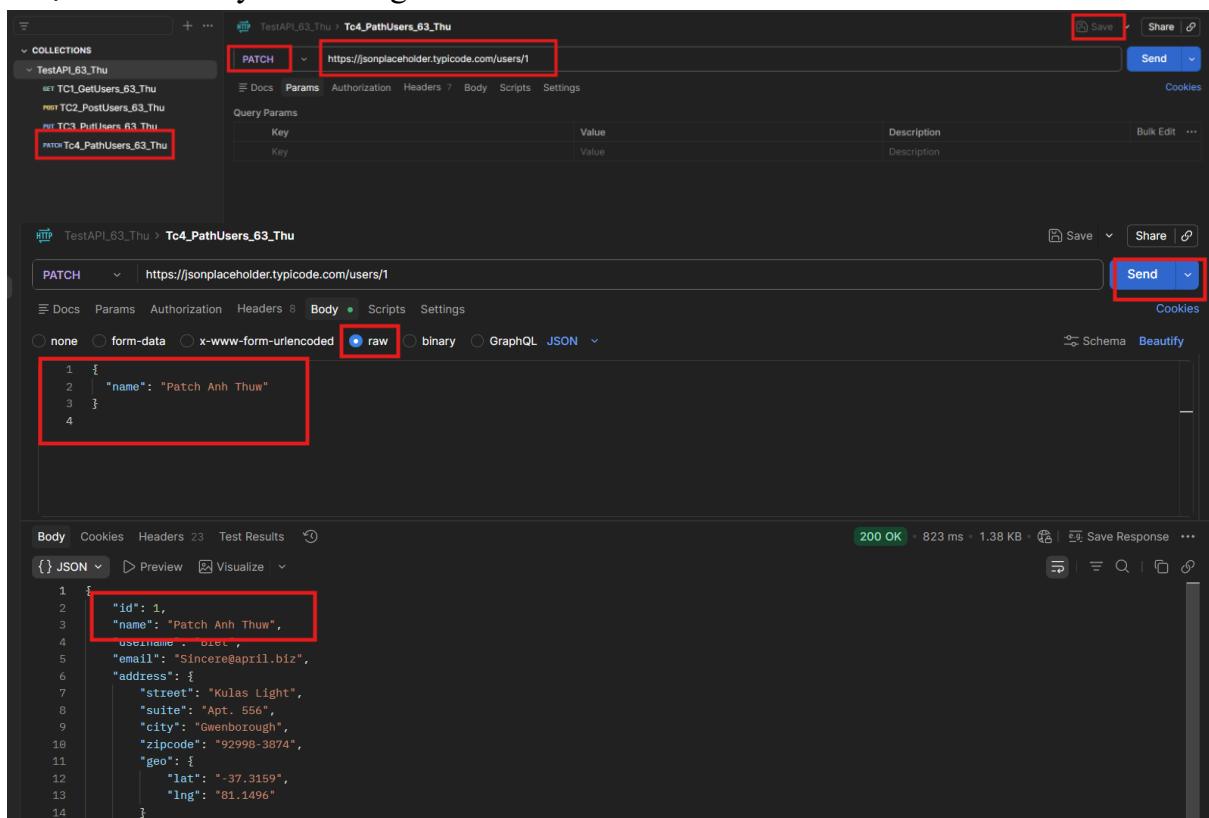


5.4 Phương thức PATCH

- PATCH: Dùng để cập nhật một phần dữ liệu của tài nguyên mà không cần gửi toàn bộ.

Dữ liệu PATCH

Chọn id 1 để thay đổi trường Name



```
{  "name": "Patch Anh Thuw"}
```

1. TC1_63_Thu_Status code is 200

```
// TC1: Kiểm tra mã trạng thái HTTP trả về là 200, đảm bảo request PATCH thành công
pm.test("TC1_63_Thu_Status code is 200", function () {
  pm.response.to.have.status(200);
});
```

TC2_63_Thu_Response not empty

```
// TC2: Kiểm tra response trả về không rỗng
pm.test("TC2_63_Thu_Response not empty", function () {
  pm.expect(pm.response.text()).to.not.be.empty;
});
pm.expect(tv4.validate(pm.response.json(), schema)).to.be.true;
});
```

2. TC3_63_Thu_Response contains patched name

```
// TC3: Kiểm tra response có chứa tên đã cập nhật
pm.test("TC3_63_Thu_Response contains patched name", function () {
  pm.expect(pm.response.text()).to.include("Patch Anh Thuw");
});
```

3. TC4_63_Thu_Response is JSON object

```
// TC4: Kiểm tra kiểu dữ liệu trả về là object JSON
pm.test("TC4_63_Thu_Response is JSON object", function () {
  let data = pm.response.json();
  pm.expect(data).to.be.an("object");
});
```

4. TC5_63_Thu_Check id exists

```
// TC5: Kiểm tra field id tồn tại
pm.test("TC5_63_Thu_Check id exists", function () {
  let data = pm.response.json();
  pm.expect(data).to.have.property("id");
});
```

5. TC6_63_Thu_Check patched name

```
// TC6: Kiểm tra name đã được cập nhật đúng
pm.test("TC6_63_Thu_Check patched name", function () {
  let data = pm.response.json();
  pm.expect(data.name).to.eql("Patch Anh Thuw");
});
```

6. TC7_63_Thu_Name is string

```
// TC7: Kiểm tra name là kiểu string
pm.test("TC7_63_Thu_Name is string", function () {
  let data = pm.response.json();
  pm.expect(data.name).to.be.a("string");
});
```

7. TC8_63_Thu_Name not empty

```
// TC8: Kiểm tra độ dài name > 0
pm.test("TC8_63_Thu_Name not empty", function () {
  let data = pm.response.json();
  pm.expect(data.name.length).to.be.above(0);
});
```

8. TC9_63_Thu_Content-Type header check

```
// TC9: Kiểm tra header Content-Type có tồn tại
pm.test("TC9_63_Thu_Content-Type header check", function () {
  pm.response.to.have.header("Content-Type");
  pm.expect(pm.response.headers.get("Content-Type"))
    .to.include("application/json");
});
```

9. TC10_63_Thu_Response time < 500ms

```
// TC10: Kiểm tra thời gian phản hồi < 500ms
pm.test("TC10_63_Thu_Response time < 500ms", function () {
  pm.expect(pm.response.responseTime).to.be.below(500);
});
```

10. TC11_63_Thu_Status text OK

```
// TC11: Kiểm tra status text có chứa "OK"
pm.test("TC11_63_Thu_Status text OK", function () {
  pm.expect(pm.response.status).to.include("OK");
});
```

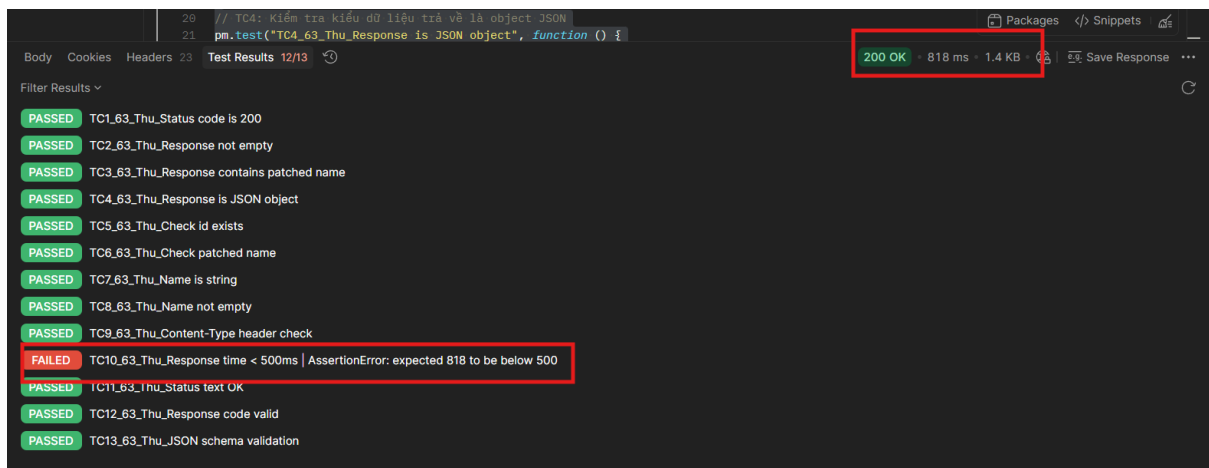
11. TC12_63_Thu_Response code valid

```
// TC12: Kiểm tra response code hợp lệ
pm.test("TC12_63_Thu_Response code valid", function () {
  pm.expect(pm.response.code).to.be.oneOf([200, 201]);
});
```

12. TC13_63_Thu_JSON schema validation

```
// TC13: Kiểm tra schema JSON trả về
pm.test("TC13_63_Thu_JSON schema validation", function () {
  const schema = {
    type: "object",
    properties: {
      id: { type: ["number", "string"] },
      name: { type: "string" }
    },
    required: ["id", "name"]
  };
});
```

Kết quả test

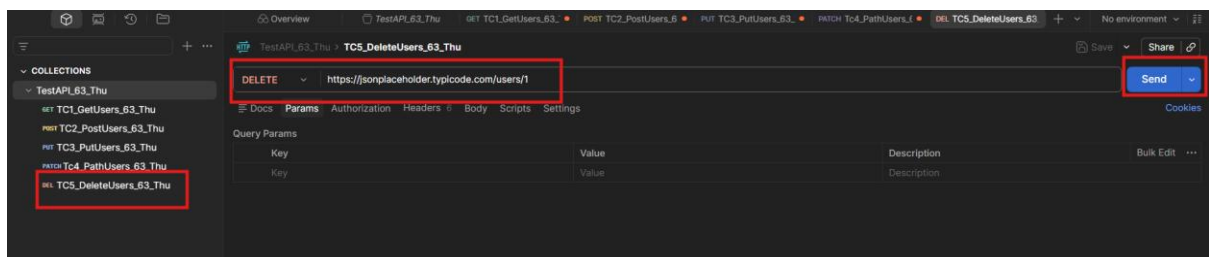


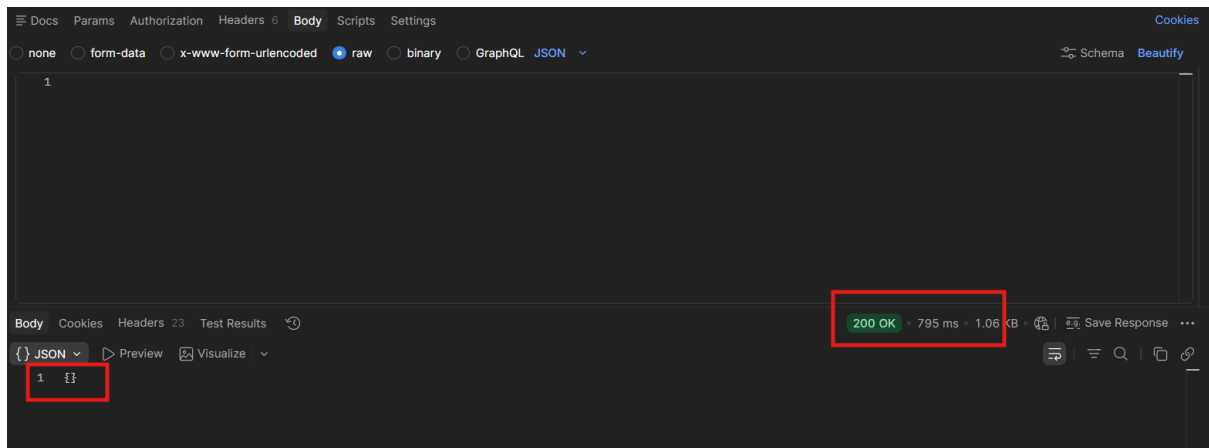
5.5 Phương thức DELETE

DELETE: Dùng để xóa dữ liệu hoặc tài nguyên khỏi server.

Dữ liệu DELETE

Chọn users ID 1 để Delete





1. *TC1_63_Thu_Status code is 200*

```
// TC1: Kiểm tra status code là 200, xác nhận request DELETE thành công
pm.test("TC1_63_Thu_Status code is 200", function () {
  pm.response.to.have.status(200);
});
```

2. *TC2_63_Thu_Response exists*

```
// TC2: Kiểm tra response không bị undefined hoặc null
pm.test("TC2_63_Thu_Response exists", function () {
  pm.expect(pm.response.text()).to.not.be.undefined;
});
```

3. *TC3_63_Thu_Response empty or {}*

```
// TC3: Kiểm tra response trả về rỗng hoặc object rỗng
pm.test("TC3_63_Thu_Response empty or {}", function () {
  let res = pm.response.text();

  pm.expect(res === "" || res === "{}").to.be.true;
});
```

4. *TC4_63_Thu_Response parse JSON*

```
// TC4: Kiểm tra response có thể parse thành JSON (nếu có dữ liệu)
pm.test("TC4_63_Thu_Response parse JSON", function () {
  try {
    let json = pm.response.json();
    pm.expect(json).to.be.an("object");
  } catch (e) {
    pm.expect(true).to.be.true; // nếu rỗng vẫn pass
  }
});
```

5. TC5_63_Thu_Content-Type header exists

```
// TC5: Kiểm tra header Content-Type tồn tại
pm.test("TC5_63_Thu_Content-Type header exists", function () {
  pm.response.to.have.header("Content-Type");
});
```

6. TC6_63_Thu_Content-Type is JSON

```
/ TC6: Kiểm tra Content-Type là JSON (nếu có)
pm.test("TC6_63_Thu_Content-Type is JSON", function () {
  let contentType = pm.response.headers.get("Content-Type");

  if (contentType) {
    pm.expect(contentType).to.include("application/json");
  } else {
    pm.expect(true).to.be.true;
  }
});
```

7. TC7_63_Thu_Response time < 500ms

```
// TC7: Kiểm tra thời gian phản hồi < 500ms
pm.test("TC7_63_Thu_Response time < 500ms", function () {
  pm.expect(pm.response.responseTime).to.be.below(500);
});
```

8. TC8_63_Thu_Status text OK

```
// TC8: Kiểm tra status text có chứa OK
pm.test("TC8_63_Thu_Status text OK", function () {
  pm.expect(pm.response.status).to.include("OK");
});
```

9. TC9_63_Thu_Response code valid

```
// TC9: Kiểm tra response code hợp lệ (200 hoặc 204)
pm.test("TC9_63_Thu_Response code valid", function () {
  pm.expect(pm.response.code).to.be.oneOf([200, 204]);
});
```

10. TC10_63_Thu_No user data returned

```
pm.test("TC10_63_Thu_No user data returned", function () {
  pm.expect(pm.response.text()).to.not.include("name");
});
```


11. TC11_63_Thu_No email returned

```
// TC11: Kiểm tra response không chứa email
pm.test("TC11_63_Thu_No email returned", function () {
  pm.expect(pm.response.text()).to.not.include("email");
});
```

12. TC12_63_Thu_No id returned

```
// TC12: Kiểm tra response không chứa id
pm.test("TC12_63_Thu_No id returned", function () {
  pm.expect(pm.response.text()).to.not.include("id");
});
```

13. TC13_63_Thu_Empty schema validation

```
// TC13: Kiểm tra schema rỗng (object hoặc empty)
pm.test("TC13_63_Thu_Empty schema validation", function () {
  let res = pm.response.text();

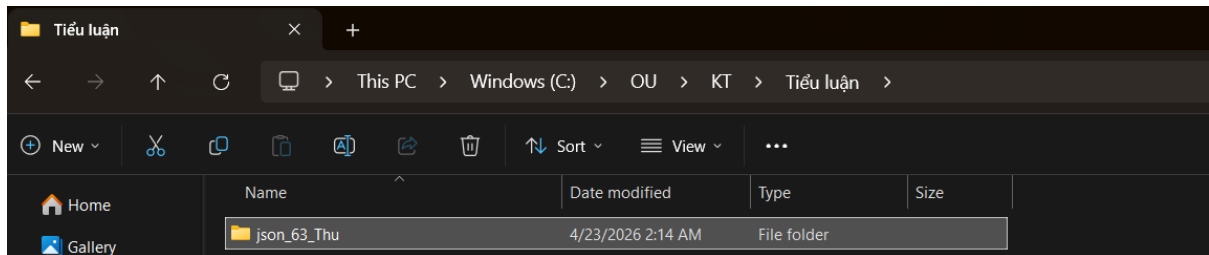
  pm.expect(res === "" || res === "{}").to.be.true;
});
```

Kết quả test

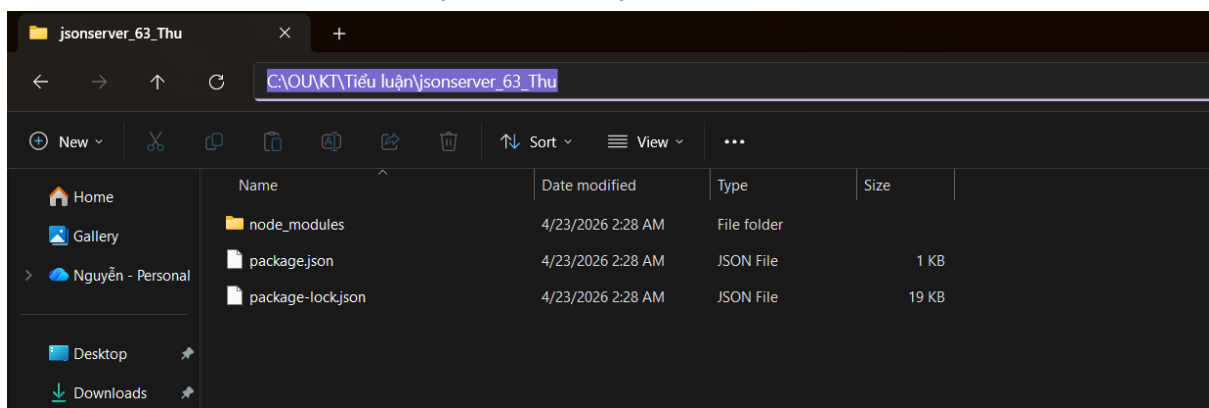
```
PASSED TC1_63_Thu_Status code is 200
PASSED TC2_63_Thu_Response exists
PASSED TC3_63_Thu_Response empty or {}
PASSED TC4_63_Thu_Response parse JSON
PASSED TC5_63_Thu_Content-Type header exists
PASSED TC6_63_Thu_Content-Type is JSON
PASSED TC7_63_Thu_Response time < 500ms
PASSED TC8_63_Thu_Status text OK
PASSED TC9_63_Thu_Response code valid
PASSED TC10_63_Thu_No user data returned
PASSED TC11_63_Thu_No email returned
PASSED TC12_63_Thu_No id returned
PASSED TC13_63_Thu_Empty schema validation
```

2. Tạo JSONSERVER riêng

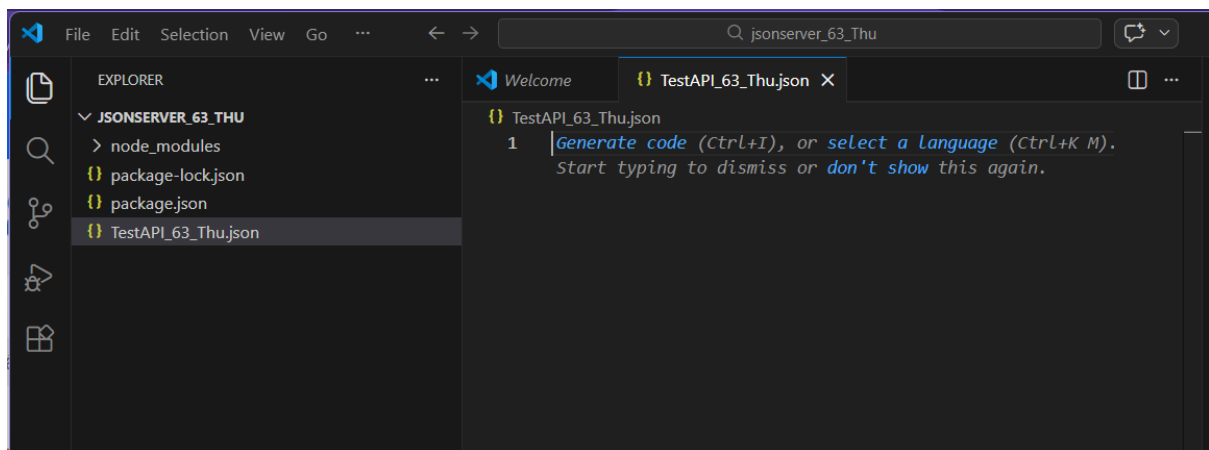
Bước 1: Cài đặt Node, tạo JsonServer trên thư mục `jsonserver_63_Thu` và chạy lệnh “`npm install json-server`” trên cmd -> Tạo thư mục để chứa các package



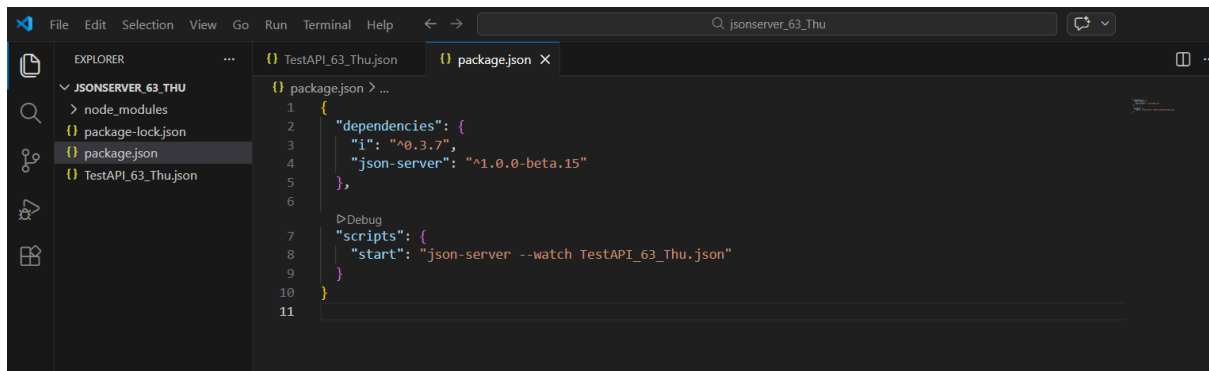
Bước 2: Sau khi Install thư mục sẽ hiển thị như sau:



Bước 3: Mở Visual Studio và tạo 1 file, đặt tên là “`TestAPI_63_Thu.json`”

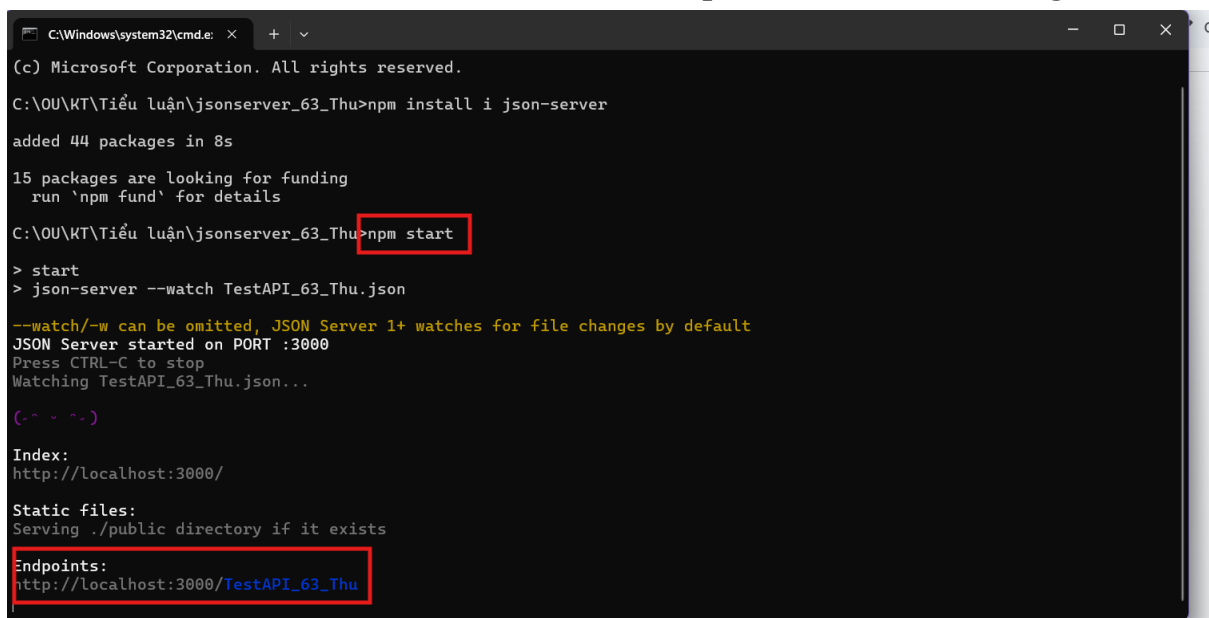


Bước 4: Mở file package.json và viết lệnh thực thi



```
1 {
2   "dependencies": {
3     "i": "^0.3.7",
4     "json-server": "^1.0.0-beta.15"
5   },
6
7   >Debug
8   "scripts": {
9     "start": "json-server --watch TestAPI_63_Thu.json"
10  }
11 }
```

Bước 5: Trở lại thư mục, mở cmd và viết lệnh “npm start” để nhận đường dẫn



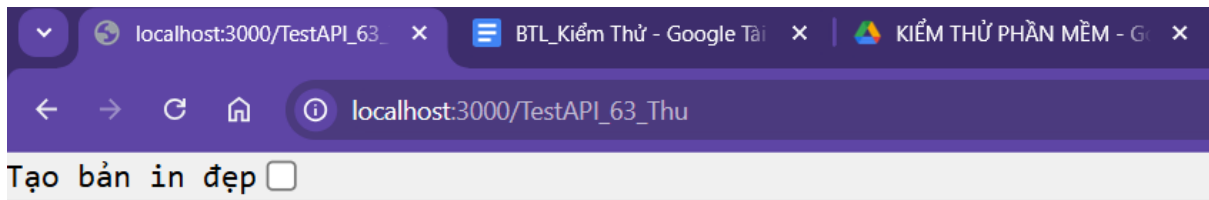
```
C:\Windows\system32\cmd.exe: X + v
(c) Microsoft Corporation. All rights reserved.
C:\OU\KT\Tiểu luận\jsonserver_63_Thu>npm install i json-server
added 44 packages in 8s
15 packages are looking for funding
  run 'npm fund' for details
C:\OU\KT\Tiểu luận\jsonserver_63_Thu>npm start
> start
> json-server --watch TestAPI_63_Thu.json
--watch/-w can be omitted, JSON Server 1+ watches for file changes by default
JSON Server started on PORT :3000
Press CTRL-C to stop
Watching TestAPI_63_Thu.json...
(^.^)

Index:
http://localhost:3000/

Static files:
Serving ./public directory if it exists

Endpoints:
http://localhost:3000/TestAPI_63_Thu
```

Bước 6: Copy đường dẫn và mở ở trình duyệt

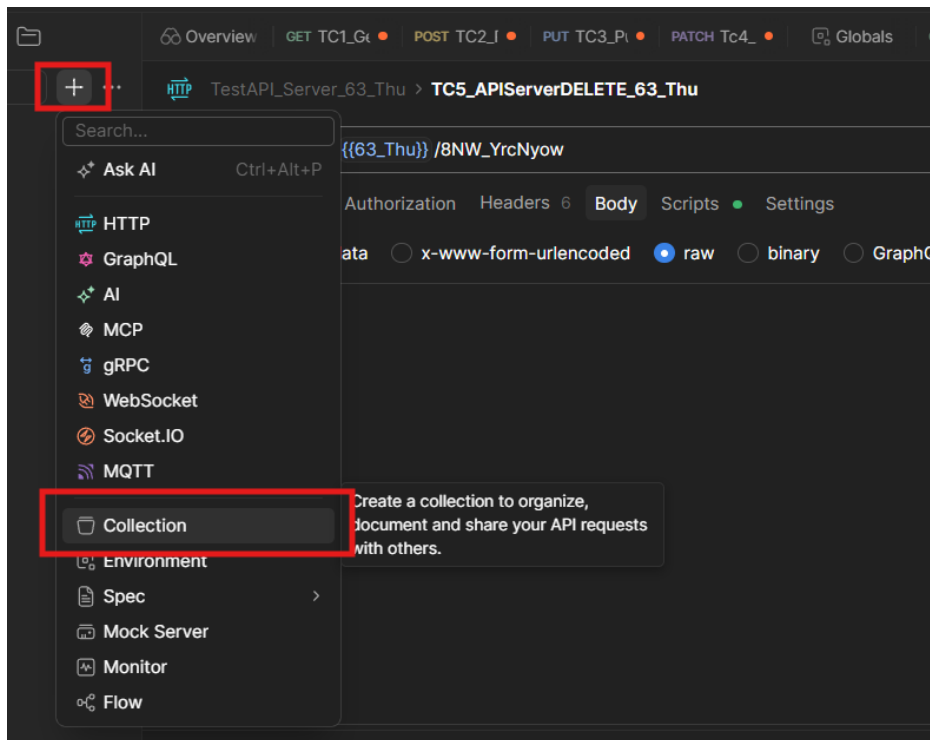


```
[
  {
    "id": "1",
    "name": "Thu",
    "email": "thu@gmail.com",
    "phone": "0762924978",
    "city": "Ho Chi Minh"
  },
  {
    "id": "2",
    "name": "An",
    "email": "anh@gmail.com",
    "phone": "0912345678",
    "city": "Ha Noi"
  },
  {
    "id": "3",
    "name": "Nguyet",
    "email": "nguyet@gmail.com",
    "phone": "0931497315",
    "city": "Da Nang"
  },
  {
    "id": "4",
    "name": "Dien",
    "email": "dien@gmail.com",
    "phone": "0934567890",
    "city": "Can Tho"
  },
  {
    "id": "5",
    "name": "Duy",

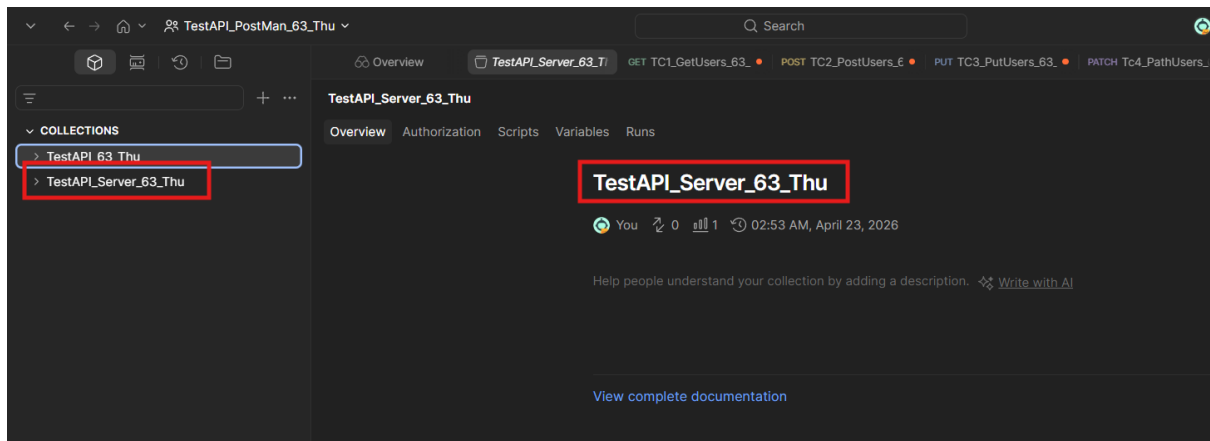
```

Bước 7: Tiếp tục để chạy cmd và mở Postman để thực hiện bài test.

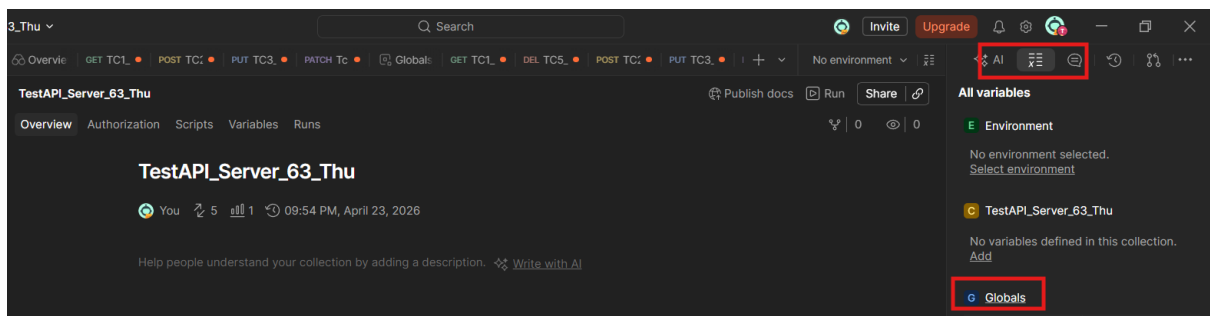
- Click vào dấu cộng chọn New Collection

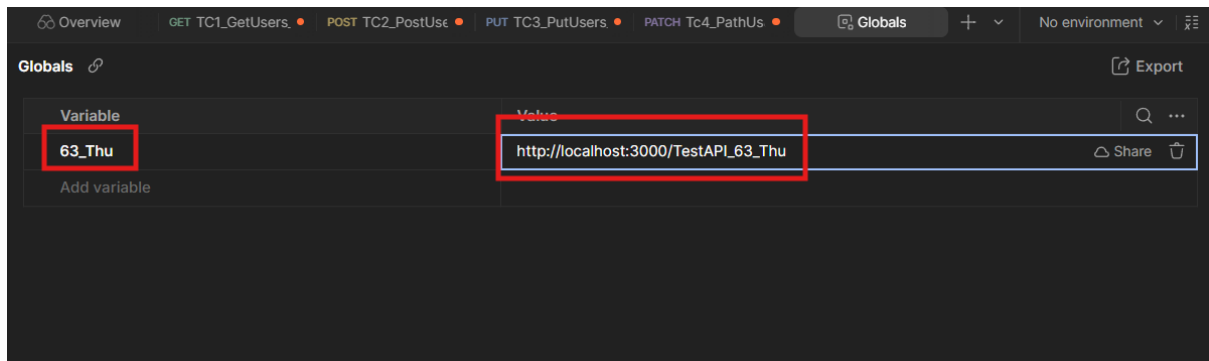


Bước 8: Đặt tên cho Collection mới là “TestAPI_Server_63_Thu”



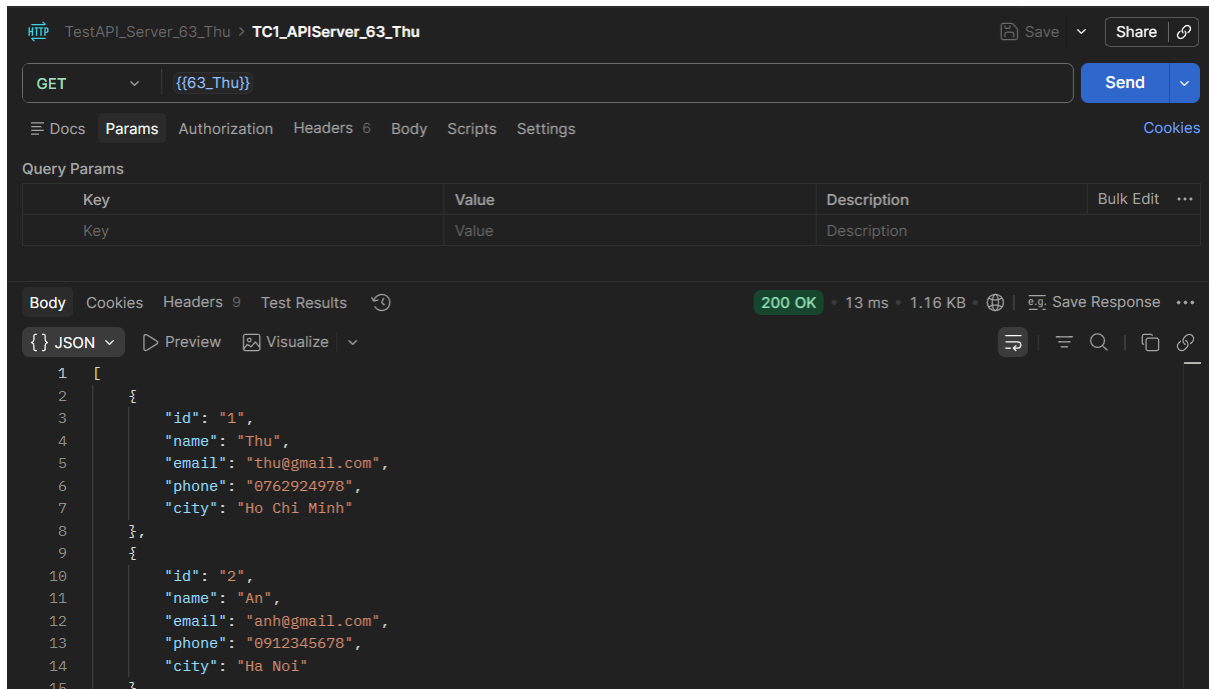
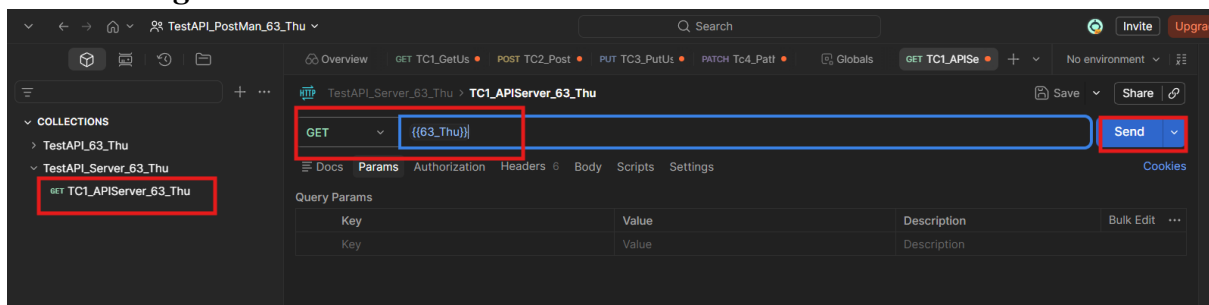
Bước 9: Chọn mục Globals để biến global (Sử dụng biến 63_Thu theo cú pháp {{63_Thu}})

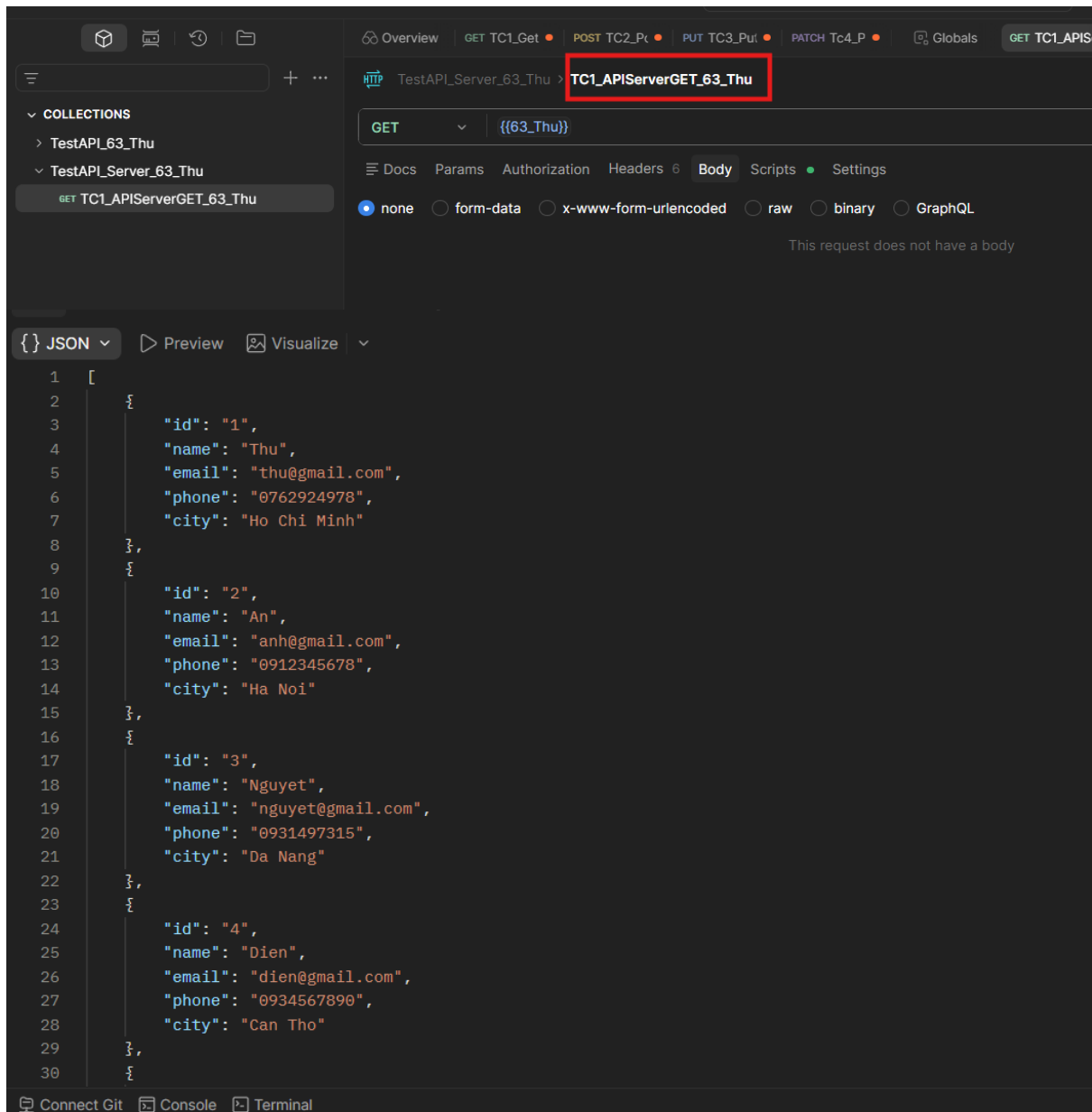




Bước 10: Thực hiện Test với các Phương Thức

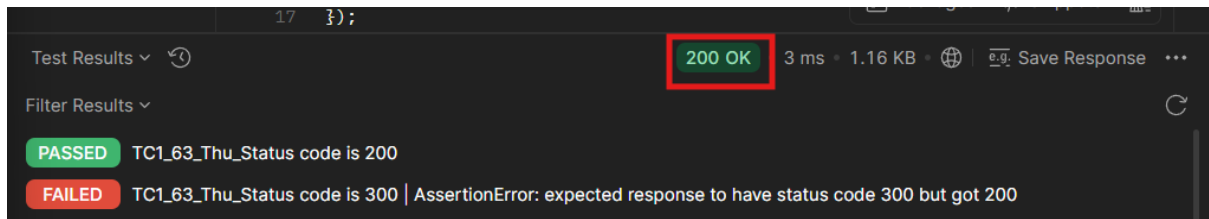
10.1 Phương thức GET





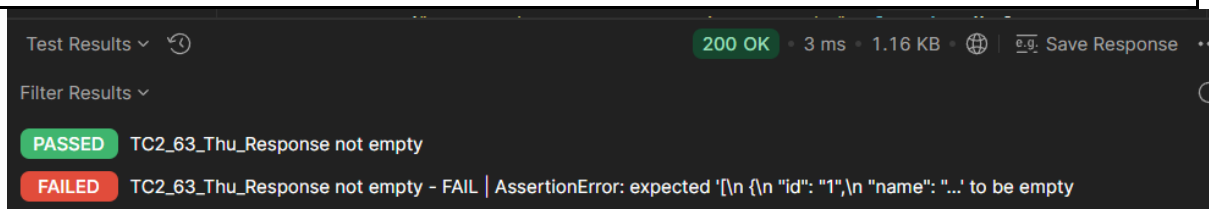
1. TC1_63_Thu_Status code is 200

```
// TC1: Kiểm tra mã trạng thái HTTP = 200
// → Xác nhận request GET được xử lý thành công, server phản hồi đúng
// → Nếu khác 200 (404, 500) → API lỗi hoặc không tồn tại endpoint
pm.test("TC1_63_Thu_Status code is 200", function () {
  pm.response.to.have.status(200);
});
pm.test("TC1_63_Thu_Status code is 300", function () {
  pm.response.to.have.status(300);
});
```



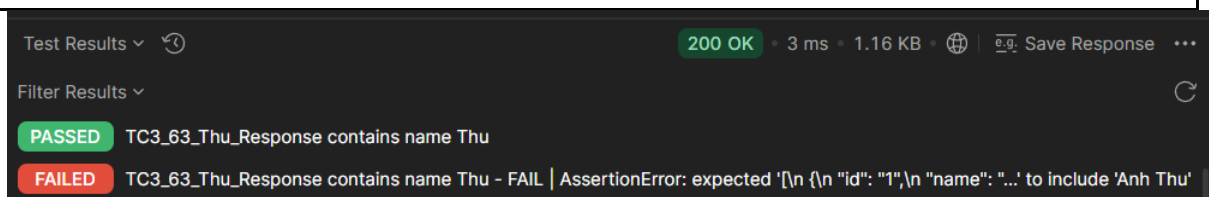
2. TC2_63_Thu_Response not empty

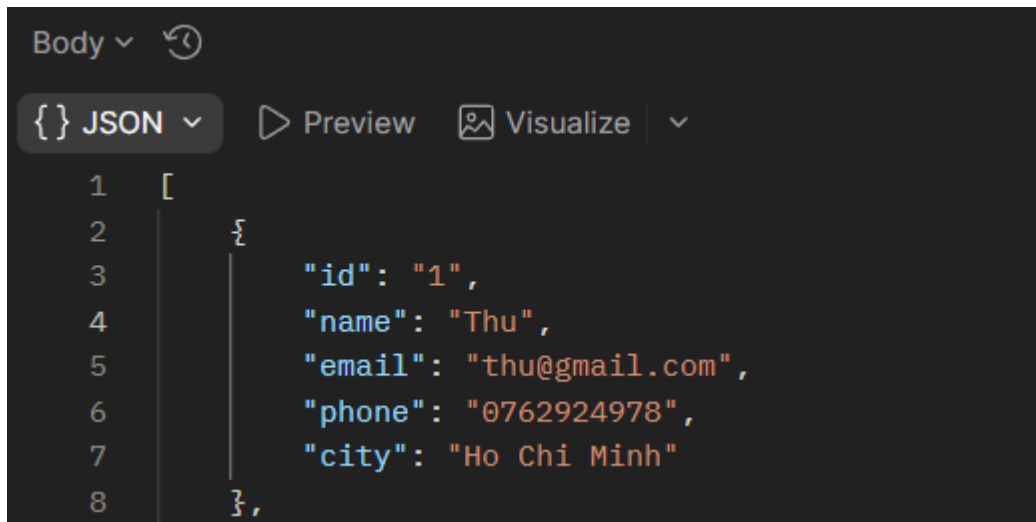
```
// TC2: Kiểm tra response không rỗng
// → Đảm bảo server có trả dữ liệu, tránh trường hợp API trả về rỗng hoặc lỗi
// → Nếu rỗng → có thể lỗi database hoặc xử lý server
pm.test("TC2_63_Thu_Response not empty", function () {
  pm.expect(pm.response.text()).to.not.be.empty;
});
// TC2: Kiểm tra response không rỗng (FAIL: expect rỗng)
pm.test("TC2_63_Thu_Response not empty - FAIL", function () {
  pm.expect(pm.response.text()).to.be.empty; // sai → fail
});
```



3. TC3_63_Thu_Response contains name Thu

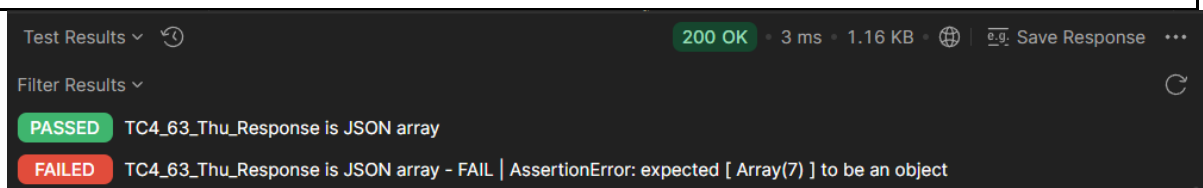
```
// TC3: Kiểm tra response có chứa "Thu"
// → Xác nhận dữ liệu có tồn tại trong danh sách
// → Nếu không có → API thiếu dữ liệu hoặc sai logic
pm.test("TC3_63_Thu_Response contains name Thu", function () {
  pm.expect(pm.response.text()).to.include("Thu");
});
// TC3: Kiểm tra response có chứa "Thu" (FAIL: chuỗi không tồn tại)
pm.test("TC3_63_Thu_Response contains name Thu - FAIL", function () {
  pm.expect(pm.response.text()).to.include("Anh Thu"); // fail
});
```





4. TC4_63_Thu_Response is JSON array

```
// TC4: Kiểm tra kiểu dữ liệu trả về là JSON array
// → Vì API trả về danh sách user nên phải là mảng []
// → Nếu không phải array → sai cấu trúc API
pm.test("TC4_63_Thu_Response is JSON array", function () {
  let data = pm.response.json();
  pm.expect(data).to.be.an("array");
});
// TC4: Kiểm tra kiểu dữ liệu là array (FAIL: check sai kiểu)
pm.test("TC4_63_Thu_Response is JSON array - FAIL", function () {
  let data = pm.response.json();
  pm.expect(data).to.be.an("object"); // fail
});
```



5. TC5_63_Thu_Array length

```
// TC5: Kiểm tra mảng có phần tử (length = 7)
pm.test("TC5_63_Thu_Array length", function () {
  let data = pm.response.json();
  pm.expect(data.length).to.eql(7);
});
// TC5: Kiểm tra mảng có phần tử (FAIL: ép length = 8)
pm.test("TC5_63_Thu_Array length - FAIL", function () {
  let data = pm.response.json();
  pm.expect(data.length).to.eql(8); // fail
});
```

```
});
```

Test Results  200 OK • 3 ms • 1.16 KB •  [Save Response](#) ...

Filter Results 

PASSED TC5_63_Thu_Array length

FAILED TC5_63_Thu_Array length - FAIL | AssertionError: expected 7 to deeply equal 8

6. TC6_63_Thu_Check id exists

```
// TC6: Kiểm tra user đầu tiên có field id
// → Xác nhận cấu trúc object đúng
// → Nếu thiếu id → dữ liệu sai format
pm.test("TC6_63_Thu_Check id exists", function () {
  let data = pm.response.json();
  pm.expect(data[0]).to.have.property("id");
});
// TC6: Kiểm tra user đầu tiên có id (FAIL: check field không tồn tại)
pm.test("TC6_63_Thu_Check feild exists - FAIL", function () {
  let data = pm.response.json();
  pm.expect(data[0]).to.have.property("old"); // fail
});
```

Test Results  200 OK • 3 ms • 1.16 KB •  [Save Response](#) ...

Filter Results 

PASSED TC6_63_Thu_Check id exists

FAILED TC6_63_Thu_Check feild exists - FAIL | AssertionError: expected { id: '1', name: 'Thu', ...(3) } to have property 'old'

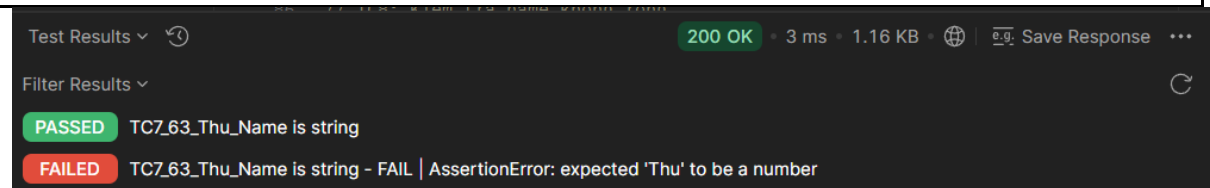
Dữ liệu không có trường “old”

```
{
  "id": "7",
  "name": "Lan",
  "email": "lan@gmail.com",
  "phone": "0967890123",
  "city": "Nha Trang"
}
```

7. TC7_63_Thu_Name is string

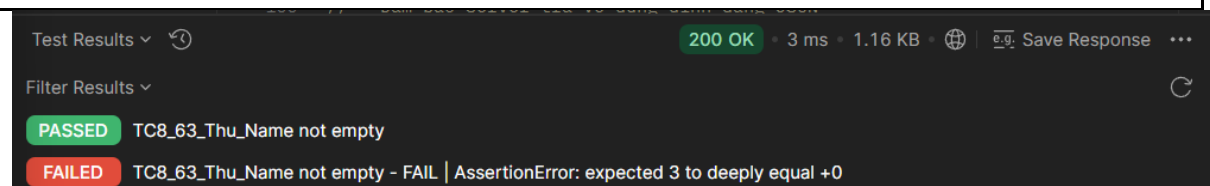
```
// TC7: Kiểm tra name là kiểu string
// → Đảm bảo dữ liệu đúng kiểu (không phải number/null)
// → Nếu sai → lỗi dữ liệu từ server
pm.test("TC7_63_Thu_Name is string", function () {
  let data = pm.response.json();
  pm.expect(data[0].name).to.be.a("string");
});
```

```
// TC7: Kiểm tra name là string (FAIL: check sai kiểu)
pm.test("TC7_63_Thu_Name is string - FAIL", function () {
  let data = pm.response.json();
  pm.expect(data[0].name).to.be.a("number"); // fail
});
```



8. TC8_63_Thu_Name not empty

```
// TC8: Kiểm tra name không rỗng
// → Đảm bảo dữ liệu hợp lệ (không phải chuỗi rỗng)
// → Nếu rỗng → dữ liệu không có ý nghĩa
pm.test("TC8_63_Thu_Name not empty", function () {
  let data = pm.response.json();
  pm.expect(data[0].name.length).to.be.above(0);
});
// TC8: Kiểm tra name không rỗng (FAIL: ép length = 0)
pm.test("TC8_63_Thu_Name not empty - FAIL", function () {
  let data = pm.response.json();
  pm.expect(data[0].name.length).to.eql(0); // fail
});
```



9. TC9_63_Thu_Content-Type header check

```
// TC9: Kiểm tra header Content-Type
// → Đảm bảo server trả về đúng định dạng JSON
// → Nếu sai → client có thể parse lỗi
pm.test("TC9_63_Thu_Content-Type header check", function () {
  pm.response.to.have.header("Content-Type");
  pm.expect(pm.response.headers.get("Content-Type"))
    .to.include("application/json");
});
// TC9: Kiểm tra header Content-Type (FAIL: header sai)
pm.test("TC9_63_Thu_Content-Type header check - FAIL", function () {
  pm.response.to.have.header("Content-Type-Sai"); // fail
});
```

Test Results  200 OK • 3 ms • 1.16 KB •  [e.g. Save Response](#) ...

Filter Results 

PASSED

TC9_63_Thu_Content-Type header check

FAILED

TC9_63_Thu_Content-Type header check - FAIL | AssertionError: expected response to have header with key 'Content-Type-Sa

Headers  200 OK • 3 ms • 1.16 KB •  [e.g. Save Response](#) ...

X-Powered-By	tinyhttp
Access-Control-Allow-Origin	*
Access-Control-Allow-Methods	GET, HEAD, PUT, PATCH, POST, DELETE
Access-Control-Allow-Headers	content-type
Content-Type	application/json
Date	Thu, 23 Apr 2026 11:48:16 GMT
Connection	keep-alive

10. TC10_63_Thu_Response time < 500ms

```
// TC10: Kiểm tra thời gian phản hồi < 500ms
// → Đánh giá hiệu năng API
// → Nếu quá cao → API chậm, cần tối ưu
pm.test("TC10_63_Thu_Response time < 500ms", function () {
  pm.expect(pm.response.responseTime).to.be.below(500);
});
// TC10: Kiểm tra response time < 500ms (FAIL: ép nhỏ hơn 1ms)
pm.test("TC10_63_Thu_Response time < 500ms - FAIL", function () {
  pm.expect(pm.response.responseTime).to.be.below(1); // fail
});
```

Test Results ▾ 🕒		200 OK	3 ms	1.16 KB	🌐	📄 Save Response	⋮
Filter Results ▾							
PASSED TC10_63_Thu_Response time < 500ms							
FAILED TC10_63_Thu_Response time < 500ms - FAIL AssertionError: expected 3 to be below 1							

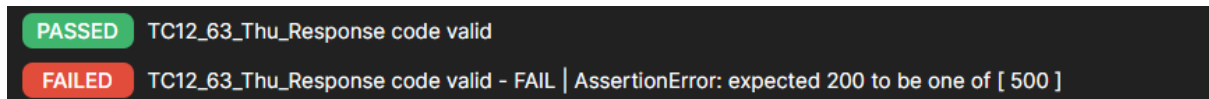
11. TC11_63_Thu_Status text OK

```
// TC11: Kiểm tra status text có chứa "OK"
// → Xác nhận phản hồi thành công về mặt mô tả
// → Nếu không có "OK" → có thể là lỗi hoặc trạng thái khác
pm.test("TC11_63_Thu_Status text OK", function () {
  pm.expect(pm.response.status).to.include("OK");
});
// TC11: Kiểm tra status text có chứa "OK" (FAIL: check sai text)
pm.test("TC11_63_Thu_Status text OK - FAIL", function () {
  pm.expect(pm.response.status).to.include("Error"); // fail
});
```



12. TC12_63_Thu_Response code valid

```
// TC12: Kiểm tra response code hợp lệ
// → Đảm bảo chỉ nhận các mã hợp lệ (200)
// → Nếu khác → API có vấn đề
pm.test("TC12_63_Thu_Response code valid", function () {
  pm.expect(pm.response.code).to.be.oneOf([200]);
});
// TC12: Kiểm tra response code hợp lệ (FAIL: code không hợp lệ)
pm.test("TC12_63_Thu_Response code valid - FAIL", function () {
  pm.expect(pm.response.code).to.be.oneOf([500]); // fail
});
```



13. TC13_63_Thu_JSON schema validation

```
// TC13: Kiểm tra schema dữ liệu
// → Đảm bảo cấu trúc JSON đúng: id, name, email, phone, city
// → Nếu sai → dữ liệu không đúng format yêu cầu
pm.test("TC13_63_Thu_JSON schema validation", function () {
  const schema = {
    type: "array",
    items: {
      type: "object",
      properties: {
        id: { type: ["number", "string"] },
        name: { type: "string" },
        email: { type: "string" },
        phone: { type: "string" },
        city: { type: "string" }
      },
      required: ["id", "name", "email"]
    }
  };
});
```

```

    pm.expect(tv4.validate(pm.response.json(), schema)).to.be.true;
  });
  // TC13: Kiểm tra schema dữ liệu (FAIL: sai kiểu dữ liệu)
  pm.test("TC13_63_Thu_JSON schema validation", function () {
    const schema = {
      type: "array",
      items: {
        type: "object",
        properties: {
          id: { type: ["number", "string"] },
          name: { type: "string" },
          email: { type: "string" },
          phone: { type: "number" },
          city: { type: "string" }
        },
        required: ["id", "name", "email"]
      }
    };

    pm.expect(tv4.validate(pm.response.json(), schema)).to.be.true;
  });

```

PASSED TC13_63_Thu_JSON schema validation

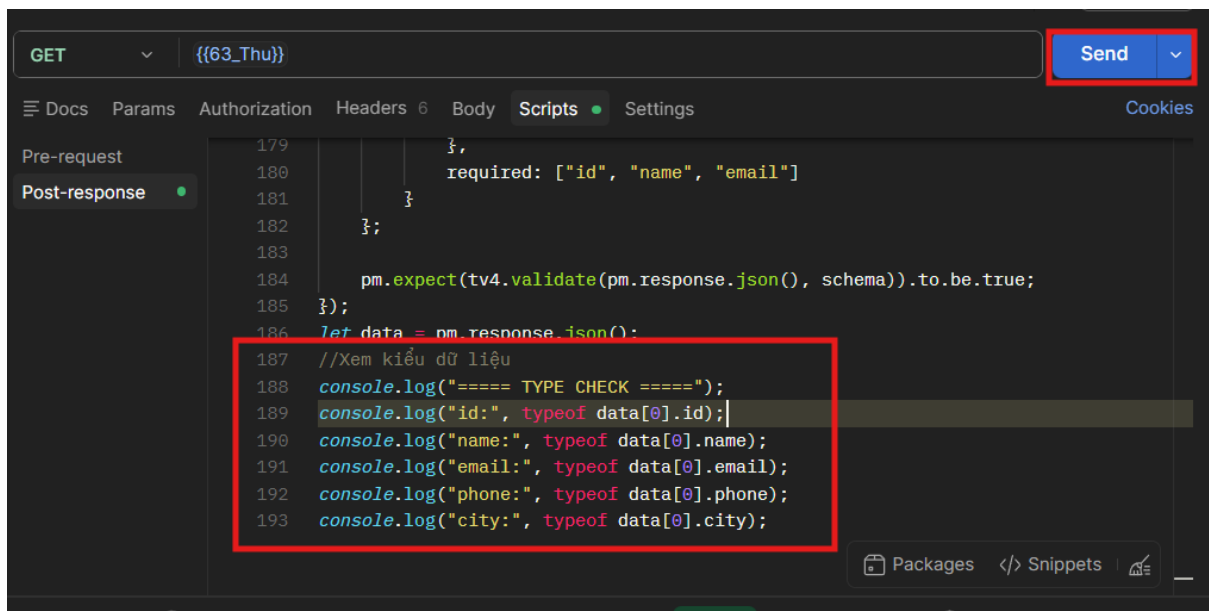
FAILED TC13_63_Thu_JSON schema validation | AssertionError: expected false to be true

Cách kiểm tra kiểu dữ liệu của dữ liệu

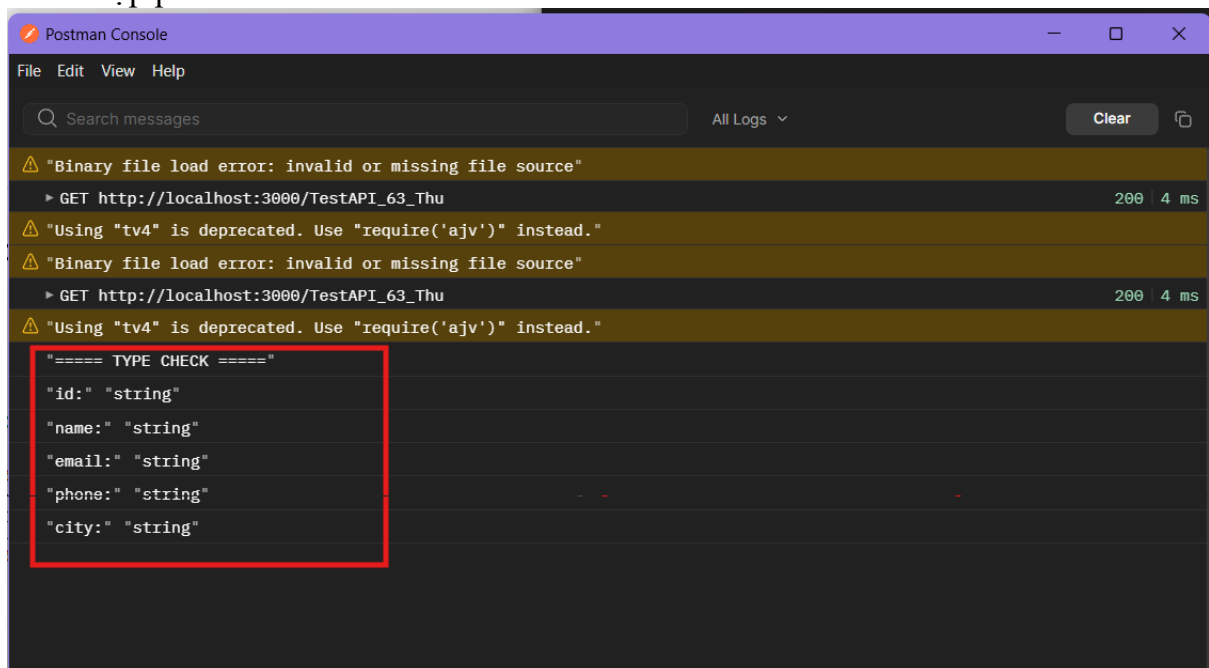
```

let data = pm.response.json();
console.log("===== TYPE CHECK =====");
console.log("id:", typeof data[0].id);
console.log("name:", typeof data[0].name);
console.log("email:", typeof data[0].email);
console.log("phone:", typeof data[0].phone);
console.log("city:", typeof data[0].city);

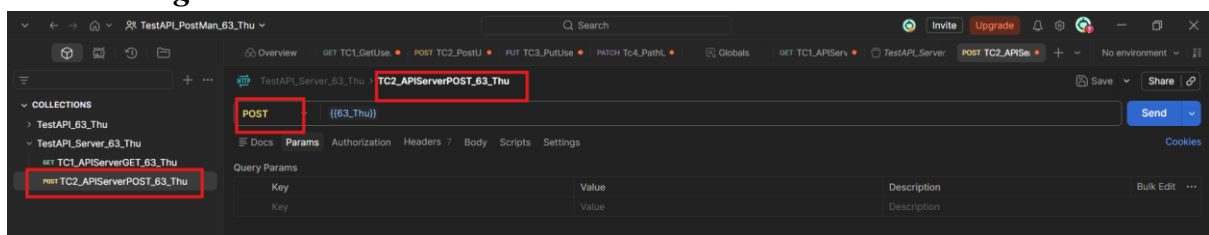
```



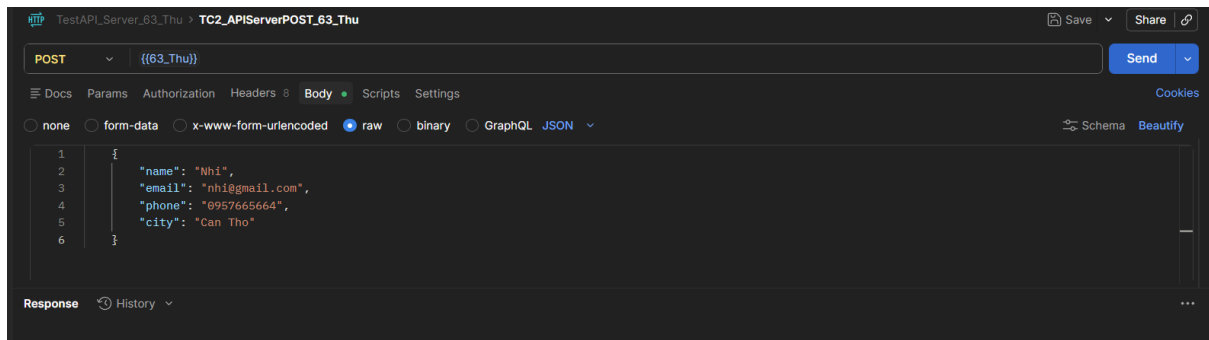
Án tổ hợp phím Ctrl + Alt + C để mở màn hình Console của PostMan



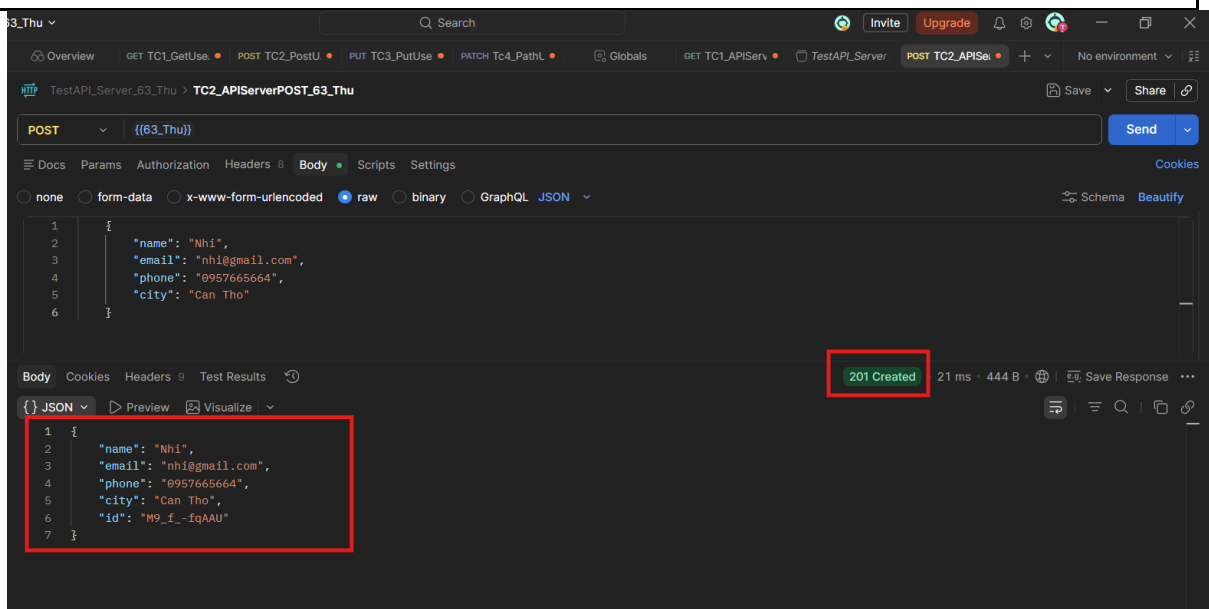
10.2 Phương thức POST



Dữ liệu POST



```
{
  "name": "Nhi",
  "email": "nhi@gmail.com",
  "phone": "0957665664",
  "city": "Can Tho"
}
```



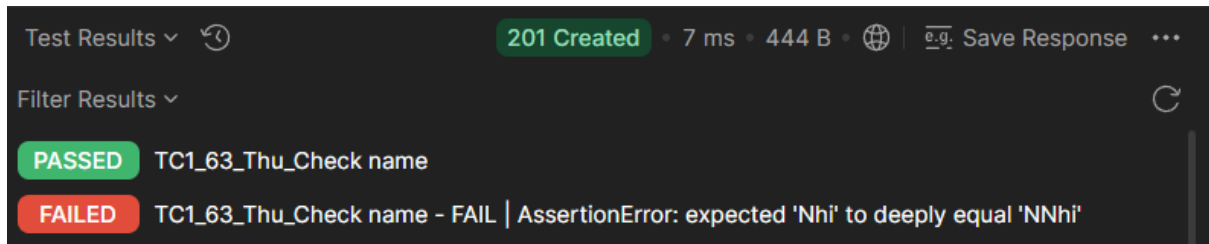
Kiểm tra

http://localhost:3000/TestAPI_63_Thu


```
localhost:3000/TestAPI_63_Thu
name : huy ,
"email": "huy@gmail.com",
"phone": "0956789012",
"city": "Hue"
},
{
  "id": "7",
  "name": "Lan",
  "email": "lan@gmail.com",
  "phone": "0967890123",
  "city": "Nha Trang"
},
{
  "name": "Nhi",
  "email": "nhi@gmail.com",
  "phone": "0957665664",
  "city": "Can Tho",
  "id": "M9_f_-fqAAU"
}
```

1. TC1_63_Thu_Check name

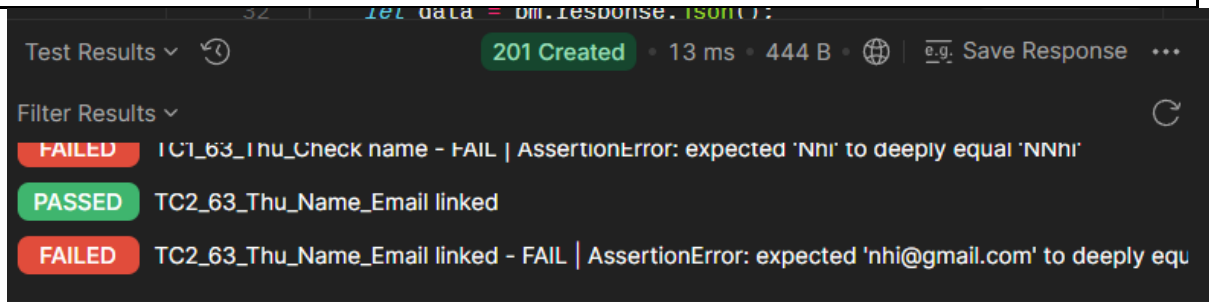
```
// TC1: Kiểm tra name đúng dữ liệu đã gửi
pm.test("TC1_63_Thu_Check name", function () {
  let data = pm.response.json();
  pm.expect(data.name).to.eql("Nhi");
});
// TC1: Kiểm tra name đúng dữ liệu đã gửi (FAIL)
pm.test("TC1_63_Thu_Check name - FAIL", function () {
  let data = pm.response.json();
  pm.expect(data.name).to.eql("NNhi");
});
```



2. TC2_63_Thu_Name_Email linked

```
// TC2: Kiểm tra email phải đúng với name = Nhi
pm.test("TC2_63_Thu_Name_Email linked", function () {
  let data = pm.response.json();
  pm.expect(data.name).to.eql("Nhi");
  pm.expect(data.email).to.eql("nhi@gmail.com");
});

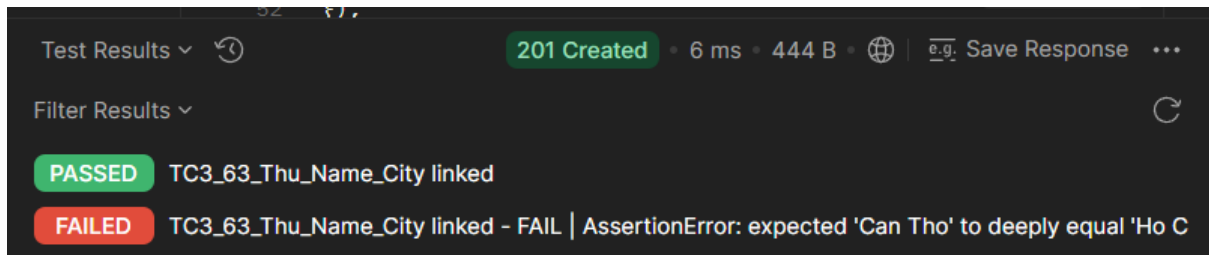
pm.test("TC2_63_Thu_Name_Email linked - FAIL", function () {
  let data = pm.response.json();
  pm.expect(data.name).to.eql("Nhi");
  pm.expect(data.email).to.eql("nnhi@gmail.com"); // sai → fail
});
```



3. TC3_63_Thu_Name_City linked

```
// TC3: Kiểm tra city phải đúng với name = Nhi
pm.test("TC3_63_Thu_Name_City linked", function () {
  let data = pm.response.json();
  pm.expect(data.name).to.eql("Nhi");
  pm.expect(data.city).to.eql("Can Tho");
});

pm.test("TC3_63_Thu_Name_City linked - FAIL", function () {
  let data = pm.response.json();
  pm.expect(data.name).to.eql("Nhi");
  pm.expect(data.city).to.eql("Ho Chi Minh");
});
```

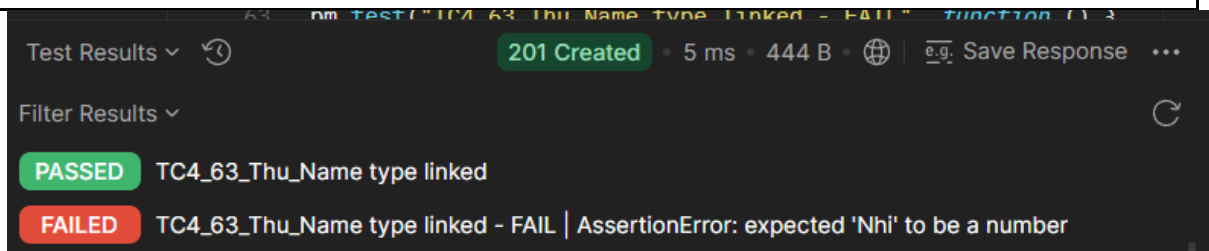


4. *TC4_63_Thu_Id exists with data*

```
// TC4: Kiểm tra id tồn tại (gắn với dữ liệu vừa tạo)
pm.test("TC4_63_Thu_Id exists with data", function () {
  let data = pm.response.json();

  pm.expect(data.name).to.eql("Nhi");
  pm.expect(data).to.have.property("id");
});
// TC4: Kiểm tra name là string (gắn với dữ liệu đúng)
pm.test("TC4_63_Thu_Name type linked", function () {
  let data = pm.response.json();

  pm.expect(data.name).to.eql("Nhi");
  pm.expect(data.name).to.be.a("number");
});
```



5. *TC5_63_Thu_Email format linked*

```
//TC5: Kiểm tra email hợp lệ và đúng với name
pm.test("TC5_63_Thu_Email format linked", function () {
  let data = pm.response.json();

  pm.expect(data.name).to.eql("Nhi");
  pm.expect(data.email).to.include("@");
  pm.expect(data.email).to.eql("nhi@gmail.com");
});
//TC5: Kiểm tra email hợp lệ và đúng với name (FAIL)
pm.test("TC5_63_Thu_Email format linked - FAIL", function () {
  let data = pm.response.json();

  pm.expect(data.name).to.eql("Nhi");
  pm.expect(data.email).to.include("#"); // cố tình sai
```

```
pm.expect(data.email).to.eql("nhi@yahoo.com"); // cố tình sai  
});
```

PASSED TC5_63_Thu_Email format linked

FAILED TC5_63_Thu_Email format linked - FAIL | AssertionError: expected 'nhi@gmail.com' to include '#'

10.3 Phương thức PUT

Dữ liệu PUT

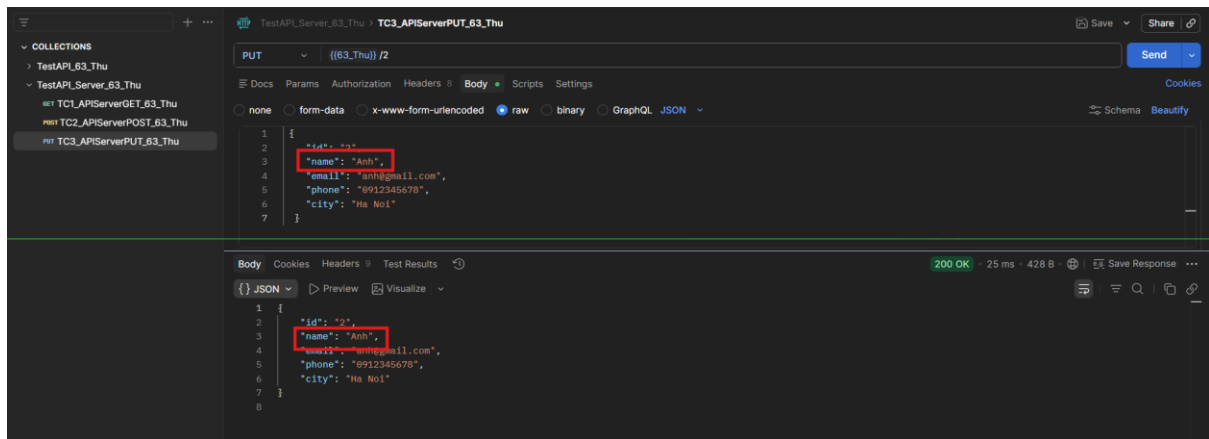
Trước khi Put dữ liệu

The top part of the image shows a web browser window with the address bar displaying `localhost:3000/TestAPI_63_Thu`. The main content area shows a JSON object:

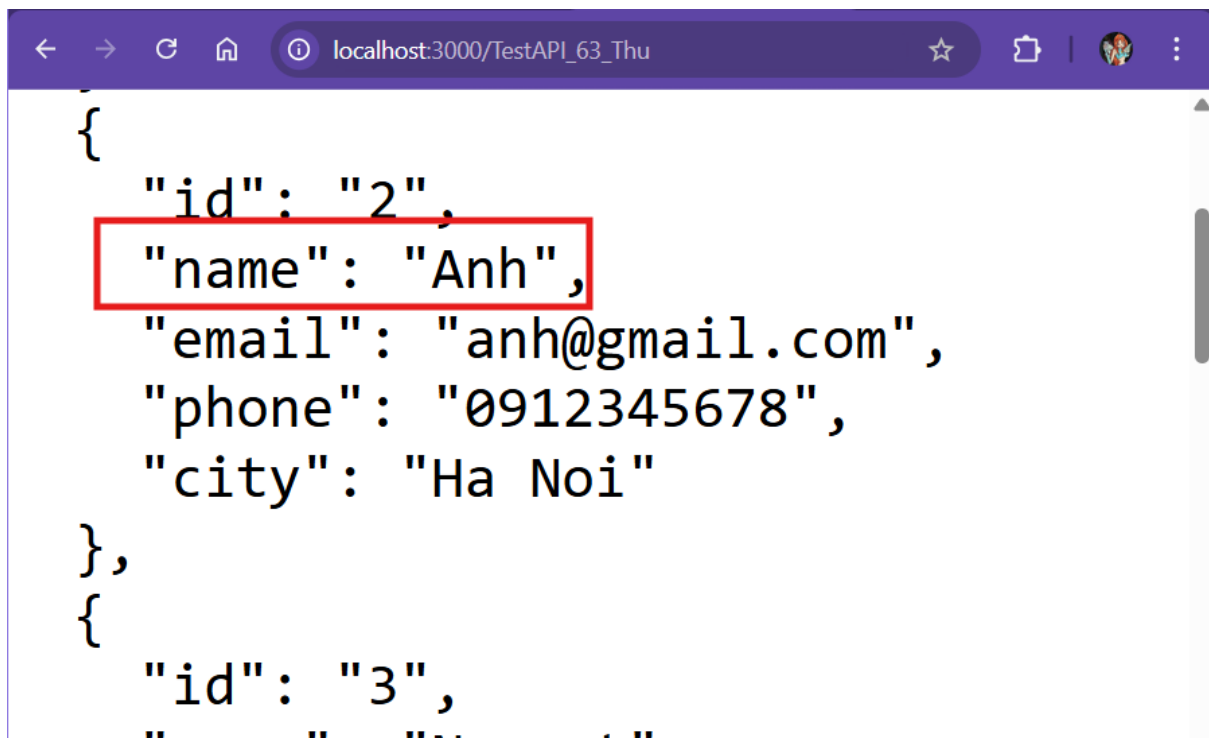
```
{  
  "id": "2",  
  "name": "An",  
  "email": "anh@gmail.com",  
  "phone": "0912345678",  
  "city": "Ha Noi"  
},
```

The bottom part of the image shows the Postman interface. The left sidebar shows a collection named `TestAPI_63_Thu` with a sub-collection `TestAPI_Server_63_Thu`. Inside this sub-collection, there are three items: `TC1_APIServerGET_63_Thu`, `TC2_APIServerPOST_63_Thu`, and `TC3_APIServerPUT_63_Thu`. The `TC3_APIServerPUT_63_Thu` item is selected and highlighted with a red box. The main panel shows the details of this PUT request, including the method `PUT`, the URL `localhost:3000/TestAPI_63_Thu`, and the request body.

Đổi trường name “An” thành “Anh”



Dữ liệu sau khi Put



1. TC1_63_Thu_Check name

```
// TC1: Kiểm tra name đã update đúng
pm.test("TC1_63_Thu_Check name", function () {
  let data = pm.response.json();
  pm.expect(data.name).to.eql("Anh");
});
pm.test("TC1_63_Thu_Check name - FAIL", function () {
  let data = pm.response.json();
  pm.expect(data.name).to.eql("An");
});
```

2. TC2_63_Thu_Name_Email linked

```
// TC2: Kiểm tra email đúng với name = Anh
pm.test("TC2_63_Thu_Name_Email linked", function () {
  let data = pm.response.json();

  pm.expect(data.name).to.eql("Anh");
  pm.expect(data.email).to.eql("anh@gmail.com");
});
pm.test("TC2_63_Thu_Name_Email linked - FAIL", function () {
  let data = pm.response.json();

  pm.expect(data.name).to.eql("Anh");
  pm.expect(data.email).to.eql("an@gmail.com");
});
```

3. TC3_63_Thu_Name_City linked

```
// TC3: Kiểm tra city đúng với name
pm.test("TC3_63_Thu_Name_City linked", function () {
  let data = pm.response.json();

  pm.expect(data.name).to.eql("Anh");
  pm.expect(data.city).to.eql("Ha Noi");
});
pm.test("TC3_63_Thu_Name_City linked - FAIL", function () {
  let data = pm.response.json();

  pm.expect(data.name).to.eql("Anh");
  pm.expect(data.city).to.eql("Hue");
});
```

4. TC4_63_Thu_Name_Phone linked

```
// TC4: Kiểm tra phone đúng với name
pm.test("TC4_63_Thu_Name_Phone linked", function () {
  let data = pm.response.json();

  pm.expect(data.name).to.eql("Anh");
  pm.expect(data.phone).to.eql("0912345678");
});
pm.test("TC4_63_Thu_Name_Phone linked - FAIL", function () {
  let data = pm.response.json();

  pm.expect(data.name).to.eql("Anh");
```

```
pm.expect(data.phone).to.eql("0000000000");
});
```

5. TC5_63_Thu_Id check

```
// TC5: Kiểm tra id đúng record đã update
pm.test("TC5_63_Thu_Id check", function () {
  let data = pm.response.json();

  pm.expect(data.name).to.eql("Anh");
  pm.expect(data.id).to.eql("2");
});

pm.test("TC5_63_Thu_Id check - FAIL", function () {
  let data = pm.response.json();

  pm.expect(data.name).to.eql("Anh");
  pm.expect(data.id).to.eql("1");
});
```

6. TC6_63_Thu_Email format

```
// TC6: Kiểm tra email format
pm.test("TC6_63_Thu_Email format", function () {
  let data = pm.response.json();

  pm.expect(data.email).to.include("@");
});

pm.test("TC6_63_Thu_Email format - FAIL", function () {
  let data = pm.response.json();

  pm.expect(data.email).to.include("#");
});
```

7. TC7_63_Thu_Phone length

```
// TC7: Kiểm tra phone có 10 số
pm.test("TC7_63_Thu_Phone length", function () {
  let data = pm.response.json();

  pm.expect(data.phone.length).to.eql(10);
});

pm.test("TC7_63_Thu_Phone length - FAIL", function () {
  let data = pm.response.json();
```

```
pm.expect(data.phone.length).to.eql(5);
});
```

8. TC8_63_Thu_Full data linked

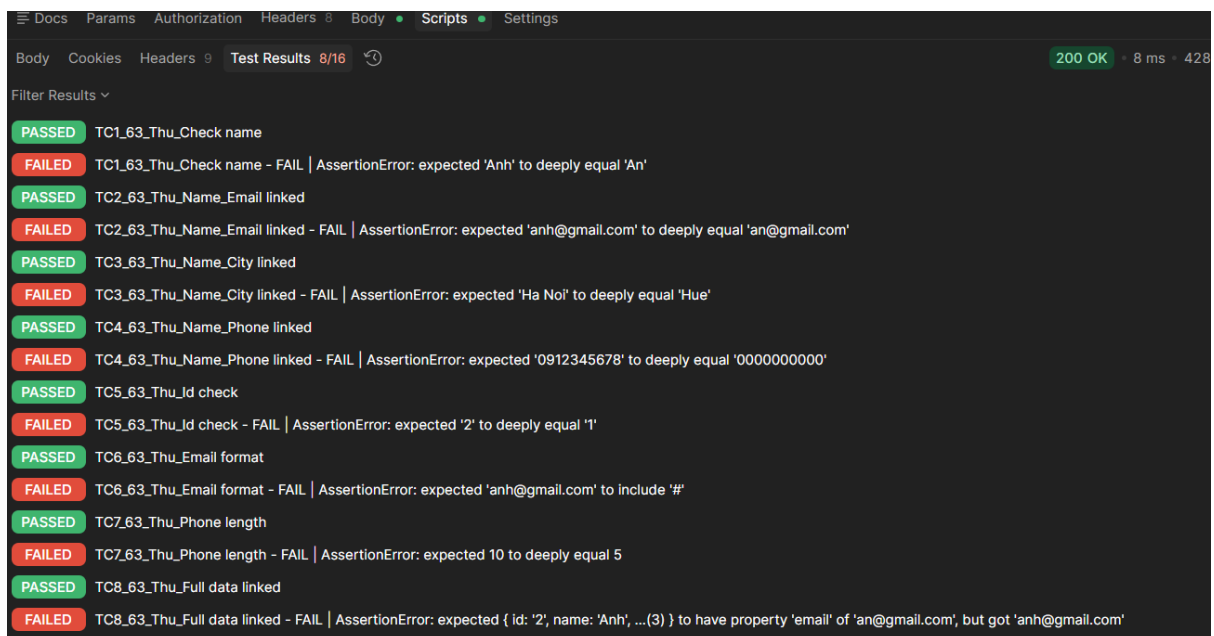
```
// TC8: Kiểm tra full dữ liệu sau khi update
pm.test("TC8_63_Thu_Full data linked", function () {
  let data = pm.response.json();

  pm.expect(data).to.include({
    name: "Anh",
    email: "anh@gmail.com",
    phone: "0912345678",
    city: "Ha Noi"
  });
});

pm.test("TC8_63_Thu_Full data linked - FAIL", function () {
  let data = pm.response.json();

  pm.expect(data).to.include({
    name: "Anh",
    email: "an@gmail.com",
    phone: "0000000000",
    city: "Hue"
  });
});
```

Kết quả test



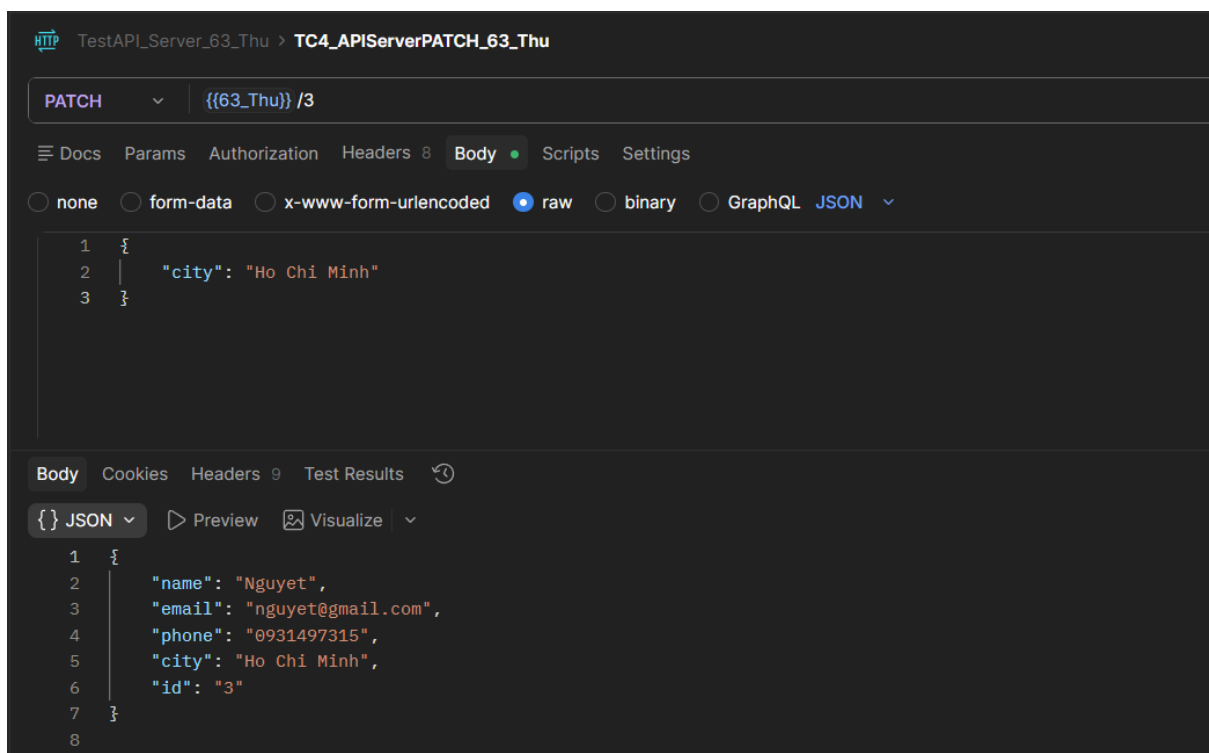
10.4 Phương thức PATCH

Dữ liệu PATCH

Dữ liệu trước khi Patch



```
{
  "id": "3",
  "name": "Nguyet",
  "email": "nguyet@gmail.com",
  "phone": "0931497315",
  "city": "Da Nang"
},
```



```
TestAPI_Server_63_Thu > TC4_APIServerPATCH_63_Thu
PATCH {{63_Thu}} /3
Body
none form-data x-www-form-urlencoded raw binary GraphQL JSON
1 {
2   "city": "Ho Chi Minh"
3 }
Body Cookies Headers 9 Test Results
{} JSON Preview Visualize
1 {
2   "name": "Nguyet",
3   "email": "nguyet@gmail.com",
4   "phone": "0931497315",
5   "city": "Ho Chi Minh",
6   "id": "3"
7 }
8
```

```
{
  "city": "Ho Chi Minh"
}
```

Dữ liệu sau khi Patch

```
{  
  "name": "Nguyet",  
  "email": "nguyet@gmail.com",  
  "phone": "0931497315",  
  "city": "Ho Chi Minh",  
  "id": "3"  
},
```

1. TC1_63_Thu_Status code is 200

```
// TC1: Kiểm tra status code 200  
pm.test("TC1_63_Thu_Status code is 200", function () {  
  pm.response.to.have.status(200);  
});  
pm.test("TC1_63_Thu_Status code is 200 - FAIL", function () {  
  pm.response.to.have.status(404);  
});
```

2. TC2_63_Thu_Check city updated

```
// TC2: Kiểm tra city đã update  
pm.test("TC2_63_Thu_Check city updated", function () {  
  let data = pm.response.json();  
  pm.expect(data.city).to.eql("Ho Chi Minh");  
});  
  
pm.test("TC2_63_Thu_Check city updated - FAIL", function () {  
  let data = pm.response.json();  
  pm.expect(data.city).to.eql("Ha Noi");  
});
```

3. TC3_63_Thu_Name unchanged

```
// TC3: Kiểm tra name KHÔNG bị thay đổi  
pm.test("TC3_63_Thu_Name unchanged", function () {  
  let data = pm.response.json();  
  
  pm.expect(data.name).to.eql("Nguyet");  
});
```

```
});  
  
pm.test("TC3_63_Thu_Name unchanged - FAIL", function () {  
    let data = pm.response.json();  
  
    pm.expect(data.name).to.eql("Thu");  
});
```

4. TC4_63_Thu_Email unchanged

```
// TC4: Kiểm tra email KHÔNG thay đổi  
pm.test("TC4_63_Thu_Email unchanged", function () {  
    let data = pm.response.json();  
  
    pm.expect(data.email).to.eql("nguyet@gmail.com");  
});  
  
pm.test("TC4_63_Thu_Email unchanged - FAIL", function () {  
    let data = pm.response.json();  
  
    pm.expect(data.email).to.eql("mai@gmail.com");  
});
```

5. TC5_63_Thu_Phone unchanged

```
// TC5: Kiểm tra phone KHÔNG thay đổi  
pm.test("TC5_63_Thu_Phone unchanged", function () {  
    let data = pm.response.json();  
  
    pm.expect(data.phone).to.eql("0931497315");  
});  
  
pm.test("TC5_63_Thu_Phone unchanged - FAIL", function () {  
    let data = pm.response.json();  
  
    pm.expect(data.phone).to.eql("0000000000");  
});
```

6. TC6_63_Thu_Id check

```
// TC6: Kiểm tra id đúng record  
pm.test("TC6_63_Thu_Id check", function () {  
    let data = pm.response.json();  
  
    pm.expect(data.id).to.eql("3");  
});
```

```
pm.test("TC6_63_Thu_Id check - FAIL", function () {  
  let data = pm.response.json();  
  
  pm.expect(data.id).to.eql("999");  
});
```

7. TC7_63_Thu_Only city updated

```
// TC7: Kiểm tra chỉ city bị thay đổi (logic PATCH)  
pm.test("TC7_63_Thu_Only city updated", function () {  
  let data = pm.response.json();  
  
  pm.expect(data).to.include({  
    name: "Nguyet",  
    email: "nguyet@gmail.com",  
    phone: "0931497315",  
    city: "Ho Chi Minh"  
  });  
});  
  
pm.test("TC7_63_Thu_Only city updated - FAIL", function () {  
  let data = pm.response.json();  
  
  pm.expect(data).to.include({  
    name: "Nguyet",  
    email: "mai@gmail.com", // sai → fail  
    phone: "0000000000",  
    city: "Hue"  
  });  
});
```

8. TC8_63_Thu_Email format

```
// TC8: Kiểm tra email format  
pm.test("TC8_63_Thu_Email format", function () {  
  let data = pm.response.json();  
  
  pm.expect(data.email).to.include("@");  
});  
  
pm.test("TC8_63_Thu_Email format - FAIL", function () {  
  let data = pm.response.json();  
  
  pm.expect(data.email).to.include("#");  
});
```

Body	Cookies	Headers	9	Test Results	8/16	200 OK	6 ms	4
Filter Results								
PASSED	TC1_63_Thu_Status code is 200							
FAILED	TC1_63_Thu_Status code is 200 - FAIL AssertionError: expected response to have status code 404 but got 200							
PASSED	TC2_63_Thu_Check city updated							
FAILED	TC2_63_Thu_Check city updated - FAIL AssertionError: expected 'Ho Chi Minh' to deeply equal 'Ha Noi'							
PASSED	TC3_63_Thu_Name unchanged							
FAILED	TC3_63_Thu_Name unchanged - FAIL AssertionError: expected 'Nguyet' to deeply equal 'Thu'							
PASSED	TC4_63_Thu_Email unchanged							
FAILED	TC4_63_Thu_Email unchanged - FAIL AssertionError: expected 'nguyet@gmail.com' to deeply equal 'mai@gmail.com'							
PASSED	TC5_63_Thu_Phone unchanged							
FAILED	TC5_63_Thu_Phone unchanged - FAIL AssertionError: expected '0931497315' to deeply equal '0000000000'							
PASSED	TC6_63_Thu_Id check							
FAILED	TC6_63_Thu_Id check - FAIL AssertionError: expected '3' to deeply equal '999'							
PASSED	TC7_63_Thu_Only city updated							
FAILED	TC7_63_Thu_Only city updated - FAIL AssertionError: expected { name: 'Nguyet', ... } to have property 'email' of 'mai@gmail.com', but got 'nguyet@gmail.com'							
PASSED	TC8_63_Thu_Email format							
FAILED	TC8_63_Thu_Email format - FAIL AssertionError: expected 'nguyet@gmail.com' to include '#'							

4.5 Phương thức DELETE

Dữ liệu DELETE

Chọn id “YU3K0cngsY”

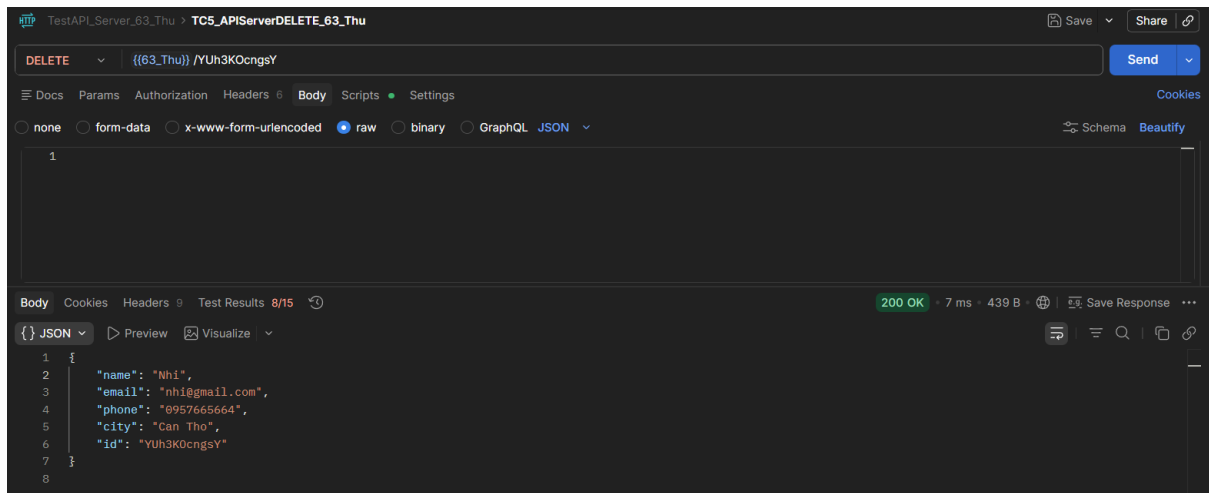
The screenshot shows a Postman interface for a DELETE API call. The collection is 'TestAPI_Server_63_Thu' and the specific request is 'TC5_APIServerDELETE_63_Thu'. The method is set to 'DELETE'. The URL is 'localhost:3000/TestAPI_63_Thu'. The response is a JSON object with the following structure:

```

{
  "id": "9BdnU1TV8vc",
  },
  {
    "name": "Nhi",
    "email": "nhi@gmail.com",
    "phone": "0957665664",
    "city": "Can Tho",
    "id": "YUh3K0cngsY"
  },
  },
  ,
  ,

```

The 'id' field in the second object is highlighted with a red box, matching the value 'YU3K0cngsY' mentioned in the text above.



1. TC1_63_Thu_Status code is 200

```
// TC1: Kiểm tra status code 200
pm.test("TC1_63_Thu_Status code is 200", function () {
  pm.response.to.have.status(200);
});

pm.test("TC1_63_Thu_Status code is 200 - FAIL", function () {
  pm.response.to.have.status(404);
});
```

2. TC2_63_Thu_Check deleted id

```
// TC2: Kiểm tra response có id đã xóa
pm.test("TC2_63_Thu_Check deleted id", function () {
  let data = pm.response.json();

  pm.expect(data.id).to.eql("YUh3KOcngsY");
});

pm.test("TC2_63_Thu_Check deleted id - FAIL", function () {
  let data = pm.response.json();

  pm.expect(data.id).to.eql("999");
});
```

3. TC3_63_Thu_Check deleted name

```
// TC3: Kiểm tra name của dữ liệu đã xóa
pm.test("TC3_63_Thu_Check deleted name", function () {
  let data = pm.response.json();

  pm.expect(data.name).to.eql("Nhi");
});
```

```
});

pm.test("TC3_63_Thu_Check deleted name - FAIL", function () {
  let data = pm.response.json();

  pm.expect(data.name).to.eql("SaiTen");
});
```

4. TC4_63_Thu_Deleted data snapshot

```
// TC4: Kiểm tra dữ liệu trả về đầy đủ trước khi xóa
pm.test("TC4_63_Thu_Deleted data snapshot", function () {
  let data = pm.response.json();

  pm.expect(data).to.include({
    name: "Nhi",
    email: "nhi@gmail.com",
    phone: "0957665664",
    city: "Can Tho"
  });
});

pm.test("TC4_63_Thu_Deleted data snapshot - FAIL", function () {
  let data = pm.response.json();

  pm.expect(data).to.include({
    name: "Nhi",
    email: "sai@gmail.com",
    phone: "0000000000",
    city: "SaiCity"
  });
});
```

5. TC5_63_Thu_Email format

```
// TC5: Kiểm tra email format
pm.test("TC5_63_Thu_Email format", function () {
  let data = pm.response.json();

  pm.expect(data.email).to.include("@");
});

pm.test("TC5_63_Thu_Email format - FAIL", function () {
  let data = pm.response.json();

  pm.expect(data.email).to.include("#");
});
```

6. TC6_63_Thu_Phone length

```
// TC6: Kiểm tra phone length
pm.test("TC6_63_Thu_Phone length", function () {
  let data = pm.response.json();

  pm.expect(data.phone.length).to.eql(10);
});

pm.test("TC6_63_Thu_Phone length - FAIL", function () {
  let data = pm.response.json();

  pm.expect(data.phone.length).to.eql(5);
});
```

7. TC7_63_Thu_All fields exist

```
// TC7: Kiểm tra object có đầy đủ field
pm.test("TC7_63_Thu_All fields exist", function () {
  let data = pm.response.json();

  pm.expect(data).to.have.all.keys(
    "id", "name", "email", "phone", "city"
  );
});

pm.test("TC7_63_Thu_All fields exist - FAIL", function () {
  let data = pm.response.json();

  pm.expect(data).to.have.all.keys(
    "id", "name", "email"
  );
});
```

8. TC8_63_Thu_Delete logic check

```
// TC8: Kiểm tra logic xóa
pm.test("TC8_63_Thu_Delete logic check", function () {
  let data = pm.response.json();

  pm.expect(data.id).to.exist;
});
```


Kết quả test

Body	Cookies	Headers	9	Test Results	8/15	🔄	200 OK	7 ms	439 B
Filter Results ▾									
PASSED	TC1_63_Thu_Status code is 200								
FAILED	TC1_63_Thu_Status code is 200 - FAIL AssertionError: expected response to have status code 404 but got 200								
PASSED	TC2_63_Thu_Check deleted id								
FAILED	TC2_63_Thu_Check deleted id - FAIL AssertionError: expected 'YUh3KOcnsgsY' to deeply equal '999'								
PASSED	TC3_63_Thu_Check deleted name								
FAILED	TC3_63_Thu_Check deleted name - FAIL AssertionError: expected 'Nhi' to deeply equal 'SaiTen'								
PASSED	TC4_63_Thu_Deleted data snapshot								
FAILED	TC4_63_Thu_Deleted data snapshot - FAIL AssertionError: expected { name: 'Nhi', ...(4) } to have property 'email' of 'sai@gmail.com', but got 'nhi@gmail.com'								
PASSED	TC5_63_Thu_Email format								
FAILED	TC5_63_Thu_Email format - FAIL AssertionError: expected 'nhi@gmail.com' to include '#'								
PASSED	TC6_63_Thu_Phone length								
FAILED	TC6_63_Thu_Phone length - FAIL AssertionError: expected 10 to deeply equal 5								
PASSED	TC7_63_Thu_All fields exist								
FAILED	TC7_63_Thu_All fields exist - FAIL AssertionError: expected { name: 'Nhi', ...(4) } to have keys 'id', 'name', and 'email'								
PASSED	TC8_63_Thu_Delete logic check								

Lưu ý:

TestCase chỉ thực hiện được 1 lần duy nhất. Chọn id sau đó viết TestCase và Send. Trong trường hợp vẫn giữ id cũ và Send lần nữa thì sẽ hiển thị kết quả như bên dưới.

Body	Cookies	Headers	9	Test Results	1/15	🔄	404 Not Found	4 ms	355 B	🌐	📄 Save Response	⋮
Filter Results ▾												
FAILED	TC1_63_Thu_Status code is 200 AssertionError: expected response to have status code 200 but got 404											
PASSED	TC1_63_Thu_Status code is 200 - FAIL											
FAILED	TC2_63_Thu_Check deleted id AssertionError: expected undefined to deeply equal 'YUh3KOcnsgsY'											
FAILED	TC2_63_Thu_Check deleted id - FAIL AssertionError: expected undefined to deeply equal '999'											
FAILED	TC3_63_Thu_Check deleted name AssertionError: expected undefined to deeply equal 'Nhi'											
FAILED	TC3_63_Thu_Check deleted name - FAIL AssertionError: expected undefined to deeply equal 'SaiTen'											
FAILED	TC4_63_Thu_Deleted data snapshot AssertionError: expected { error: 'Not Found' } to have property 'name'											
FAILED	TC4_63_Thu_Deleted data snapshot - FAIL AssertionError: expected { error: 'Not Found' } to have property 'name'											
FAILED	TC5_63_Thu_Email format AssertionError: the given combination of arguments (undefined and string) is invalid for this assertion. You can use an array, a map, an object, a set, a string, or a weakset instead											
FAILED	TC5_63_Thu_Email format - FAIL AssertionError: the given combination of arguments (undefined and string) is invalid for this assertion. You can use an array, a map, an object, a set, a string, or a weakset instead											
FAILED	TC6_63_Thu_Phone length TypeError: Cannot read properties of undefined (reading 'length') 🔄 Fix test											
FAILED	TC6_63_Thu_Phone length - FAIL TypeError: Cannot read properties of undefined (reading 'length') 🔄 Fix test											
FAILED	TC7_63_Thu_All fields exist AssertionError: expected { error: 'Not Found' } to have keys 'id', 'name', 'email', 'phone', and 'city'											
FAILED	TC7_63_Thu_All fields exist - FAIL AssertionError: expected { error: 'Not Found' } to have keys 'id', 'name', and 'email'											
FAILED	TC8_63_Thu_Delete logic check AssertionError: expected undefined to exist											